

SURICATA RULES

8.1 Rules Format

Signatures play a very important role in Suricata. In most occasions people are using existing rulesets.

The official way to install rulesets is described in *Rule Management with Suricata-Update*.

There are a number of free rulesets that can be used via `suricata-update`. To aid in learning about writing rules, the Emerging Threats Open ruleset is free and a good reference that has a wide range of signature examples.

This Suricata Rules document explains all about signatures; how to read, adjust and create them.

A rule/signature consists of the following:

- The **action**, determining what happens when the rule matches.
- The **header**, defining the protocol, IP addresses, ports and direction of the rule.
- The **rule options**, defining the specifics of the rule.

An example of a rule is as follows:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";  
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;  
sid:123; rev:1;)
```

In this example, red is the action, green is the header and blue are the options.

We will be using the above signature as an example throughout this section, highlighting the different parts of the signature.

8.1.1 Action

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";  
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;  
sid:123; rev:1;)
```

Valid actions are:

- `alert` - generate an alert.
- `pass` - stop further inspection of the packet.
- `drop` - drop packet and generate alert.
- `reject` - send RST/ICMP unreachable error to the sender of the matching packet.
- `rejectsrc` - same as just *reject*.
- `rejectdst` - send RST/ICMP error packet to receiver of the matching packet.

- rejectboth - send RST/ICMP error packets to both sides of the conversation.

Note

In IPS mode, using any of the *reject* actions also enables *drop*.

For more information see [Action-order](#).

8.1.2 Protocol

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";  
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;  
sid:123; rev:1;)
```

This keyword in a signature tells Suricata which protocol it concerns. You can choose between four basic protocols:

- tcp (for tcp-traffic)
- udp
- icmp
- ip (ip stands for 'all' or 'any')

There are a couple of additional TCP related protocol options:

- tcp-pkt (for matching content in individual tcp packets)
- tcp-stream (for matching content only in a reassembled tcp stream)

There are also a few so-called application layer protocols, or layer 7 protocols you can pick from. These are:

- http (either HTTP1 or HTTP2)
- http1
- http2
- ftp
- tls (this includes ssl)
- smb
- dns
- dcerpc
- dhcp
- ssh
- smtp
- imap
- pop3
- modbus (disabled by default)
- dnp3 (disabled by default)
- enip (disabled by default)
- nfs
- ike

- krb5
- bittorrent-dht
- ntp
- dhcp
- rfb
- rdp
- snmp
- tftp
- sip
- websocket

The availability of these protocols depends on whether the protocol is enabled in the configuration file, `suricata.yaml`.

If you have a signature with the protocol declared as 'http', Suricata makes sure the signature will only match if the TCP stream contains http traffic.

Explicit rule hooks

In Suricata 8 the protocol field can be used to force evaluation of a rule at a specific explicit protocol state. This takes the format of:

```
<proto>:<hook>
```

Where each application protocol comes with a default set of hooks, as well as per protocol specific hooks.

More details can be found in [Explicit rule hook \(states\)](#).

Note

While developed for the firewall usecase, these hooks can be used in IDS/IPS rules as well.

8.1.3 Source and destination

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;
sid:123; rev:1;)
```

The first emphasized part is the traffic source, the second is the traffic destination (note the direction of the directional arrow).

With the source and destination, you specify the source of the traffic and the destination of the traffic, respectively. You can assign IP addresses, (both IPv4 and IPv6 are supported) and IP ranges. These can be combined with operators:

Operator	Description
../..	IP ranges (CIDR notation)
!	exception/negation
[.., ..]	grouping

Normally, you would also make use of variables, such as `$HOME_NET` and `$EXTERNAL_NET`. The `suricata.yaml` configuration file specifies the IP addresses these concern. The respective `$HOME_NET` and `$EXTERNAL_NET` settings will be used in place of the variables in your rules.

See [Rule-vars](#) for more information.

Rule usage examples:

Example	Meaning
<code>!1.1.1.1</code>	Every IP address but 1.1.1.1
<code>![1.1.1.1, 1.1.1.2]</code>	Every IP address but 1.1.1.1 and 1.1.1.2
<code>\$HOME_NET</code>	Your setting of HOME_NET in yaml
<code>[\$EXTERNAL_NET, !\$HOME_NET]</code>	EXTERNAL_NET and not HOME_NET
<code>[10.0.0.0/24, !10.0.0.5]</code>	10.0.0.0/24 except for 10.0.0.5
<code>[..., [...]]</code>	
<code>[..., ![...]]</code>	

Warning

If you set your configuration to something like this:

```
HOME_NET: any
EXTERNAL_NET: !$HOME_NET
```

You cannot write a signature using `$EXTERNAL_NET` because it evaluates to 'not any', which is an invalid value.

Note

Please note that the source and destination address can also be matched via the `ip.src` and `ip.dst` keywords (See [IP Addresses Match](#)). These keywords are mostly used in conjunction with the dataset feature ([Datasets](#)).

8.1.4 Ports (source and destination)

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;
sid:123; rev:1;)
```

The first emphasized part is the source port, the second is the destination port (note the direction of the directional arrow).

Traffic comes in and goes out through ports. Different protocols have different port numbers. For example, the default port for HTTP is 80 while 443 is typically the port for HTTPS. Note, however, that the port does not dictate which protocol is used in the communication. Rather, it determines which application is receiving the data.

The ports mentioned above are typically the destination ports. Source ports, i.e. the application that sent the packet, typically get assigned a random port by the operating system. When writing a rule for your own HTTP service, you would typically write `any -> 80`, since that would mean any packet from any source port to your HTTP application (running on port 80) is matched.

In setting ports you can make use of special operators as well. Operators such as:

Operator	Description
:	port ranges
!	exception/negation
[..., ..]	grouping

Rule usage examples:

Example	Meaning
[80, 81, 82]	port 80, 81 and 82
[80: 82]	Range from 80 till 82
[1024:]	From 1024 till the highest port-number
!80	Every port but 80
[80:100,!99]	Range from 80 till 100 but 99 excluded
[1:80,!2,4]	Range from 1-80, except ports 2 and 4
[..., [...]]	

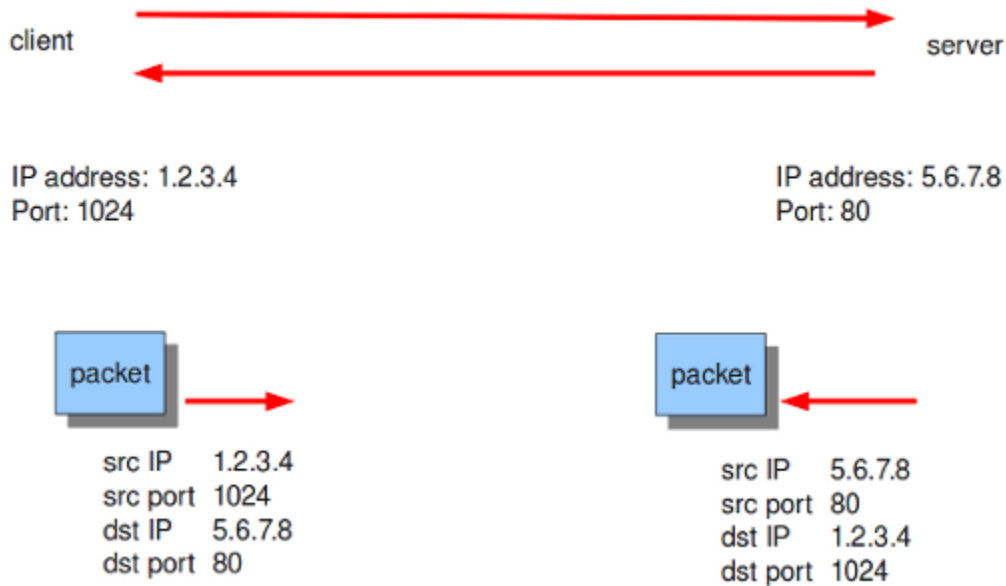
8.1.5 Direction

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;
sid:123; rev:1;)
```

The directional arrow indicates which way the signature will be evaluated. In most signatures an arrow to the right (->) is used. This means that only packets with the same direction can match. There is also the double arrow (=>), which respects the directionality as ->, but allows matching on bidirectional transactions, used with keywords matching each direction. Finally, it is also possible to have a rule match either directions (<>):

```
source -> destination
source => destination
source <> destination (either directions)
```

The following example illustrates direction. In this example there is a client with IP address 1.2.3.4 using port 1024. A server with IP address 5.6.7.8, listening on port 80 (typically HTTP). The client sends a message to the server and the server replies with its answer.



Now, let's say we have a rule with the following header:

```
alert tcp 1.2.3.4 1024 -> 5.6.7.8 80
```

Only the traffic from the client to the server will be matched by this rule, as the direction specifies that we do not want to evaluate the response packet.

Now, if we have a rule with the following header:

```
alert tcp 1.2.3.4 any <> 5.6.7.8 80
```

Suricata will duplicate it and use the same rule with headers in both directions :

```
alert tcp 1.2.3.4 any -> 5.6.7.8 80 alert tcp 5.6.7.8 80 -> 1.2.3.4 any
```

Warning

There is no 'reverse' style direction, i.e. there is no <-.

Transactional rules

Here is an example of a transactional rule:

```
alert http any any => 5.6.7.8 80 (msg:"matching both uri and status"; sid: 1; http.uri; content: "/download"; http.stat_code; content: "200");
```

It will match on flows to 5.6.7.8 and port 80. And it will match on a full transaction, using both the uri from the request, and the stat_code from the response. As such, it will match only when Suricata got both request and response.

Transactional rules can use direction-ambiguous keywords, by specifying the direction.

```
alert http any any => 5.6.7.8 80 (msg:"matching json to server and xml to client"; sid: 1; http.content_type: to_server; content: "json"; http.content_type: to_client; content: "xml");
```

Transactional rules have some limitations :

- They cannot use direction-ambiguous keywords
- They are only meant to work on transactions with first a request to the server, and then a response to the client, and not the other way around (not tested).
- They cannot have `fast_pattern` or `prefilter` the direction to client if they also have a streaming buffer on the direction to server, see example below.
- They will refuse to load if a single directional rule is enough.

This rule cannot have the `fast_pattern` to client, as `file.data` is a streaming buffer and will refuse to load.

```
alert http any any => any any (file.data: to_server; content: "123"; http.stat_code; content: "500"; fast_patten;)
```

If not explicit, a transactional rule will choose a `fast_pattern` to server by default

8.1.6 Rule options

The rest of the rule consists of options. These are enclosed by parenthesis and separated by semicolons. Some options have settings (such as `msg`), which are specified by the keyword of the option, followed by a colon, followed by the settings. Others have no settings; they are simply the keyword (such as `nocase`):

```
<keyword>: <settings>;
<keyword>;
```

Rule options have a specific ordering and changing their order would change the meaning of the rule.

Note

The characters `;` and `"` have special meaning in the Suricata rule language and must be escaped when used in a rule option value. For example:

```
msg: "Message with semicolon\;"
```

As a consequence, you must also escape the backslash, as it functions as an escape character.

The rest of this chapter in the documentation documents the use of the various keywords.

Some generic details about keywords follow.

Disabling Alerts

There is a way to disable alert generation for a rule using the keyword `noalert`. When this keyword is part of a rule, no alert is generated if the other portions of the rule match. That is, the other rule actions will *still be applied*. Using `noalert` can be helpful when a rule is collecting or setting state using *flowbits*, *datasets* or other state maintenance constructs of the rule language. See *Thresholding Keywords* for other ways to control alert frequency.

The following rules demonstrate `noalert` with a familiar pattern:

- The first rule marks state without generating an alert.
- The second rule generates an alert if the state is set and additional qualifications are met.

```
alert http any any -> $HOME_NET any (msg:"noalert example: set state"; flow:established,to_server;
xbits:set,SC.EXAMPLE,track ip_dst, expire 10; noalert; http.method; content:"GET"; sid:1; )
```

```
alert http any any -> $HOME_NET any (msg:"noalert example: state use"; flow:established,to_server;
xbits:isset,SC.EXAMPLE,track ip_dst; http.method; content:"POST"; sid: 2; )
```

In IPS mode, `noalert` is commonly used in when Suricata should *drop* network packets without generating alerts (example below). The following rule is a simplified example showing how `noalert` could be used with IPS deployments to drop inbound SSH requests.

```
drop tcp any any -> any 22 (msg:"Drop inbound SSH traffic"; noalert; sid: 3)
```

Modifier Keywords

Some keywords function act as modifiers. There are two types of modifiers.

- The older style '**content modifiers**' look back in the rule, e.g.:

```
alert http any any -> any any (content:"index.php"; http_uri; sid:1;)
```

In the above example the pattern 'index.php' is modified to inspect the HTTP uri buffer.

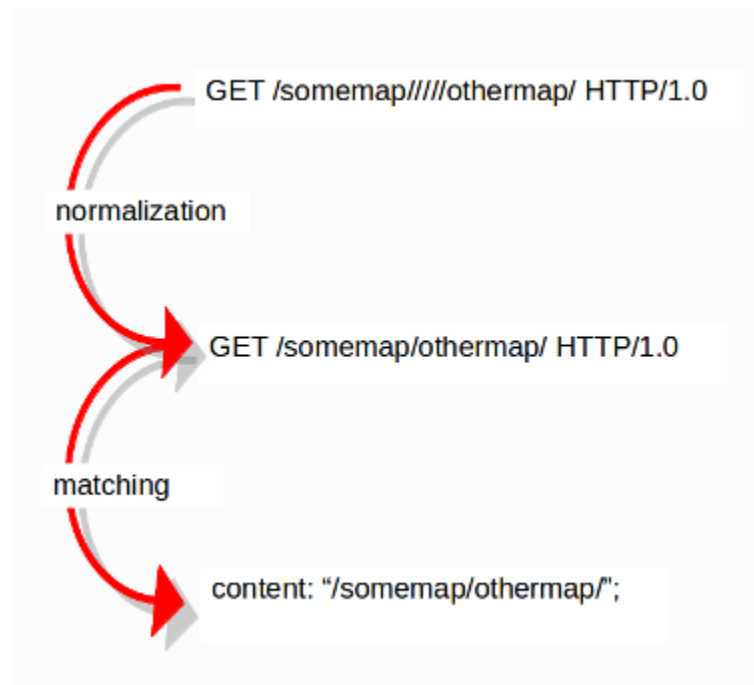
- The more recent type is called the '**sticky buffer**'. It places the buffer name first and all keywords following it apply to that buffer, for instance:

```
alert http any any -> any any (http_response_line; content:"403 Forbidden"; sid:1;)
```

In the above example the pattern '403 Forbidden' is inspected against the HTTP response line because it follows the `http_response_line` keyword.

Normalized Buffers

A packet consists of raw data. HTTP and reassembly make a copy of those kinds of packets data. They erase anomalous content, combine packets etcetera. What remains is a called the 'normalized buffer':



Because the data is being normalized, it is not what it used to be; it is an interpretation. Normalized buffers are: all HTTP-keywords, reassembled streams, TLS-, SSL-, SSH-, FTP- and dcerpc-buffers.

Note that there are some exceptions, e.g. the `http_raw_uri` keyword. See [http.uri](#) for more information.

8.2 Meta Keywords

Meta keywords have no effect on Suricata's inspection of network traffic; they do have an effect on the way Suricata reports events/alerts.

8.2.1 msg (message)

The keyword msg gives contextual information about the signature and the possible alert.

The format of msg is:

```
msg: "some description";
```

Examples:

```
msg: "ET MALWARE Win32/RecordBreaker CnC Checkin";
msg: "ET EXPLOIT SMB-DS DCERPC PnP bind attempt";
```

To continue the example from the previous chapter, the msg component of the signature is emphasized below:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;
sid:123; rev:1;)
```

Tip

It is a standard practice in rule writing to make the first part of the signature msg uppercase and to indicate the class of the signature.

It is also standard practice that msg is the first keyword in the signature.

Note

The following characters must be escaped inside the msg: ; \ "

8.2.2 sid (signature ID)

The keyword sid gives every signature its own id. This id is stated with a number greater than zero. The format of sid is:

```
sid:123;
```

Example of sid in a signature:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;
sid:123; rev:1;)
```

Tip

It is a standard practice in rule writing that the signature sid is provided as the last keyword (or second-to-last if there is a rev) of the signature.

There are reserved ranges of sids, the reservations are recorded at <https://sidallocation.org/>.

Note

This value must be unique for all rules within the same *rule group* (gid).

As Suricata-update currently considers the rule's sid only (cf. [Bug#5447](#)), it is advisable to opt for a completely unique sid altogether.

8.2.3 rev (revision)

The sid keyword is commonly accompanied by the rev keyword. Rev represents the version of the signature. If a signature is modified, the number of rev will be incremented by the signature writers. The format of rev is:

```
rev:123;
```

Example of rev in a signature:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;
sid:123; rev:1;)
```

Tip

It is a standard practice in rule writing that the rev keyword is expressed after the sid keyword. The sid and rev keywords are commonly put as the last two keywords in a signature.

8.2.4 gid (group ID)

The gid keyword can be used to give different groups of signatures another id value (like in sid). Suricata by default uses gid 1. It is possible to modify the default value. In most cases, it will be unnecessary to change the default gid value. Changing the gid value has no technical implications, the value is only noted in alert data.

Example of the gid value in an alert entry in the fast.log file. In the part [1:123], the first 1 is the gid (123 is the sid and 1 is the rev).

```
07/12/2022-21:59:26.713297 [**] [1:123:1] HTTP GET Request Containing Rule in URI [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.225.121:12407 -> 172.16.105.84:80
```

8.2.5 classtype

The classtype keyword gives information about the classification of rules and alerts. It consists of a short name, a long name and a priority. It can tell for example whether a rule is just informational or is about a CVE. For each classtype, the classification.config has a priority that will be used in the rule.

Example classtype definition:

```
config classification: web-application-attack,Web Application Attack,1
config classification: not-suspicious,Not Suspicious Traffic,3
```

Once we have defined the classification in the configuration file, we can use the classtypes in our rules. A rule with classtype web-application-attack will be assigned a priority of 1 and the alert will contain 'Web Application Attack' in the Suricata logs:

classtype	Alert	Priority
web-application-attack	Web Application Attack	1
not-suspicious	Not Suspicious Traffic	3

Our continuing example also has a classtype: bad-unknown:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method; content:"GET"; http.uri; content:"rule"; fast_pattern; classtype:bad-unknown;
sid:123; rev:1;)
```

Tip

It is a standard practice in rule writing that the classtype keyword comes before the sid and rev keywords (as shown in the example rule).

8.2.6 reference

The reference keyword is used to document where information about the signature and about the problem the signature tries to address can be found. The reference keyword can appear multiple times in a signature. This keyword is meant for signature-writers and analysts who investigate why a signature has matched. It has the following format:

```
reference: type, reference
```

A typical reference to www.info.com would be:

```
reference:url,www.info.com
```

There are several systems that can be used as a reference. A commonly known example is the CVE-database, which assigns numbers to vulnerabilities, to prevent having to type the same URL over and over again. An example reference of a CVE:

```
reference:cve,CVE-2014-1234
```

This would make a reference to <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-1234>.

All the reference types are defined in the reference.config configuration file.

8.2.7 priority

The priority keyword comes with a mandatory numeric value which can range from 1 to 255. The values 1 through 4 are commonly used. The highest priority is 1. Signatures with a higher priority will be examined first. Normally signatures have a priority determined through a classtype definition. The classtype definition can be overridden by defining the priority keyword in the signature. The format of priority is:

```
priority:1;
```

8.2.8 metadata

The metadata keyword allows additional, non-functional, information to be added to the signature. While the format is free-form, it is recommended to stick to *[key, value]* pairs as Suricata can include these in eve alerts. The format is:

```
metadata: key value;
metadata: key value, key value;
```

8.2.9 target

The `target` keyword allows the rules writer to specify which side of the alert is the target of the attack. If specified, the alert event is enhanced to contain information about source and target.

The format is:

```
target:[src_ip|dest_ip]
```

If the value is `src_ip` then the source IP in the generated event (`src_ip` field in JSON) is the target of the attack. If target is set to `dest_ip` then the target is the destination IP in the generated event.

8.2.10 requires

The `requires` keyword allows a rule to require specific Suricata features to be enabled, specific keywords to be available, or the Suricata version to match an expression. Rules that do not meet the requirements will be ignored, and Suricata will not treat them as errors.

Requirements that follow the valid format of `<keyword> <expression>` but are not known to Suricata are allowed for future compatibility, however unknown requirement expressions will lead to the requirement not being met, skipping the rule.

When parsing rules, the parser attempts to process the `requires` keywords before others. This allows it to occur after keywords that may only be present in specific versions of Suricata, as specified by the `requires` statement. However, the keywords preceding it must still adhere to the basic known formats of Suricata rules.

The format is:

```
requires: feature geoip, version >= 7.0.0, keyword foobar
```

To require multiple features, the feature sub-keyword must be specified multiple times:

```
requires: feature geoip, feature lua
```

Alternatively, *and* expressions may be expressed like:

```
requires: version >= 7.0.4 < 8
```

and *or* expressions may be expressed with `|` like:

```
requires: version >= 7.0.4 < 8 | >= 8.0.3
```

to express that a rule requires version 7.0.4 or greater, but less than 8, **OR** greater than or equal to 8.0.3. Which could be useful if a keyword wasn't added until 7.0.4 and the 8.0.3 patch releases, as it would not exist in 8.0.1.

This can be extended to multiple release branches:

```
requires: version >= 7.0.10 < 8 | >= 8.0.5 < 9 | >= 9.0.3
```

If no *minor* or *patch* version component is provided, it will default to 0.

The `version` may only be specified once, if specified more than once the rule will log an error and not be loaded.

The `requires` keyword was introduced in Suricata 7.0.3 and 8.0.0.

8.3 IP Keywords

8.3.1 ttl

The ttl keyword is used to check for a specific IP time-to-live value in the header of a packet. The format is:

```
ttl:<number>;
```

For example:

```
ttl:10;
```

ttl uses an *unsigned 8-bit integer*.

At the end of the ttl keyword you can enter the value on which you want to match. The Time-to-live value determines the maximal amount of time a packet can be in the Internet-system. If this field is set to 0, then the packet has to be destroyed. The time-to-live is based on hop count. Each hop/router the packet passes subtracts one from the packet TTL counter. The purpose of this mechanism is to limit the existence of packets so that packets can not end up in infinite routing loops.

Example of the ttl keyword in a rule:

```
alert ip $EXTERNAL_NET any -> $HOME_NET any (msg:"IP Packet With TTL 0"; ttl:0; classtype:misc-activity; sid:1; rev:1;)
```

8.3.2 ipopts

With the ipopts keyword you can check if a specific IP option is set. Ipopts has to be used at the beginning of a rule. You can only match on one option per rule. There are several options on which can be matched. These are:

IP Option	Description
rr	Record Route
eol	End of List
nop	No Op
ts	Time Stamp
sec	IP Security
esec	IP Extended Security
lsrr	Loose Source Routing
ssrr	Strict Source Routing
satid	Stream Identifier
qs	Quick-Start
rtralt	Router Alert
cipso	Commercial Security
any	any IP options are set

Format of the ipopts keyword:

```
ipopts: <name>;
```

For example:

```
ipopts: ts;
```

Example of ipopts in a rule:

```
alert ip $EXTERNAL_NET any -> $HOME_NET any (msg:"IP Packet with timestamp option"; ipopts:ts; classtype:misc-activity; sid:2; rev:1;)
```

8.3.3 sameip

Every packet has a source IP-address and a destination IP-address. It can be that the source IP is the same as the destination IP. With the sameip keyword you can check if the IP address of the source is the same as the IP address of the destination. The format of the sameip keyword is:

```
sameip;
```

Example of sameip in a rule:

```
alert ip any any -> any any (msg:"IP Packet with the same source and destination IP"; sameip; classtype:bad-unknown; sid:3; rev:1;)
```

8.3.4 ip_proto

With the ip_proto keyword you can match on the IP protocol in the packet-header. You can use the name or the number of the protocol. You can match for example on the following protocols:

1	ICMP	Internet Control Message
6	TCP	Transmission Control Protocol
17	UDP	User Datagram
47	GRE	General Routing Encapsulation
50	ESP	Encap Security Payload for IPv6
51	AH	Authentication Header for Ipv6
58	IPv6-ICMP	ICMP for Ipv6

For the complete list of protocols and their numbers see http://en.wikipedia.org/wiki/List_of_IP_protocol_numbers

Example of ip_proto in a rule:

```
alert ip any any -> any any (msg:"IP Packet with protocol 1"; ip_proto:1; classtype:bad-unknown; sid:5; rev:1;)
```

The named variant of that example would be:

```
ip_proto:ICMP;
```

8.3.5 ipv4.hdr

Sticky buffer to match on content contained within an IPv4 header.

Example rule:

```
alert ip any any -> any any (msg:"IPv4 header keyword example"; ipv4.hdr; content:"|06|"; offset:9; depth:1; sid:1; rev:1;)
```

This example looks if byte 10 of IPv4 header has value 06, which indicates that the IPv4 protocol is TCP.

8.3.6 ipv6.hdr

Sticky buffer to match on content contained within an IPv6 header.

Example rule:

```
alert ip any any -> any any (msg:"IPv6 header keyword example"; ipv6.hdr; content:"|06|"; offset:6; depth:1; sid:1; rev:1;)
```

This example looks if byte 7 of IPv6 header has value 06, which indicates that the IPv6 protocol is TCP.

8.3.7 id

With the `id` keyword, you can match on a specific IP ID value. The ID identifies each packet sent by a host and increments usually with one with each packet that is being send. The IP ID is used as a fragment identification number. Each packet has an IP ID, and when the packet becomes fragmented, all fragments of this packet have the same ID. In this way, the receiver of the packet knows which fragments belong to the same packet. (IP ID does not take care of the order, in that case offset is used. It clarifies the order of the fragments.)

Format of `id`:

```
id:<number>;
```

Example of `id` in a rule:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"id keyword example"; id:1; content:"content[3a 20]";
fast_pattern; classtype:misc-activity; sid:12; rev:1;)
```

8.3.8 geoip

The `geoip` keyword enables matching on the source, destination or source and destination IPv4 addresses of network traffic, and to see to which country it belongs. To be able to do this, Suricata uses the GeoIP2 API of MaxMind.

The syntax of `geoip`:

```
geoip: src,RU;
geoip: both,CN,RU;
geoip: dst,CN,RU,IR;
geoip: both,US,CA,UK;
geoip: any,CN,IR;
```

Option	Description
both	Both source and destination have to match with the given <code>geoip(s)</code>
any	Either the source or the destination has to match with the given <code>geoip(s)</code> .
dest	The destination matches with the given <code>geoip</code> .
src	The source matches with the given <code>geoip</code> .

`geoip` currently only supports IPv4. As it uses the GeoIP2 API of MaxMind, `libmaxminddb` must be compiled in. You must download and install the GeoIP2 or GeoLite2 database editions desired. Visit the MaxMind site at <https://dev.maxmind.com/geoip/geoip2-free-geolocation-data> for details.

You must also supply the location of the GeoIP2 or GeoLite2 database file on the local system in the YAML-file configuration (for example):

```
geoip-database: /usr/local/share/GeoIP/GeoLite2-Country.mmdb
```

8.3.9 fragbits (IP fragmentation)

With the `fragbits` keyword, you can check if the fragmentation and reserved bits are set in the IP header. The `fragbits` keyword should be placed at the beginning of a rule. `Fragbits` is used to modify the fragmentation mechanism. During routing of messages from one Internet module to the other, it can occur that a packet is bigger than the maximal packet size a network can process. In that case, a packet can be send in fragments. This maximum of the packet size is called Maximal Transmit Unit (MTU).

You can match on the following bits:

M - More Fragments
D - Do **not** Fragment
R - Reserved Bit

Matching on this bits can be more specified with the following modifiers:

+ match on the specified bits, plus any others
* match if any of the specified bits are set
! match if the specified bits are not set

Format:

```
fragbits:[*+!]<[MDR]>;
```

Example of fragbits in a rule:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"fragbits keyword example non-fragmented packet with fragment offset>0"; fragbits:M; fragoffset:>0; classtype:bad-unknown; sid:123; rev:1;)
```

8.3.10 fragoffset

With the fragoffset keyword you can match on specific decimal values of the IP fragment offset field. If you would like to check the first fragments of a session, you have to combine fragoffset 0 with the More Fragment option. The fragmentation offset field is convenient for reassembly. The id is used to determine which fragments belong to which packet and the fragmentation offset field clarifies the order of the fragments.

You can use the following modifiers:

< match if the value is smaller than the specified value
> match if the value is greater than the specified value
! match if the specified value is not present

Format of fragoffset:

```
fragoffset:[!|<|>]<number>;
```

Example of fragoffset in a rule:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"fragoffset keyword example invalid non-fragmented packet with fragment offset>0"; fragbits:M; fragoffset:>0; classtype:bad-unknown; sid:13; rev:1;)
```

8.3.11 tos

The tos keyword can match on specific decimal values of the IP header TOS field. The tos keyword can have a value from 0 - 255. This field of the IP header has been updated by [rfc2474](#) to include functionality for [Differentiated services](#). Note that the value of the field has been defined with the right-most 2 bits having the value 0. When specifying a value for tos, ensure that the value follows this.

E.g, instead of specifying the decimal value 34 (hex 22), right shift twice and use decimal 136 (hex 88).

You can specify hexadecimal values with a leading x, e.g. x88.

Format of tos:

```
tos:[!]<number>;
```

Example of tos in a rule:

```
alert ip any any -> any any (msg:"tos keyword example tos value 8"; flow:established; tos:8; classtype:not-suspicious; sid:123; rev:1;)
```

Example of tos with a negated value:

```
alert ip any any -> any any (msg:"tos keyword example with negated content"; flow:established,to_server; tos:!8; classtype:bad-unknown; sid:14; rev:1;)
```

8.4 TCP keywords

8.4.1 tcp.flags

The `tcp.flags` keyword checks for specific [TCP flag bits](#).

The following flag bits may be checked:

Flag	Description
F	FIN - Finish
S	SYN - Synchronize sequence numbers
R	RST - Reset
P	PSH - Push
A	ACK - Acknowledgment
U	URG - Urgent
C	CWR - Congestion Window Reduced
E	ECE - ECN-Echo
0	No TCP Flags Set

The following modifiers can be set to change the match criteria:

Modifier	Description
+	match on the bits, plus any others
*	match if any of the bits are set
!	match if the bits are not set

To handle writing rules for session initiation packets such as ECN where a SYN packet is sent with CWR and ECE flags set, an option mask may be used by appending a comma and masked values. For example, a rule that checks for a SYN flag, regardless of the values of the reserved bits is `tcp.flags:S,CE;`

Format of `tcp.flags`:

```
tcp.flags:[modifier]<test flags>[,<ignore flags>];
tcp.flags:[!|*|+]<FSRPAUCE0>[,<FSRPAUCE>];
```

Example:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Example tcp.flags sig"; tcp.flags:FPU,CE; classtype:misc-activity; sid:1; rev:1;)
```

It is also possible to use the `tcp.flags` content as a `fast_pattern` by using the `prefilter` keyword. For more information on `prefilter` usage see [Prefiltering Keywords](#). Example:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Example tcp.flags sig"; tcp.flags:FPU,CE; prefilter; classtype:misc-activity; sid:1; rev:1;)
```

8.4.2 seq

The `seq` keyword can be used in a signature to check for a specific TCP sequence number. A sequence number is a number that is generated practically at random by both endpoints of a TCP-connection. The client and the server both create a sequence number, which increases by one with every byte that they send. So this sequence number is different for both sides. This sequence number has to be acknowledged by both sides of the connection.

Through sequence numbers, TCP handles acknowledgement, order and retransmission. Its number increases with every data-byte the sender has sent. The `seq` helps keeping track of to what place in a data-stream a byte belongs. If the SYN flag is set at 1, then the sequence number of the first byte of the data is this number plus 1 (so, 2).

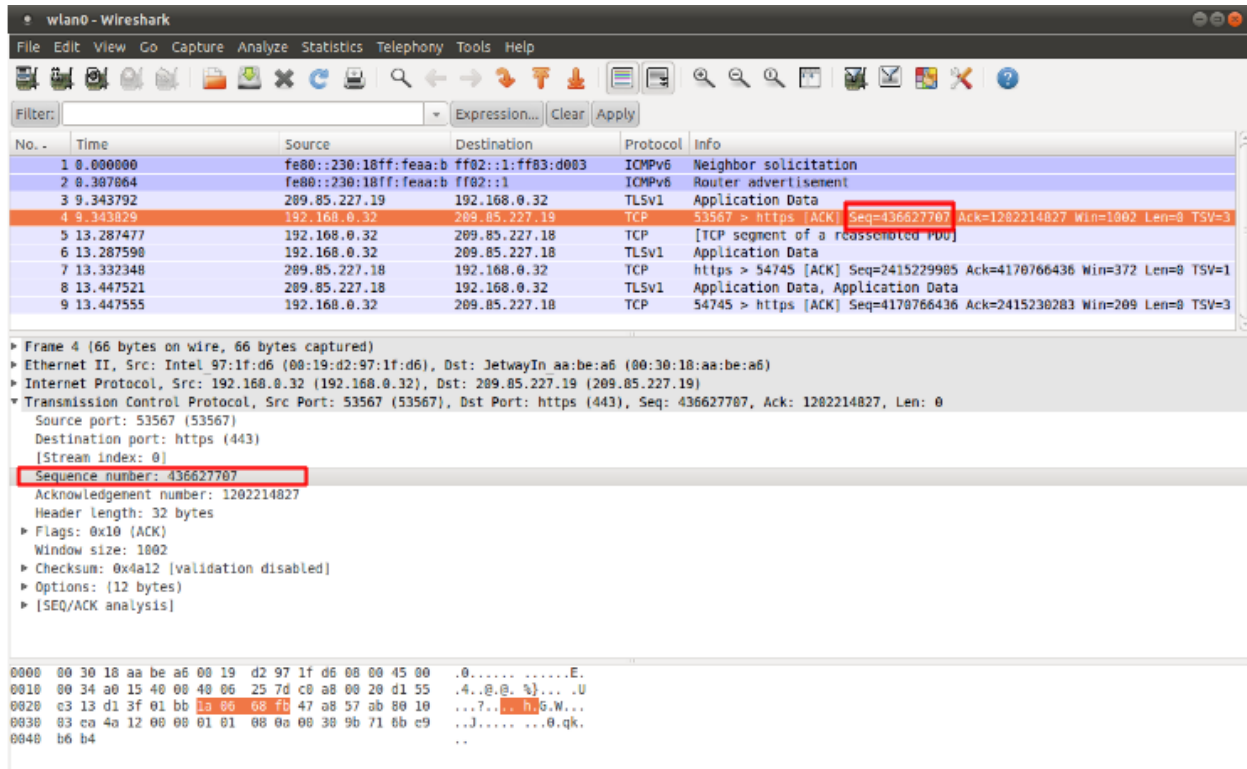
Example:

```
seq:0;
```

Example of `seq` in a signature:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"GPL SCAN NULL"; flow:stateless; ack:0; flags:0; seq:0; reference:arachnids,4; classtype:attempted-recon; sid:2100623; rev:7;)
```

Example of `seq` in a packet (Wireshark):



8.4.3 ack

The `ack` keyword can be used in a signature to check for a specific TCP acknowledgement number.

The `ack` is the acknowledgement of the receipt of all previous (data)-bytes send by the other side of the TCP-connection. In most occasions every packet of a TCP connection has an ACK flag after the first SYN and a ack-number which increases with the receipt of every new data-byte.

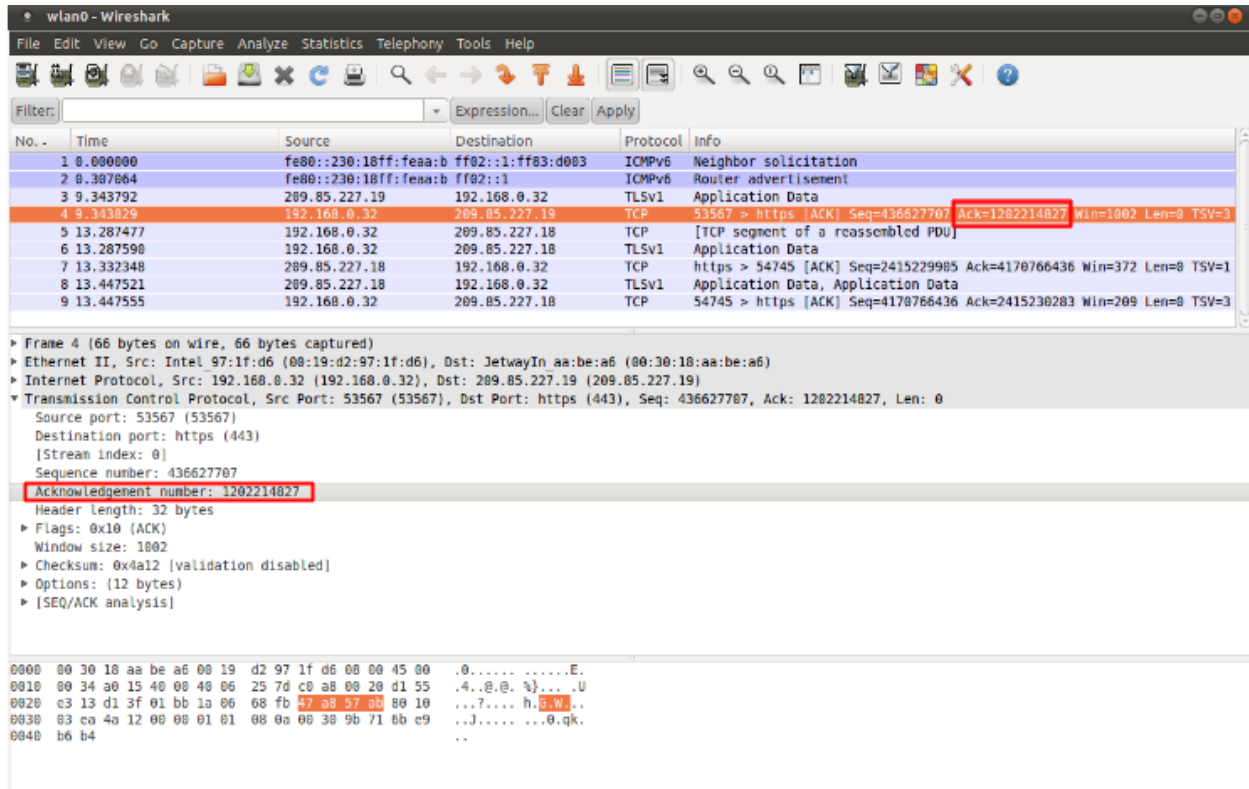
Format of `ack`:

```
ack:1;
```

Example of ack in a signature:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"GPL SCAN NULL"; flow:stateless; ack:0; flags:0; seq:0; reference:arachnids,4; classtype:attempted-recon; sid:2100623; rev:7;)
```

Example of ack in a packet (Wireshark):



8.4.4 window

The window keyword is used to check for a specific TCP window size.

The TCP window size is a mechanism that has control of the data-flow. The window is set by the receiver (receiver advertised window size) and indicates the amount of bytes that can be received. This amount of data has to be acknowledged by the receiver first, before the sender can send the same amount of new data.

This mechanism is used to prevent the receiver from being overflowed by data. The value of the window size is limited and can be 2 to 65.535 bytes. To make more use of your bandwidth you can use a bigger TCP-window.

The format of the window keyword is:

```
window:[!]<number>;
```

Example of window in a rule:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"GPL DELETED typot trojan traffic"; flow:stateless; flags:S,12; window:55808; reference:mcafee,100406; classtype:trojan-activity; sid:2182; rev:8;)
```

8.4.5 tcp.mss

Match on the TCP MSS option value. Will not match if the option is not present.

tcp.mss uses an *unsigned 16-bit integer*.

The format of the keyword is:

```
tcp.mss:<min>-<max>;  
tcp.mss:[<|>]<number>;  
tcp.mss:<value>;
```

Example rule:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (flow:stateless; flags:S,12; tcp.mss:<536; sid:1234; rev:5;)
```

8.4.6 tcp.wscale

Match on the TCP window scaling option value. Will not match if the option is not present.

tcp.wscale uses an *unsigned 8-bit integer*.

The format of the keyword is:

```
tcp.wscale:<min>-<max>;  
tcp.wscale:[<|>]<number>;  
tcp.wscale:<value>;
```

Example rule:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (flow:stateless; flags:S,12; tcp.wscale:>10; sid:1234; rev:5;)
```

8.4.7 tcp.hdr

Sticky buffer to match on the whole TCP header.

Example rule:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (flags:S,12; tcp.hdr; content:"|02 04|"; offset:20;  
byte_test:2,<,536,0,big,relative; sid:1234; rev:5;)
```

This example starts looking after the fixed portion of the header, so into the variable sized options. There it will look for the MSS option (type 2, option len 4) and using a byte_test determine if the value of the option is lower than 536. The *tcp.mss* option will be more efficient, so this keyword is meant to be used in cases where no specific keyword is available.

8.5 UDP keywords

8.5.1 udp.hdr

Sticky buffer to match on the whole UDP header.

Example rule:

```
alert udp any any -> any any (udp.hdr; content:"|00 08|"; offset:4; depth:2; sid:1234; rev:5;)
```

This example matches on the length field of the UDP header. In this case the length of 8 means that there is no payload. This can also be matched using *dsiz:0*;

8.6 ICMP keywords

ICMP (Internet Control Message Protocol) is a part of IP. IP at itself is not reliable when it comes to delivering data (datagram). ICMP gives feedback in case problems occur. It does not prevent problems from happening, but helps in understanding what went wrong and where. If reliability is necessary, protocols that use IP have to take care of reliability themselves. In different situations ICMP messages will be send. For instance when the destination is unreachable, if there is not enough buffer-capacity to forward the data, or when a datagram is send fragmented when it should not be, etcetera. More can be found in the list with message-types.

There are four important contents of a ICMP message on which can be matched with corresponding ICMP-keywords. These are: the type, the code, the id and the sequence of a message.

8.6.1 itype

The itype keyword is for matching on a specific ICMP type (number). ICMP has several kinds of messages and uses codes to clarify those messages. The different messages are distinct by different names, but more important by numeric values. For more information see the table with message-types and codes.

itype uses an *unsigned 8-bit integer*.

The format of the itype keyword:

```
itype:min<>max;
itype:[<|>]<number>;
```

Example This example looks for an ICMP type greater than 10:

```
itype:>10;
```

Example of the itype keyword in a signature:

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"GPL SCAN Broadscan Smurf Scanner"; dsize:4;
icmp_id:0; icmp_seq:0; itype:8; classtype:attempted-recon; sid:2100478; rev:4;)
```

The following lists all ICMP types known at the time of writing. A recent table can be found [at the website of IANA](#)

ICMP Type	Name
0	Echo Reply
3	Destination Unreachable
4	Source Quench
5	Redirect
6	Alternate Host Address
8	Echo
9	Router Advertisement
10	Router Solicitation
11	Time Exceeded
12	Parameter Problem
13	Timestamp
14	Timestamp Reply
15	Information Request
16	Information Reply
17	Address Mask Request
18	Address Mask Reply
30	Traceroute
31	Datagram Conversion Error
32	Mobile Host Redirect
33	IPv6 Where-Are-You
34	IPv6 I-Am-Here
35	Mobile Registration Request
36	Mobile Registration Reply
37	Domain Name Request
38	Domain Name Reply
39	SKIP
40	Photuris
41	Experimental mobility protocols such as Seamoby

8.6.2 icode

With the icode keyword you can match on a specific ICMP code. The code of a ICMP message clarifies the message. Together with the ICMP-type it indicates with what kind of problem you are dealing with. A code has a different purpose with every ICMP-type.

icode uses an *unsigned 8-bit integer*.

The format of the icode keyword:

```
icode:min<max;  
icode:[<|>]<number>;
```

Example: This example looks for an ICMP code greater than 5:

```
icode:>5;
```

Example of the icode keyword in a rule:

```
alert icmp $HOME_NET any -> $EXTERNAL_NET any (msg:"GPL MISC Time-To-Live Exceeded in Transit";  
icode:0; itype:11; classtype:misc-activity; sid:2100449; rev:7;)
```

The following lists the meaning of all ICMP types. When a code is not listed, only type 0 is defined and has the meaning of the ICMP code, in the table above. A recent table can be found [at the website of IANA](#)

ICMP Code	ICMP Type	Description
3	0	Net Unreachable
	1	Host Unreachable
	2	Protocol Unreachable
	3	Port Unreachable
	4	Fragmentation Needed and Don't Fragment was Set
	5	Source Route Failed
	6	Destination Network Unknown
	7	Destination Host Unknown
	8	Source Host Isolated
	9	Communication with Destination Network is Administratively Prohibited
	10	Communication with Destination Host is Administratively Prohibited
	11	Destination Network Unreachable for Type of Service
	12	Destination Host Unreachable for Type of Service
	13	Communication Administratively Prohibited
	14	Host Precedence Violation
	15	Precedence cutoff in effect
5	0	Redirect Datagram for the Network (or subnet)
	1	Redirect Datagram for the Host
	2	Redirect Datagram for the Type of Service and Network
	3	Redirect Datagram for the Type of Service and Host
9	0	Normal router advertisement
	16	Doesn't route common traffic
11	0	Time to Live exceeded in Transit
	1	Fragment Reassembly Time Exceeded
12	0	Pointer indicates the error
	1	Missing a Required Option
	2	Bad Length
40	0	Bad SPI
	1	Authentication Failed
	2	Decompression Failed
	3	Decryption Failed
	4	Need Authentication
	5	Need Authorization

8.6.3 icmp_id

With the `icmp_id` keyword you can match on specific ICMP id-values. Every ICMP-packet gets an id when it is being send. At the moment the receiver has received the packet, it will send a reply using the same id so the sender will recognize it and connects it with the correct ICMP-request.

Format of the icmp_id keyword:

```
icmp_id:<number>;
```

Example: This example looks for an ICMP ID of 0:

```
icmp_id:0;
```

Example of the icmp_id keyword in a rule:

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"GPL SCAN Broadscan Smurf Scanner"; dsize:4; icmp_id:0; icmp_seq:0; itype:8; classtype:attempted-recon; sid:2100478; rev:4;)
```

8.6.4 icmp_seq

You can use the icmp_seq keyword to check for a ICMP sequence number. ICMP messages all have sequence numbers. This can be useful (together with the id) for checking which reply message belongs to which request message.

Format of the icmp_seq keyword:

```
icmp_seq:<number>;
```

Example: This example looks for an ICMP Sequence of 0:

```
icmp_seq:0;
```

Example of icmp_seq in a rule:

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"GPL SCAN Broadscan Smurf Scanner"; dsize:4; icmp_id:0; icmp_seq:0; itype:8; classtype:attempted-recon; sid:2100478; rev:4;)
```

Note

Some pcap analysis tools, like wireshark, may give both a little endian and big endian value for icmp_seq. The icmp_seq keyword matches on the big endian value, this is due to Suricata using the network byte order (big endian) to perform the match comparison.

8.6.5 icmpv4.hdr

Sticky buffer to match on the whole ICMPv4 header.

8.6.6 icmpv6.hdr

Sticky buffer to match on the whole ICMPv6 header.

8.6.7 icmpv6.mtu

Match on the ICMPv6 MTU optional value. Will not match if the MTU is not present.

icmpv6.mtu uses an *unsigned 32-bit integer*.

The format of the keyword:

```
icmpv6.mtu:<min>-<max>;  
icmpv6.mtu:[<|>]<number>;  
icmpv6.mtu:<value>;
```

Example rule:

```
alert ip $EXTERNAL_NET any -> $HOME_NET any (icmpv6.mtu:<1280; sid:1234; rev:5;)
```

8.7 Payload Keywords

Payload keywords inspect the content of the payload of a packet or stream.

8.7.1 content

The content keyword is very important in signatures. Between the quotation marks you can write on what you would like the signature to match. The most simple format of content is:

```
content: ".....";
```

It is possible to use several contents in a signature.

Contents match on bytes. There are 256 different values of a byte (0-255). You can match on all characters; from a till z, upper case and lower case and also on all special signs. But not all of the bytes are printable characters. For these bytes heximal notations are used. Many programming languages use 0x00 as a notation, where 0x means it concerns a binary value, however the rule language uses |00| as a notation. This kind of notation can also be used for printable characters.

Example:

```
|61| is a
|61 61| is aa
|41| is A
|21| is !
|0D| is carriage return
|0A| is line feed
```

There are characters you can not use in the content because they are already important in the signature. For matching on these characters you should use the heximal notation. These are:

```
"      |22|
;      |3B|
:      |3A|
|      |7C|
```

It is a convention to write the heximal notation in upper case characters.

To write for instance http:// in the content of a signature, you should write it like this: `content: "http|3A|//";` If you use a heximal notation in a signature, make sure you always place it between pipes. Otherwise the notation will be taken literally as part of the content.

A few examples:

```
content:"a|0D|bc";
content:"|61 0D 62 63|";
content:"a|0D|b|63|";
```

It is possible to let a signature check the whole payload for a match with the content or to let it check specific parts of the payload. We come to that later. If you add nothing special to the signature, it will try to find a match in all the bytes of the payload.

```
drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"ET TROJAN Likely Bot Nick in IRC (USA +..)";
flow:established,to_server; flowbits:isset,is_proto_irc; content:"NICK "; pcre:"/NICK .*USA.*[0-9]{3,}/i"; refer-
ence:url,doc.emergingthreats.net/2008124; classtype:trojan-activity; sid:2008124; rev:2;)
```

By default the pattern-matching is case sensitive. The content has to be accurate, otherwise there will not be a match.



content:"abc";	✗
content:"aBc";	✗
content:"abC";	✓

Legend:

✓	match
✗	no match
	match in the payload
	no match in the payload

It is possible to use the ! for exceptions in contents as well.

For example:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"Outdated Firefox on
Windows"; content:"User-Agent|3A| Mozilla/5.0 |28|Windows|3B| ";
content:"Firefox/3."; distance:0; content:! "Firefox/3.6.13";
distance:-10; sid:90000000; rev:1;)
```

You see `content:! "Firefox/3.6.13";`. This means an alert will be generated if the used version of Firefox is not 3.6.13.

Note

The following characters must be escaped inside the content: ; \ "

8.7.2 nocase

If you do not want to make a distinction between uppercase and lowercase characters, you can use nocase. The keyword nocase is a content modifier.

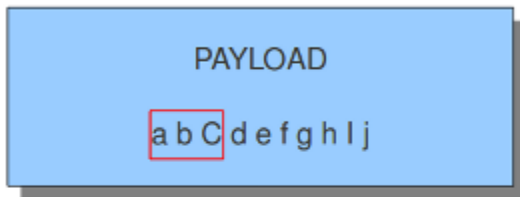
The format of this keyword is:

```
nocase;
```

You have to place it after the content you want to modify, like:

```
content: "abc"; nocase;
```

Example nocase:



```
content:"abc"; nocase; ✓
```

```
content:"aBc"; nocase; ✓
```

```
content:"abC"; nocase; ✓
```

It has no influence on other contents in the signature.

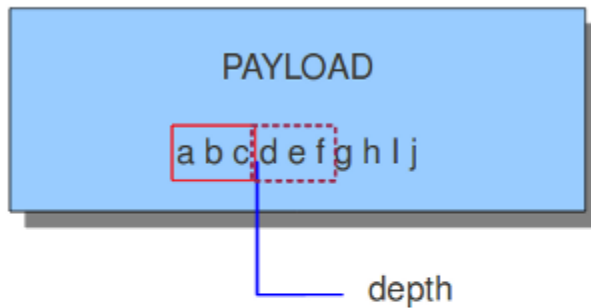
8.7.3 depth

The depth keyword is an absolute content modifier. It comes after the content. The depth content modifier comes with a mandatory numeric value, like:

```
depth: 12;
```

The number after depth designates how many bytes from the beginning of the payload will be checked.

Example:



content:"def"; depth:3;

X

content:"abc"; depth:3;

✓

8.7.4 startswith

The `startswith` keyword is similar to `depth`. It takes no arguments and must follow a `content` keyword. It modifies the content to match exactly at the start of a buffer.

Example:

```
content:"GET|20|"; startswith;
```

`startswith` is a short hand notation for:

```
content:"GET|20|"; depth:4; offset:0;
```

`startswith` cannot be mixed with `depth`, `offset`, `within` or `distance` for the same pattern.

8.7.5 endswith

The `endswith` keyword is similar to `isdataat:!1,relative;`. It takes no arguments and must follow a `content` keyword. It modifies the content to match exactly at the end of a buffer.

Example:

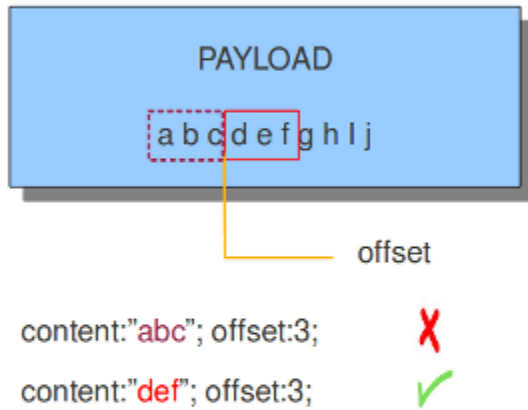
```
content:".php"; endswith;
```

`endswith` is a short hand notation for:

```
content:".php"; isdataat:!1,relative;
```

8.7.6 offset

The offset keyword designates from which byte in the payload will be checked to find a match. For instance `offset:3;` checks the fourth byte and further.

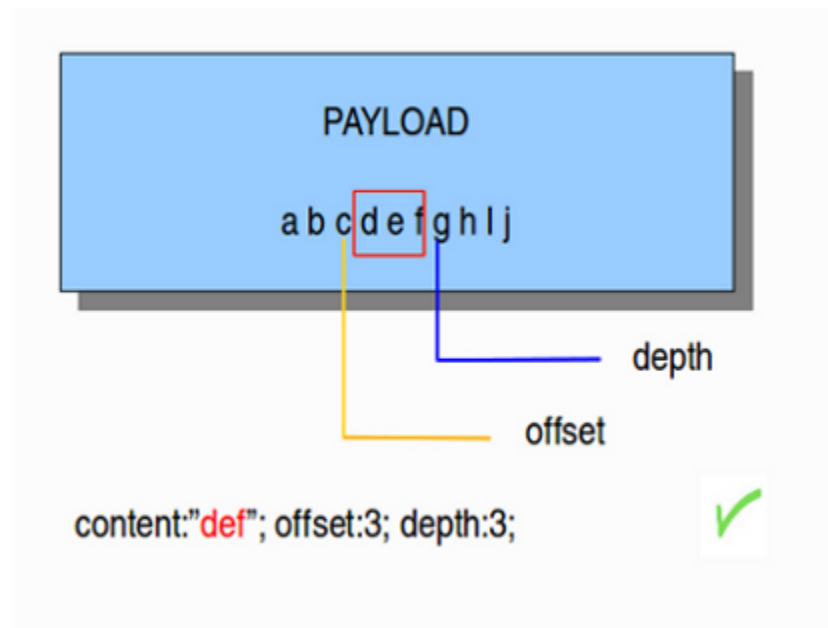


The keywords offset and depth can be combined and are often used together.

For example:

```
content:"def"; offset:3; depth:3;
```

If this was used in a signature, it would check the payload from the third byte till the sixth byte.



8.7.7 distance

The keyword distance is a relative content modifier. This means it indicates a relation between this content keyword and the content preceding it. Distance has its influence after the preceding match. The keyword distance comes with a mandatory numeric value. The value you give distance, determines the byte in the payload from which will be checked for a match relative to the previous match. Distance only determines where Suricata will start looking for a pattern. So, distance:5; means the pattern can be anywhere after the previous match + 5 bytes. For limiting how far after the last match Suricata needs to look, use 'within'.

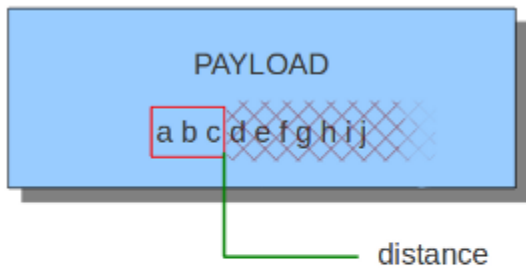
The absolute value for distance must be less than or equal to 1MB (1048576).

Examples of distance:

content:"abc"; content:"klm"; distance: 0;

1 2 3

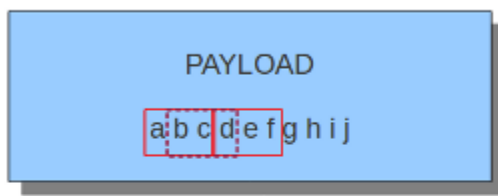
The distance (3), tells how the second (2) content relates to the first (1) content.



content:"abc"; content:"klm"; distance: 0; ❌

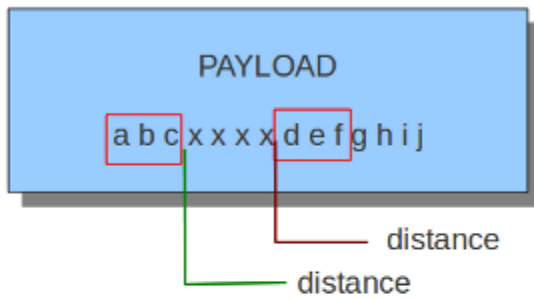


checked area using 'distance'



content:"abc"; content:"def"; distance:0; ✅

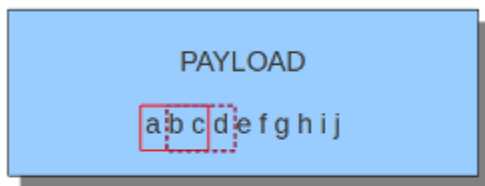
content:"abc"; content:"bcd"; distance:0; ❌



content:"abc"; content:"def"; distance:0; ✓

content:"abc"; content:"def"; distance:4; ✓

Distance can also be a negative number. It can be used to check for matches with partly the same content (see example) or for a content even completely before it. This is not very often used though. It is possible to attain the same results with other keywords.



content:"abc"; content:"bcd"; distance:-2; ✓

8.7.8 within

The keyword `within` is relative to the preceding match. The keyword `within` comes with a mandatory numeric value. Using `within` makes sure there will only be a match if the content matches with the payload within the set amount of bytes. `Within` can not be 0 (zero)

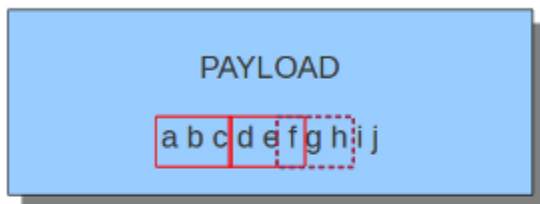
The absolute value for `within` must be less than or equal to 1MB (1048576).

Example:



The keyword `within` (3), tells how the second (2) content relates to the first (1) content.

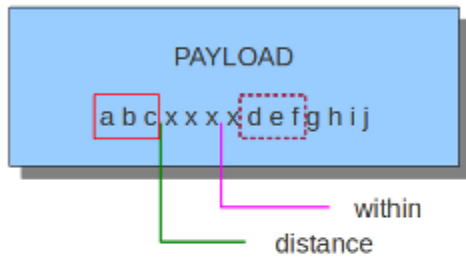
Example of matching with `within`:



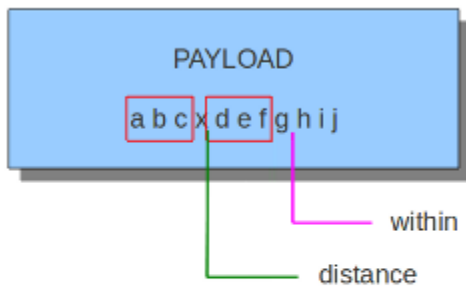
content:"abc"; content:"def"; within:3; ✓
 content:"abc"; content:"fgh"; within:3; ✗

The second content has to fall/come 'within 3 ' from the first content.

As mentioned before, `distance` and `within` can be very well combined in a signature. If you want Suricata to check a specific part of the payload for a match, use `within`.



content:"abc"; content:"def"; distance:0; within:3; ❌



content:"abc"; content:"def"; distance:1; within:4; ✅

8.7.9 rawbytes

The rawbytes keyword has no effect but is included to be compatible with signatures that use it, for example signatures used with Snort.

8.7.10 isdataat

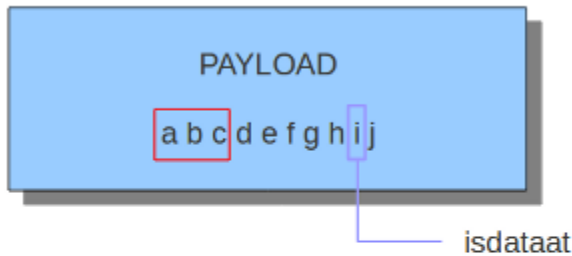
The purpose of the isdataat keyword is to look if there is still data at a specific part of the payload. The keyword starts with a number (the position) and then optional followed by 'relative' separated by a comma and the option rawbytes. You use the word 'relative' to know if there is still data at a specific part of the payload relative to the last match.

So you can use both examples:

```
isdataat:512;
isdataat:50, relative;
```

The first example illustrates a signature which searches for byte 512 of the payload. The second example illustrates a signature searching for byte 50 after the last match.

You can also use the negation (!) before isdataat.



content:"abc"; isdataat:6, relative; ✓

content:"abc"; isdataat:8, relative; ✗

absolute vs relative values

The absolute isdataat checks will succeed if the offset used is **less than** the size of the inspection buffer.

For *relative* isdataat checks, there is a **1 byte difference** vs the absolute handling.

Matching will succeed if the relative offset is **less than or equal to** the size of the inspection buffer. This is different from absolute isdataat checks.

As an example, consider a 32 byte payload:

rule statement	Match?
isdataat:31;	Yes
isdataat:32;	No
isdataat:31,relative;	Yes
isdataat:32,relative;	Yes
isdataat:33,relative;	No

Another example, consider the following payload:

Index	0	1	2	3	4	5	6	7
Value	a	b	c	d	e	f	g	h

Then the following rules match the payload as follows:

Rule statement	Match?
isdataat:7;	Yes
isdataat:8;	No
isdataat:7,relative;	Yes
isdataat:8,relative;	Yes
isdataat:9,relative;	No
payload:"c"; isdataat:4,relative;	Yes
payload:"c"; isdataat:5,relative;	Yes
payload:"c"; isdataat:6,relative;	No

These differences are also discussed in *Differences From Snort*.

A discussion of this difference can be found at <https://redmine.openinfosecfoundation.org/issues/8031>

8.7.11 absent

The keyword **absent** checks that a sticky buffer does not exist. It can be used without any argument to match only on absent buffer :

Example of **absent** in a rule:

```
alert http any any -> any any (msg:"HTTP request without referer"; http.referer; absent; sid:1; rev:1;)
```

It can take an argument "or_else" to match on absent buffer or on what comes next such as negated content, for instance :

```
alert http any any -> any any (msg:"HTTP request without referer"; http.referer; absent: or_else; content: !"abc"; sid:1; rev:1;)
```

For files (i.e `file.data`), **absent** means there are no files in the transaction.

8.7.12 bsize

With the **bsize** keyword, you can match on the length of the buffer. This adds precision to the content match, previously this could have been done with **isdataat**.

bsize uses an *unsigned 64-bit integer*.

An optional operator can be specified; if no operator is present, the operator will default to '='. When a relational operator is used, e.g., '<', '>' or '<>' (range), the **bsize** value will be compared using the relational operator. Ranges are exclusive.

If one or more **content** keywords precedes **bsize**, each occurrence of **content** will be inspected and an error will be raised if the content length and the **bsize** value prevent a match.

Format:

```
bsize:<number>;
bsize:=<number>;
bsize:<<number>;
bsize:><number>;
bsize:<lo-number><><hi-number>;
```

Examples of **bsize** in a rule:

```
alert dns any any -> any any (msg:"bsize exact buffer size"; dns.query; content:"google.com"; bsize:10; sid:1; rev:1;)
```

```
alert dns any any -> any any (msg:"bsize less than value"; dns.query; content:"google.com"; bsize:<25; sid:2; rev:1;)
```

```
alert dns any any -> any any (msg:"bsize buffer less than or equal value"; dns.query; content:"google.com"; bsize:<=20; sid:3; rev:1;)
```

```
alert dns any any -> any any (msg:"bsize buffer greater than value"; dns.query; content:"google.com"; bsize:>8; sid:4; rev:1;)
```

```
alert dns any any -> any any (msg:"bsize buffer greater than or equal value"; dns.query; content:"google.com"; bsize:>=8; sid:5; rev:1;)
```

```
alert dns any any -> any any (msg:"bsize buffer range value"; dns.query; content:"google.com"; bsize:8<>20; sid:6; rev:1;)
```

```
alert dns any any -> any any (msg:"test bsize rule"; dns.query; content:"short"; bsize:<10; sid:124; rev:1;)
```

```
alert dns any any -> any any (msg:"test bsize rule"; dns.query; content:"longer string"; bsize:>10; sid:125; rev:1;)
```

```
alert dns any any -> any any (msg:"test bsize rule"; dns.query; content:"middle"; bsize:6<>15; sid:126; rev:1;)
```

To emphasize how range works: in the example above, a match will occur if `bsize` is greater than 6 and less than 15.

8.7.13 dsize

With the `dsize` keyword, you can match on the size of the packet payload/data. You can use the keyword for example to look for abnormal sizes of payloads which are equal to some `n` i.e. `'dsize:n'` not equal `'dsize:!n'` less than `'dsize:<n'` or greater than `'dsize:>n'` This may be convenient in detecting buffer overflows.

`dsize` cannot be used when using app/streamlayer protocol keywords (i.e. `http.uri`)

`dsize` uses an *unsigned 16-bit integer*.

Format:

```
dsize:[<>!]number; || dsize:min<>max;
```

Examples of `dsize` values:

```
alert tcp any any -> any any (msg:"dsize exact size"; dsize:10; sid:1; rev:1;)
```

```
alert tcp any any -> any any (msg:"dsize less than value"; dsize:<10; sid:2; rev:1;)
```

```
alert tcp any any -> any any (msg:"dsize less than or equal value"; dsize:<=10; sid:3; rev:1;)
```

```
alert tcp any any -> any any (msg:"dsize greater than value"; dsize:>8; sid:4; rev:1;)
```

```
alert tcp any any -> any any (msg:"dsize greater than or equal value"; dsize:>=10; sid:5; rev:1;)
```

```
alert tcp any any -> any any (msg:"dsize range value"; dsize:8<>20; sid:6; rev:1;)
```

```
alert tcp any any -> any any (msg:"dsize not equal value"; dsize:!9; sid:7; rev:1;)
```

8.7.14 byte_test

The `byte_test` keyword extracts `<num of bytes>` and performs an operation selected with `<operator>` against the value in `<test value>` at a particular `<offset>`. The `<bitmask value>` is applied to the extracted bytes (before the operator is applied), and the final result will be right shifted one bit for each trailing `0` in the `<bitmask value>`.

Format:

```
byte_test:<num of bytes> | <variable_name>, [!]<operator>, <test value>, <offset> [,
↪relative] \
[,<endian>][, string, <num type>][, dce][, bitmask <bitmask value>];
```

<num of bytes>	The number of bytes selected from the packet to be converted or the name of a byte_extract/byte_math variable.
<operator>	• [!] Negation can prefix other operators • < less than • > greater than • = equal • <= less than or equal • >= greater than or equal • & bitwise AND • ^ bitwise OR
<value>	Value to test the converted value against [hex or decimal accepted]
<offset>	Number of bytes into the payload
[relative]	Offset relative to last content match
[endian]	Type of number being read: - big (Most significant byte at lowest address) - little (Most significant byte at the highest address)
[string]	• hex - Converted string represented in hex
<num>	• dec - Converted string represented in decimal • oct - Converted string represented in octal
[dce]	Allow the DCE module to determine the byte order
[bitmask]	Applies the AND operator on the bytes converted

Example:

```

alert tcp any any -> any any \
  (msg:"Byte_Test Example - Num = Value"; \
  content:"|00 01 00 02|"; byte_test:2,=,0x01,0;)

alert tcp any any -> any any \
  (msg:"Byte_Test Example - Num = Value relative to content"; \
  content:"|00 01 00 02|"; byte_test:2,=,0x03,2,relative;)

alert tcp any any -> any any \
  (msg:"Byte_Test Example - Num != Value"; content:"|00 01 00 02|"; \
  byte_test:2,! =,0x06,0;)

alert tcp any any -> any any \
  (msg:"Byte_Test Example - Detect Large Values"; content:"|00 01 00 02|"; \
  byte_test:2,>,1000,1,relative;)

alert tcp any any -> any any \
  (msg:"Byte_Test Example - Lowest bit is set"; \
  content:"|00 01 00 02|"; byte_test:2,&,0x01,12,relative;)

alert tcp any any -> any any (msg:"Byte_Test Example - Compare to String"; \
  content:"foobar"; byte_test:4,=,1337,1,relative,string,dec;)

```

8.7.15 byte_math

The byte_math keyword adds the capability to perform mathematical operations on extracted values with an existing variable or a specified value.

When relative is included, there must be a previous content or pcre match.

Note: if `oper` is `/` and the divisor is 0, there will never be a match on the `byte_math` keyword.

The result can be stored in a result variable and referenced by other rule options later in the rule.

Keyword	Modifier
<code>content</code>	<code>offset,depth,distance,within</code>
<code>byte_test</code>	<code>offset,value</code>
<code>byte_jump</code>	<code>offset</code>
<code>isdataat</code>	<code>offset</code>

Format:

```
byte_math:bytes <num of bytes> | <variable-name> , offset <offset>, oper <operator>, \
    rvalue <rvalue>, \
    result <result_var> [, relative] [, endian <endian>] [, string <number-type>] \
    [, dce] [, bitmask <value>];
```

<num of bytes>	The number of bytes selected from the packet or the name of a <code>byte_extract</code> variable.
<offset>	Number of bytes into the payload
oper <operator>	Mathematical operation to perform: <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code><<</code> , <code>>></code>
rvalue <rvalue>	Value to perform the math operation with
result <result-var>	Where to store the computed value
[relative]	Offset relative to last content match
[endian <type>]	- big (Most significant byte at lowest address) - little (Most significant byte at the highest address) - dce (Allow the DCE module to determine the byte order)
[string <num_type>]	- hex Converted data is represented in hex - dec Converted data is represented in decimal - oct Converted data is represented as octal
[dce]	Allow the DCE module to determine the byte order
[bitmask] <value>	The AND operator will be applied to the extracted value The result will be right shifted by the number of bits equal to the number of trailing zeros in the mask

Example:

```
alert tcp any any -> any any \
    (msg:"Testing bytemath_body"; \
    content:"|00 04 93 F3|"; \
    content:"|00 00 00 07|"; distance:4; within:4; \
    byte_math:bytes 4, offset 0, oper +, rvalue \
    248, result var, relative;)

alert udp any any -> any any \
    (byte_extract: 1, 0, extracted_val, relative; \
    byte_math: bytes 1, offset 1, oper +, rvalue extracted_val, result var; \
    byte_test: 2, =, var, 13; \
    msg:"Byte extract and byte math with byte test verification");)
```

8.7.16 byte_jump

The `byte_jump` keyword allows for the ability to select a `<num of bytes>` from an `<offset>` and moves the detection pointer to that position. Content matches will then be based off the new position.

Format:

```
byte_jump:<num of bytes> | <variable-name>, <offset> [, relative][, multiplier <mult_
↪value>] \
    [, <endian>][, string, <num_type>][, align][, from_beginning][, from_end] \
    [, post_offset <value>][, dce][, bitmask <value>;
```

<num of bytes>	The number of bytes selected from the packet to be converted or the name of a <code>byte_extract/byte_math</code> variable.
<offset>	Number of bytes into the payload
[relative]	Offset relative to last content match
[multiplier]	Multiple the converted byte by the <value>
<value>	
[endian]	- big (Most significant byte at lowest address) - little (Most significant byte at the highest address)
[string]	hex Converted data is represented in hex
<num_type>	- dec Converted data is represented in decimal - oct Converted data is represented as octal
[align]	Rounds the number up to the next 32bit boundary
[from_beginning]	Jumps forward from the beginning of the packet, instead of where the detection pointer is set
[from_end]	Jump will begin at the end of the payload, instead of where the detection point is set
[post_offset]	After the jump operation has been performed, it will jump an additional number of bytes specified by <value>
<value>	
[dce]	Allow the DCE module to determine the byte order
[bitmask]	The AND operator will be applied by <value> and the converted bytes, then jump operation is performed
<value>	

Example:

```
alert tcp any any -> any any \
    (msg:"Byte_Jump Example"; \
    content:"Alice"; byte_jump:2,0; content:"Bob");

alert tcp any any -> any any \
    (msg:"Byte_Jump Multiple Jumps"; \
    byte_jump:2,0; byte_jump:2,0,relative; content:"foobar"; distance:0; within:6;)

alert tcp any any -> any any \
    (msg:"Byte_Jump From the End -8 Bytes"; \
    byte_jump:0,0, from_end, post_offset -8; \
    content:"|6c 33 33 74|"; distance:0 within:4;)
```

8.7.17 byte_extract

The `byte_extract` keyword extracts `<num of bytes>` at a particular `<offset>` and stores it in `<var_name>`. The value in `<var_name>` can be used in any modifier that takes a number as an option and in the case of `byte_test` it can be used as a value.

Format:

```
byte_extract:<num of bytes>, <offset>, <var_name>, [,relative] [,multiplier <mult-value>] \
    [,<endian>] [, dce] [, string [, <num_type>] [, align <align-value>];
```

<num of bytes>	The number of bytes selected from the packet to be extracted
<offset>	Number of bytes into the payload
<var_name>	The name of the variable in which to store the value
[relative]	Offset relative to last content match
multiplier <value>	multiply the extracted bytes by <mult-value> before storing
[endian]	Type of number being read: - big (Most significant byte at lowest address) - little (Most significant byte at the highest address)
[string]	hex - Converted string represented in hex
<num>	- dec - Converted string represented in decimal - oct - Converted string represented in octal
[dce]	Allow the DCE module to determine the byte order
align <align-value>	Round the extracted value up to the next <align-value> byte boundary post-multiplication (if any); <align-value> may be 2 or 4

Keyword	Modifier
content	offset,depth,distance,within
byte_test	offset,value
byte_math	rvalue
byte_jump	offset
isdataat	offset

Example:

```
alert tcp any any -> any any \
    (msg:"Byte_Extract Example Using distance"; \
    content:"Alice"; byte_extract:2,0,size; content:"Bob"; distance:size; within:3; \
    sid:1;)
alert tcp any any -> any any \
    (msg:"Byte_Extract Example Using within"; \
    flow:established,to_server; content:"|00 FF|"; \
    byte_extract:1,0,len,relative; content:"|5c 00|"; distance:2; within:len; sid:2;)
alert tcp any any -> any any \
    (msg:"Byte_Extract Example Comparing Bytes"; \
    flow:established,to_server; content:"|00 FF|"; \
    byte_extract:2,0,cmp_ver,relative; content:"FooBar"; distance:0; byte_test:2,=, \
    cmp_ver,0; sid:3;)
```

8.7.18 entropy

The entropy keyword calculates the Shannon entropy value for content and compares it with an entropy value. When there is a match, rule processing will continue. Entropy values are between 0.0 and 8.0, inclusive. Internally, entropy is represented as a 64-bit floating point value.

The entropy keyword syntax is the keyword entropy followed by options and the entropy value and operator used to determine if the values agree.

The minimum entropy keyword specification is:

```
entropy: value <entropy-spec>
```

This results in the calculated entropy value being compared with *entropy-spec* using the (default) equality operator.

Example:

```
entropy: 7.01
```

A match occurs when the calculated entropy and specified entropy values agree. This is determined by calculating the entropy value and comparing it with the value from the rule using the specified operator.

Example:

```
entropy: <7.01
```

Options have default values: - bytes is equal to the current content length - offset is 0 - equality comparison

When entropy keyword options are specified, all options and "value" must be comma-separated. Options and value may be specified in any order.

The complete format for the **entropy** keyword is:

```
entropy: [bytes <byteval>] [offset <offsetval>] value <operator><entropy-value>
```

This example shows all possible options with default values and an entropy value of 4.037:

```
entropy: bytes 0, offset 0, value = 4.037
```

The following operators are available:

```
* = (default): Match when calculated value equals entropy value
* < Match when calculated value is strictly less than entropy value
* <= Match when calculated value is less than or equal to entropy value
* > Match when calculated value is strictly greater than entropy value
* >= Match when calculated value is greater than or equal to entropy value
* != Match when calculated value is not equal to entropy value
* x-y Match when calculated value is within the exclusive range
* !x-y Match when calculated value is not within the exclusive range
```

This example matches if the *file.data* content for an HTTP transaction has a Shannon entropy value of 4 or higher:

```
alert http any any -> any any (msg:"entropy simple test"; file.data; entropy: value >= 4;
  ↳ sid:1;)
```

Logging

When the **entropy** rule keyword is provided and the rule is evaluated, the *calculated entropy* value is associated with the flow even if the calculated entropy value didn't result in a match or alert. Subsequent logging of event types that include the flow, including alerts, will contain the **entropy** value in the **metadata** section of an output log. The follow is an example that shows the calculated entropy value with the buffer on which the value was computed:

```
"metadata": {
  "entropy": {
    "file_data": 4.265743301617466
```

(continues on next page)

(continued from previous page)

```
}
}
```

The events where entropy is logged will depend largely on how it's used within a rule and the rule's protocol.

For example -- this rule -- when evaluated by Suricata -- will result in the *calculated entropy* being included in the alert, flow and http events. Depending on the traffic and Suricata configuration, other event types may include the entropy value:

```
alert http any any -> any any (flow:established; file.data; entropy: value > 4.4; sid: 1;
→)
```

8.7.19 rpc

The rpc keyword can be used to match in the SUNRPC CALL on the RPC procedure numbers and the RPC version.

You can modify the keyword by using a wild-card, defined with *. With this wild-card you can match on all version and/or procedure numbers.

RPC (Remote Procedure Call) is an application that allows a computer program to execute a procedure on another computer (or address space). It is used for inter-process communication. See http://en.wikipedia.org/wiki/Inter-process_communication

Format:

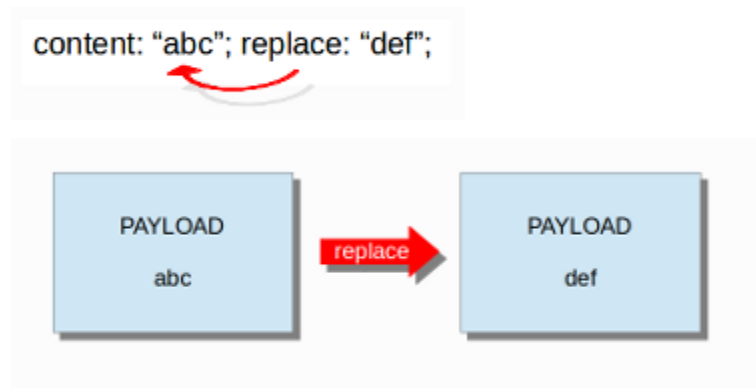
```
rpc:<application number>, [<version number>|*], [<procedure number>|*]>;
```

Example of the rpc keyword in a rule:

```
alert udp $EXTERNAL_NET any -> $HOME_NET 111 (msg:"RPC portmap request yppasswdd"; rpc:100009,*,*;
reference:bugtraq,2763; classtype:rpc-portmap-decode; sid:1296; rev:4;)
```

8.7.20 replace

The replace content modifier can only be used in IPS. It adjusts network traffic. It changes the content it follows ('abc') into another ('def'), see example:



The replace modifier has to contain as many characters as the content it replaces. It can only be used with individual packets. It will not work for *Normalized Buffers* like HTTP uri or a content match in the reassembled stream.

The checksums will be recalculated by Suricata and changed after the replace keyword is being used.

8.7.21 pcre (Perl Compatible Regular Expressions)

The keyword `pcre` matches specific on regular expressions. More information about regular expressions can be found here http://en.wikipedia.org/wiki/Regular_expression.

The complexity of `pcre` comes with a high price though: it has a negative influence on performance. So, to mitigate Suricata from having to check `pcre` often, `pcre` is mostly combined with 'content'. In that case, the content has to match first, before `pcre` will be checked.

Format of `pcre`:

```
pcre: "<regex>/opts";
```

Example of `pcre`. In this example there will be a match if the payload contains six numbers following:

```
pcre: "/[0-9]{6}/";
```

Example of `pcre` in a signature:

```
drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"ET TROJAN Likely Bot Nick in IRC (USA +..)";  
flow:established,to_server; flowbits:isset,is_proto_irc; content:"NICK "; pcre:"/NICK .*USA.*[0-9]{3,}/i"; refer-  
ence:url,doc.emergingthreats.net/2008124; classtype:trojan-activity; sid:2008124; rev:2;)
```

There are a few qualities of `pcre` which can be modified:

- By default `pcre` is case-sensitive.
- The `.` (dot) is a part of regex. It matches on every byte except for newline characters.
- By default the payload will be inspected as one line.

These qualities can be modified with the following characters:

```
i    pcre is case insensitive  
s    pcre does check newline characters  
m    can make one line (of the payload) count as two lines
```

These options are perl compatible modifiers. To use these modifiers, you should add them to `pcre`, behind regex. Like this:

```
pcre: "<regex>/i";
```

Pcre compatible modifiers

There are a few `pcre` compatible modifiers which can change the qualities of `pcre` as well. These are:

- **A:** A pattern has to match at the beginning of a buffer. (In `pcre` `^` is similar to **A**.)
- **E:** Ignores newline characters at the end of the buffer/payload.
- **G:** Inverts the greediness.

Note

The following characters must be escaped inside the content: ; \ "

PCRE extraction

It is possible to capture groups from the regular expression and log them into the alert events.

There are 3 capabilities:

- **pkt**: the extracted group is logged as `pkt` variable in `metadata.pktvars`
- **alert**: the extracted group is logged to the `alert.context` subobject
- **flow**: the extracted group is stored in a flow variable and end up in the `metadata.flowvars`

To use the feature, parameters of `pcre` keyword need to be updated. After the regular `pcre` regex and options, a comma-separated list of variable names. The prefix here is `flow:`, `pkt:` or `alert:` and the names can contain special characters now. The names map to the capturing substring expressions in order

```
pcre:"/([a-z+)\\[a-z]+\[(.+)\](.+)\[changelog$/GUR, \
flow:ua/ubuntu/repo,flow:ua/ubuntu/pkg/base, \
flow:ua/ubuntu/pkg/version";
```

This would result in the alert event having something like

```
"metadata": {
  "flowvars": [
    {"ua/ubuntu/repo": "fr"},
    {"ua/ubuntu/pkg/base": "curl"},
    {"ua/ubuntu/pkg/version": "2.2.1"}
  ]
}
```

The other events on the same flow such as the `flow` one will also have the flow vars.

If this is not wanted, you can use the `alert:` construct to only get the event in the alert

```
pcre:"/([a-z+)\\[a-z]+\[(.+)\](.+)\[changelog$/GUR, \
alert:ua/ubuntu/repo,alert:ua/ubuntu/pkg/base, \
alert:ua/ubuntu/pkg/version";
```

With that syntax, the result of the extraction will appear like

```
"alert": {
  "context": {
    "ua/ubuntu/repo": "fr",
    "ua/ubuntu/pkg/base": "curl",
    "ua/ubuntu/pkg/version": "2.2.1"
  ]
}
```

A combination of the extraction scopes can be combined.

It is also possible to extract key/value pair in the `pkt` scope. One capture would be the key, the second the value. The notation is similar to the last

```
pcre: "^/([A-Z]+) (.*)\r\n/, pkt:key,pkt:value";
```

key and value are simply hardcoded names to trigger the key/value extraction. As a consequence, they can't be used as name for the variables.

Suricata's modifiers

Suricata has its own specific pcre modifiers. These are:

- R: Match relative to the last pattern match. It is similar to distance:0;
- U: Makes pcre match on the normalized uri. It matches on the uri_buffer just like uricontent and content combined with http_uri. U can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-uri buffer. Read more about *HTTP URI Normalization*.

PAYLOAD
/index.html

content: "/index."; http_uri; content: "htm"; http_uri; distance:0;



content: "index."; http_uri; pcre: "/html?\$/UR";



content: "index."; http_uri; pcre: "/^/index\\.html?\$/U";



PAYLOAD
/index.htm

content: "/index."; http_uri; content: "htm"; http_uri; distance:0;



content: "index."; http_uri; pcre: "/html?\$/UR";

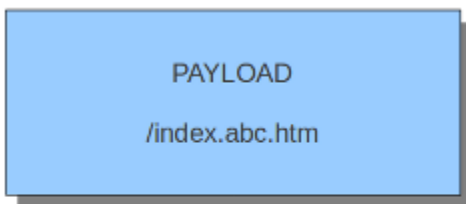


content: "index."; http_uri; pcre: "/^/index\\.html?\$/U";





content:"/index."; http_uri; content:"htm"; http_uri; distance:0; ✓
 content:"index."; http_uri; pcre:"/html?\$/UR"; ✗
 content:"index."; http_uri; pcre:"/^/index\\.html?\$/U"; ✗



content:"/index."; http_uri; content:"htm"; http_uri; distance:0; ✗
 content:"index."; http_uri; pcre:"/html?\$/UR"; ✓
 content:"index."; http_uri; pcre:"/^/index\\.html?\$/U"; ✗

- **I:** Makes pcre match on the HTTP-raw-uri. It matches on the same buffer as http_raw_uri. I can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-raw-uri buffer. Read more about [HTTP URI Normalization](#).
- **P:** Makes pcre match on the HTTP- request-body. So, it matches on the same buffer as http_client_body. P can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-request body.
- **Q:** Makes pcre match on the HTTP- response-body. So, it matches on the same buffer as http_server_body. Q can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-response body.
- **H:** Makes pcre match on the HTTP-header. H can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-header body.
- **D:** Makes pcre match on the unnormalized header. So, it matches on the same buffer as http_raw_header. D can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-raw-header.
- **M:** Makes pcre match on the request-method. So, it matches on the same buffer as http_method. M can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-method buffer.
- **C:** Makes pcre match on the HTTP-cookie. So, it matches on the same buffer as http_cookie. C can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-cookie buffer.

- S: Makes pcre match on the HTTP-stat-code. So, it matches on the same buffer as `http_stat_code`. S can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-stat-code buffer.
- Y: Makes pcre match on the HTTP-stat-msg. So, it matches on the same buffer as `http_stat_msg`. Y can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-stat-msg buffer.
- B: You can encounter B in signatures but this is just for compatibility. So, Suricata does not use B but supports it so it does not cause errors.
- O: Overrides the configures pcre match limit.
- V: Makes pcre match on the HTTP-User-Agent. So, it matches on the same buffer as `http_user_agent`. V can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-User-Agent buffer.
- W: Makes pcre match on the HTTP-Host. So, it matches on the same buffer as `http_host`. W can be combined with /R. Note that R is relative to the previous match so both matches have to be in the HTTP-Host buffer.

Changes from PCRE1 to PCRE2

The upgrade from PCRE1 to PCRE2 changes the behavior for some PCRE expressions.

- `\I` is a valid pcre in PCRE1, with a useless escape, so equivalent to `I`, but it is no longer the case in PCRE2. There are other characters than `I` exhibiting this pattern
- `[\d-a]` is a valid pcre in PCRE1, with either a digit, a dash or the character `a`, but the dash must now be escaped with PCRE2 as `[\d\-a]` to get the same behavior
- `pcre2_substring_copy_bynumber` now returns an error `PCRE2_ERROR_UNSET` instead of `pcre_copy_substring` returning no error and giving an empty string. If the behavior of some use case is no longer the expected one, please let us know.

8.8 Integer Keywords

Many keywords will match on an integer value on the network traffic. These are unsigned integers that can be 8, 16, 32 or 64 bits.

Simple example:

```
bsize:integer value;
```

The integer value can be written as base-10 like `100` or as an hexadecimal value like `0x64`.

The integer value can also have a unit/multiplier as a case-insensitive suffix: `* kb/kib : 1024 * mb/mib : 1048576 * gb/gib : 1073741824`

The most direct example is to match for equality, but there are different modes.

8.8.1 Comparison modes

Integers can be matched for

- Equality
- Inequality
- Greater than
- Less than

- Range
- Negated range
- Bitmask
- Negated Bitmask

Note

Comparisons are strict by default. Ranges are thus exclusive. That means a range between 1 and 4 will match 2 and 3, but neither 1 nor 4. Negated range !1-4 will match for 1 or below and for 4 or above.

Examples:

```

bsize:19; # equality
bsize:=0x13; # equality
bsize:!0x14; # inequality
bsize!=20; # inequality
bsize:>21; # greater than
bsize:>=21; # greater than or equal
bsize:<22; # lesser than
bsize:<=22; # lesser than or equal
bsize:19-22; # range between value1 and value2
bsize:!19-22; # negated range between value1 and value2
bsize:&0xc0=0x80; # bitmask mask is compared to value for equality
bsize:&0xc0!=0; # bitmask mask is compared to value for inequality

```

8.8.2 Enumerations

Some integers on the wire represent an enumeration, that is, some values have a string/meaning associated to it. Rules can be written using one of these strings to check for equality or inequality. This is meant to make rules more human-readable and equivalent for matching.

Examples:

```

websocket.opcode:text;
websocket.opcode:1; # behaves the same

websocket.opcode:!ping;
websocket.opcode:!9; # behaves the same

```

8.8.3 Bitmasks

Some integers on the wire represent multiple bits. Some of these bits have a string/meaning associated to it. Rules can be written using a list (comma-separated) of these strings, where each item can be negated.

There is no right shift for trailing zeros applied here (even if there is one for `byte_test` and `byte_math`). That means a rule with `websocket.flags:&0xc0=2` will be rejected as invalid as it can never match.

Examples:

```

websocket.flags:fin,!comp;
websocket.flags:&0xc0=0x80; # behaves the same

```

8.9 Transformations

Transformation keywords turn the data at a sticky buffer into something else. Some transformations support options for greater control over the transformation process

Example:

```
alert http any any -> any any (file_data; strip_whitespace; \
  content:"window.navigate("; sid:1;)
```

This example will match on traffic even if there are one or more spaces between the `navigate` and `(`.

The transforms can be chained. They are processed in the order in which they appear in a rule. Each transform's output acts as input for the next one.

Example:

```
alert http any any -> any any (http_request_line; compress_whitespace; to_sha256; \
  content:"|54A9 7A8A B09C 1B81 3725 2214 51D3 F997 F015 9DD7 049E E5AD CED3 945A FC79_
  ↪7401|"; sid:1;)
```

Note

not all sticky buffers support transformations yet

8.9.1 dotprefix

Takes the buffer, and prepends a `.` character to help facilitate concise domain checks. For example, an input string of `hello.google.com` would be modified and become `.hello.google.com`. Additionally, adding the dot allows `google.com` to match against `content:".google.com"`

Example:

```
alert dns any any -> any any (dns.query; dotprefix; \
  content:".microsoft.com"; sid:1;)
```

This example will match on `windows.update.microsoft.com` and `maps.microsoft.com.au` but not `windows.update.fakemicrosoft.com`.

This rule can be used to match on the domain only; example:

```
alert dns any any -> any any (dns.query; dotprefix; \
  content:".microsoft.com"; endswith; sid:1;)
```

This example will match on `windows.update.microsoft.com` but not `windows.update.microsoft.com.au`.

Finally, this rule can be used to match on the TLD only; example:

```
alert dns any any -> any any (dns.query; dotprefix; \
  content:".co.uk"; endswith; sid:1;)
```

This example will match on `maps.google.co.uk` but not `maps.google.co.nl`.

8.9.2 domain

Takes the buffer and extracts the domain name from it. The domain name is defined using [Mozilla's Public Suffix List](#). This implies that it is using traditional top level domain such as .com but also some specific domain like airport.aero or execute-api.cn-north-1.amazonaws.com.cn where declaration of sub domain by users below the domain is possible.

Example:

```
alert tls any any -> any any (tls.sni; domain; \
    dataset:isnet,domains,type string,load domains.lst; sid:1;)
```

This example will match on all domains contained in the file domains.lst. For example, if domains.lst contains oisf.net then webshop.oisf.net will match.

8.9.3 tld

Takes the buffer and extracts the Top Level Domain (TLD) name from it. The TLD name is defined using [Mozilla's Public Suffix List](#). This implies that it will have traditional TLD such as com but also some specific domain like airport.aero or execute-api.cn-north-1.amazonaws.com.cn where declaration of sub domain by users below the domain is possible.

Example:

```
alert tls any any -> any any (tls.sni; tld; \
    dataset:isnet,tlds,type string,load tlds.lst; sid:1;)
```

This example will match on all TLDs contained in the file tlds.lst. For example, if tlds.lst contains net then oisf.net will match.

8.9.4 strip_whitespace

Strips all whitespace as considered by the isspace() call in C.

Example:

```
alert http any any -> any any (file_data; strip_whitespace; \
    content:"window.navigate("; sid:1;)
```

8.9.5 compress_whitespace

Compresses all consecutive whitespace into a single space.

8.9.6 to_lowercase

Converts the buffer to lowercase and passes the value on.

This example alerts if http.uri contains this text has been converted to lowercase

Example:

```
alert http any any -> any any (http.uri; to_lowercase; \
    content:"this text has been converted to lowercase"; sid:1;)
```

8.9.7 to_md5

Takes the buffer, calculates the MD5 hash and passes the raw hash value on.

Example:

```
alert http any any -> any any (http_request_line; to_md5; \
  content:"|54 A9 7A 8A B0 9C 1B 81 37 25 22 14 51 D3 F9 97|"; sid:1;)
```

8.9.8 to_uppercase

Converts the buffer to uppercase and passes the value on.

This example alerts if http.uri contains THIS TEXT HAS BEEN CONVERTED TO UPPERCASE

Example:

```
alert http any any -> any any (http.uri; to_uppercase; \
  content:"THIS TEXT HAS BEEN CONVERTED TO UPPERCASE"; sid:1;)
```

8.9.9 to_sha1

Takes the buffer, calculates the SHA-1 hash and passes the raw hash value on.

Example:

```
alert http any any -> any any (http_request_line; to_sha1; \
  content:"|54A9 7A8A B09C 1B81 3725 2214 51D3 F997 F015 9DD7|"; sid:1;)
```

8.9.10 to_sha256

Takes the buffer, calculates the SHA-256 hash and passes the raw hash value on.

Example:

```
alert http any any -> any any (http_request_line; to_sha256; \
  content:"|54A9 7A8A B09C 1B81 3725 2214 51D3 F997 F015 9DD7 049E E5AD CED3 945A FC79_
  ↪7401|"; sid:1;)
```

8.9.11 pcrexform

Takes the buffer, applies the required regular expression, and outputs the *first captured expression*.

Note

this transform requires a mandatory option string containing a regular expression.

This example alerts if http.request_line contains /dropper.php Example:

```
alert http any any -> any any (msg:"HTTP with pcrexform"; http.request_line; \
  pcrexform:"[a-zA-Z]+\s+(\.*)\s+HTTP"; content:"/dropper.php"; sid:1;)
```

8.9.12 url_decode

Decodes url-encoded data, ie replacing '+' with space and '%HH' with its value. This does not decode unicode '%uZZZZ' encoding

8.9.13 xor

Takes the buffer, applies xor decoding.

Note

this transform requires a mandatory option which is the hexadecimal encoded xor key.

This example alerts if http.uri contains password= xored with 4-bytes key 0d0ac8ff Example:

```
alert http any any -> any any (msg:"HTTP with xor"; http.uri; \
  xor:"0d0ac8ff"; content:"password="; sid:1;)
```

8.9.14 header_lowercase

This transform is meant for HTTP/1 HTTP/2 header names normalization. It lowercases the header names, while keeping untouched the header values.

The implementation uses a state machine : - it lowercases until it finds ':' - it does not change until it finds a new line and switch back to first state

This example alerts for both HTTP/1 and HTTP/2 with a authorization header Example:

```
alert http any any -> any any (msg:"HTTP authorization"; http.header_names; \
  header_lowercase; content:"authorization:"; sid:1;)
```

8.9.15 strip_pseudo_headers

This transform is meant for HTTP/1 HTTP/2 header names normalization. It strips HTTP2 pseudo-headers (names and values).

The implementation just strips every line beginning by :.

This example alerts for both HTTP/1 and HTTP/2 with only a user agent Example:

```
alert http any any -> any any (msg:"HTTP ua only"; http.header_names; \
  bsize:16; content:"|0d 0a|User-Agent|0d 0a 0d 0a|"; nocase; sid:1;)
```

8.9.16 from_base64

This transform is similar to the keyword base64_decode: the buffer is decoded using the optional values for mode, offset and bytes and is available for matching on the decoded data.

After this transform completes, the buffer will contain only bytes that could be bases64-decoded. If the decoding process encountered invalid bytes, those will not be included in the buffer.

The option values must be , separated and can appear in any order.

Note

`from_base64` follows RFC 4648 by default i.e. encounter with any character that is not found in the base64 alphabet leads to rejection of that character and the rest of the string.

Format:

```
from_base64: [[bytes <value>] [, offset <offset_value> [, mode: strict|rfc4648|rfc2045]]]
from_base64
```

There are defaults for each of the options: - `bytes` defaults to the length of the input buffer - `offset` defaults to 0 and must be less than 65536 - `mode` defaults to `rfc4648`

The second example shows the rule keyword only which will cause the default values for each option to be used.

Note that both `bytes` and `offset` may be variables from `byte_extract` and/or `byte_math` in later versions of Suricata. They are not supported yet.

Mode `rfc4648` applies RFC 4648 decoding logic which is suitable for encoding binary data that can be safely sent by email, used in a URL, or included with HTTP POST requests.

Mode `rfc2045` applies RFC 2045 decoding logic which supports strings, including those with embedded spaces, line breaks, and any non base64 alphabet.

Mode `strict` will fail if an invalid character is found in the encoded bytes.

The following examples will alert when the buffer contents match (see the last `content` value for the expected strings).

This example uses the defaults and transforms `"VGhpcyBpcyBTdXJpY2F0YQ=="` to `"This is Suricata"`:

```
content: "VGhpcyBpcyBTdXJpY2F0YQ=="; from_base64; content: "This is Suricata";
```

This example transforms `"dGhpc2lzYXRlc3QK"` to `"thisisatest"`:

```
content: "/?arg=dGhpc2lzYXRlc3QK"; from_base64: offset 6, mode rfc4648; \
content: "thisisatest";
```

This example transforms `"Zm 9v Ym Fy"` to `"foobar"`:

```
content: "/?arg=Zm 9v Ym Fy"; from_base64: offset 6, mode rfc2045; \
content: "foobar";
```

8.9.17 luaxform

This transform allows a Lua script to apply a transformation to a buffer.

Lua scripts that are used for transformations *must* contain a function named `transform`. The function signature is:

```
-- transform function arguments:
-- input - the buffer to be transformed. Use #input to determine byte count
-- args - The arguments from the rule. Access each as args[0], args[1], ...
function transform(input, args)
```

Lua scripts can have other functions; Suricata only invokes the `transform` function.

Lua transforms can be passed optional arguments -- see the examples below -- but they are not required to do so. Specify comma-separated arguments in the rule, e.g:

```
luaform:transform.lua, bytes 0, offset 2
```

A Lua transform function is not invoked if the buffer is empty or the Lua framework is not accessible (rare).

Lua transform functions must return two values (see below) or the buffer is not modified.

Note that the arguments and values are passed without validation nor interpretation. There is a maximum of 10 arguments.

The Lua transform function is invoked with these parameters:

- *input* The buffer provided to the transform
- *arguments* The list of arguments.

Lua transform functions must return two values [Lua datatypes shown]:

- *buffer* [Lua string] The return buffer containing the original input buffer or buffer modified by the transform.
- *bytes* [Lua integer] Number of bytes in return buffer.

This example supplies the HTTP data to a Lua transform and the transform results are checked with *content*.

Example:

```
alert http any any -> any any (msg:"Lua Xform example"; flow:established; \
    file.data; luaform:./lua/lua-transform.lua; content: "abc"; sid: 2;)
```

This example supplies the HTTP data to a Lua transform with arguments that specify the offset and byte count for the transform. The resulting buffer is then checked with a *content* match.

Example:

```
alert http any any -> any any (msg:"Lua Xform example"; flow:established; \
    file.data; luaform:./lua/lua-transform.lua, bytes 12, offset 13; content: "abc";
    → sid: 1;)
```

The following Lua script shows a transform that handles arguments: *bytes* and *offset* and uses those values (or defaults, if there are no arguments) for applying the uppercase transform to the buffer.

```
local function get_value(item, key)
    if string.find(item, key) then
        local _, value = string.match(item, "(%a+)%s*(%d*)")
        if value ~= "" then
            return tonumber(value)
        end
    end
    return nil
end

-- Arguments supported
local bytes_key = "bytes"
local offset_key = "offset"

function transform(input, args)
    local bytes = #input
    local offset = 0
```

(continues on next page)

(continued from previous page)

```

-- Look for optional bytes and offset arguments
for i, item in ipairs(args) do
    local value = get_value(item, bytes_key)
    if value ~= nil then
        bytes = value
    else
        local value = get_value(item, offset_key)
        if value ~= nil then
            offset = value
        end
    end
end
local str_len = #input
if offset < 0 or offset > str_len then
    print("offset is out of bounds: " .. offset)
    return nil
end
str_len = str_len - offset
if bytes < 0 or bytes > str_len then
    print("invalid bytes " .. bytes .. " or bytes > length " .. bytes .. " length " ..
↪... str_len)
    return nil
end
local sub = string.sub(input, offset + 1, offset + bytes)
return string.upper(sub), bytes
end

```

8.9.18 gunzip

Takes the buffer, applies gunzip decompression.

This transform takes an optional argument which is a comma-separated list of key-values. Key and value are separated by a space. The only key being interpreted is `max-size`, which is the max output size. Default for `max-size` is 1024. If the decompressed data were to be larger than `max-size`, the transform will decompress data up to `max-size`. Value 0 is forbidden for `max-size` (there is no unlimited value).

This example alerts if `http.uri` contains base64-encoded gzipped value Example:

```

alert http any any -> any any (msg:"from_base64 + gunzip";
    http.uri; content:"/gzb64?value="; fast_pattern;
    from_base64: offset 13 ;
    gunzip: max-size 1024; content:"This is compressed then base64-encoded";
↪startswith; endswith;
    sid:2; rev:1;)

```

8.9.19 zlib_deflate

Takes the buffer, applies zlib decompression.

This transform takes an optional argument which is a comma-separated list of key-values. Key and value are separated by a space. The only key being interpreted is `max-size`, which is the max output size. Default for `max-size` is 1024. If the decompressed data were to be larger than `max-size`, the transform will decompress data up to `max-size`. Value 0 is forbidden for `max-size` (there is no unlimited value).

This example alerts if `http.uri` contains base64-encoded zlib-compressed value Example:

```
alert http any any -> any any (msg:"from_base64 + gunzip";
    http.uri; content: "/zb64?value="; fast_pattern;
    from_base64: offset 12 ;
    zlib_deflate; content: "This is compressed then base64-encoded"; startswith;
    ↪endswith;
    sid:2; rev:1;)
```

8.10 Prefiltering Keywords

8.10.1 fast_pattern

Suricata Fast Pattern Determination Explained

If the 'fast_pattern' keyword is explicitly set in a rule, Suricata will use that as the fast pattern match. The 'fast_pattern' keyword can only be set once per rule. If 'fast_pattern' is not set, Suricata automatically determines the content to use as the fast pattern match.

The following explains the logic Suricata uses to automatically determine the fast pattern match to use.

Be aware that if there are positive (i.e. non-negated) content matches, then negated content matches are ignored for fast pattern determination. Otherwise, negated content matches are considered.

The fast_pattern selection criteria are as follows:

1. Suricata first identifies all content matches that have the highest "priority" that are used in the signature. The priority is based off of the buffer being matched on and generally application layer buffers have a higher priority (lower number is higher priority). The buffer *http_method* is an exception and has lower priority than the general *content* buffer.
2. Within the content matches identified in step 1 (the highest priority content matches), the longest (in terms of character/byte length) content match is used as the fast pattern match.
3. If multiple content matches have the same highest priority and qualify for the longest length, the one with the highest character/byte diversity score ("Pattern Strength") is used as the fast pattern match. See [Appendix A](#) for details on the algorithm used to determine Pattern Strength.
4. If multiple content matches have the same highest priority, qualify for the longest length, and the same highest Pattern Strength, the buffer ("list_id") that was *registered last* is used as the fast pattern match.
5. If multiple content matches have the same highest priority, qualify for the longest length, the same highest Pattern Strength, and have the same list_id (i.e. are looking in the same buffer), then the one that comes first (from left-to-right) in the rule is used as the fast pattern match.

It is worth noting that for content matches that have the same priority, length, and Pattern Strength, 'http_stat_msg', 'http_stat_code', and 'http_method' take precedence over regular 'content' matches.

Appendices

Appendix A - Pattern Strength Algorithm

From detect-engine-mpm.c. Basically the Pattern Strength "score" starts at zero and looks at each character/byte in the passed in byte array from left to right. If the character/byte has not been seen before in the array, it adds 3 to the score if it is an alpha character; else it adds 4 to the score if it is a printable character, 0x00, 0x01, or 0xFF; else it adds 6 to the score. If the character/byte has been seen before it adds 1 to the score. The final score is returned.

```

/** \brief Predict a strength value for patterns
 *
 * Patterns with high character diversity score higher.
 * Alpha chars score not so high
 * Other printable + a few common codes a little higher
 * Everything else highest.
 * Longer patterns score better than short patterns.
 *
 * \param pat pattern
 * \param patlen length of the pattern
 *
 * \retval s pattern score
 */
uint32_t PatternStrength(uint8_t *pat, uint16_t patlen) {
    uint8_t a[256];
    memset(&a, 0, sizeof(a));
    uint32_t s = 0;
    uint16_t u = 0;
    for (u = 0; u < patlen; u++) {
        if (a[pat[u]] == 0) {
            if (isalpha(pat[u]))
                s += 3;
            else if (isprint(pat[u]) || pat[u] == 0x00 || pat[u] == 0x01 || pat[u] == 0xFF)
                s += 4;
            else
                s += 6;
            a[pat[u]] = 1;
        } else {
            s++;
        }
    }
    return s;
}

```

Only one content of a signature will be used in the Multi Pattern Matcher (MPM). If there are multiple contents, then Suricata uses the 'strongest' content. This means a combination of length, how varied a content is, and what buffer it is looking in. Generally, the longer and more varied the better. For full details on how Suricata determines the fast pattern match, see [Suricata Fast Pattern Determination Explained](#).

Sometimes a signature writer concludes he wants Suricata to use another content than it does by default.

For instance:

```

User-agent: Mozilla/5.0 Badness;

content:"User-Agent|3A|";
content:"Badness"; distance:0;

```

In this example you see the first content is longer and more varied than the second one, so you know Suricata will use this content for the MPM. Because 'User-Agent:' will be a match very often, and 'Badness' appears less often in network traffic, you can make Suricata use the second content by using 'fast_pattern'.

```
content:"User-Agent|3A|";
content:"Badness"; distance:0; fast_pattern;
```

The keyword `fast_pattern` modifies the content previous to it.

```
content:"User-Agent|3A|";
content:"Badness"; distance:0; fast_pattern;
```



Fast-pattern can also be combined with all previous mentioned keywords, and all mentioned HTTP-modifiers.

fast_pattern:only

Sometimes a signature contains only one content. In that case it is not necessary Suricata will check it any further after a match has been found in MPM. If there is only one content, the whole signature matches. Suricata notices this automatically. In some signatures this is still indicated with `'fast_pattern:only'`. Although Suricata does not need `fast_pattern:only`, it does support it.

fast_pattern:'chop'

If you do not want the MPM to use the whole content, you can use `fast_pattern 'chop'`.

For example:

```
content: "aaaaaaaaabc"; fast_pattern:8,4;
```

This way, MPM uses only the last four characters.

8.10.2 prefilter

The prefilter engines for other non-MPM keywords can be enabled in specific rules by using the `'prefilter'` keyword.

In the following rule the TTL test will be used in prefiltering instead of the single byte pattern:

```
alert ip any any -> any any (ttl:123; prefilter; content:"a"; sid:1;)
```

For more information on how to configure the prefilter engines, see [Prefilter Engines](#)

8.11 Flow Keywords

8.11.1 flowbits

Flowbits consists of two parts. The first part describes the action it is going to perform, the second part is the name of the flowbit.

There are multiple packets that belong to one flow. Suricata keeps those flows in memory. For more information see [Flow Settings](#).

Flowbits can make sure an alert will be generated when for example two different packets match. An alert will only be generated when both packets match. So, when the second packet matches, Suricata has to know if the first packet was a match too. Flowbits mark the flow if a packet matches so Suricata 'knows' it should generate an alert when the second packet matches as well.

Flowbits have different actions. These are:

flowbits: set, name

Will set the condition/'name', if present, in the flow.

flowbits: isset, name

Can be used in the rule to make sure it generates an alert when the rule matches and the condition is set in the flow.

flowbits: toggle, name

Reverses the present setting. So for example if a condition is set, it will be unset and vice-versa.

flowbits: unset, name

Can be used to unset the condition in the flow.

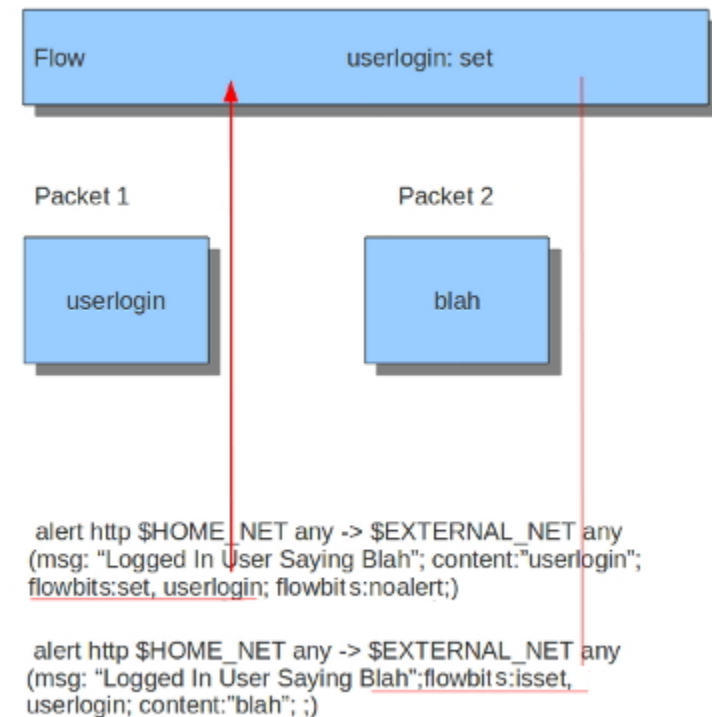
flowbits: isnotset, name

Can be used in the rule to make sure it generates an alert when it matches and the condition is not set in the flow.

flowbits: noalert

No alert will be generated by this rule.

Example:



When you take a look at the first rule you will notice it would generate an alert if it would match, if it were not for the 'flowbits: noalert' at the end of that rule.

The purpose of this rule is to check for a match on 'userlogin' and mark that in the flow. So, there is no need to generate an alert. The second rule has no effect without the first rule. If the first rule matches, the flowbit sets that specific condition to be present in the flow. Now the second rule can be checked whether or not the previous packet fulfills the first condition. If the second rule matches now, an alert will be generated.

Note

flowbit names are case-sensitive.

Note

It is possible to use flowbits several times in a rule and combine the different functions.

Note

It is possible to perform an *OR* operation with flowbits using the | (pipe).

alert http any any -> any any (msg:"User1 or User2 logged in"; content:"login"; flowbits:isset,user1|user2; sid:1;)

8.11.2 flow

The flow keyword can be used to match on direction of the flow, so to/from client or to/from server. It can also match if the flow is established or not. The flow keyword can also be used to say the signature has to match on stream only (only_stream) or on packet only (no_stream).

So with the flow keyword you can match on:

to_client

Match on packets from server to client.

to_server

Match on packets from client to server.

from_client

Match on packets from client to server (same as to_server).

from_server

Match on packets from server to client (same as to_client).

established

Match on established connections.

not_established

Match on packets that are not part of an established connection.

stateless

Match on packets that are part of a flow, regardless of connection state. (This means that packets that are not seen as part of a flow won't match).

only_stream

Match on packets that have been reassembled by the stream engine.

no_stream

Match on packets that have not been reassembled by the stream engine. Will not match packets that have been reassembled.

only_frag

Match packets that have been reassembled from fragments.

no_frag

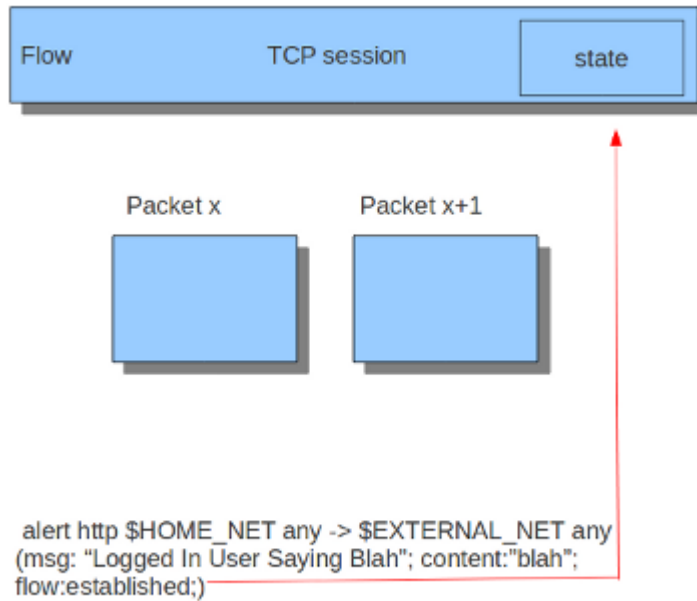
Match packets that have not been reassembled from fragments.

Multiple flow options can be combined, for example:

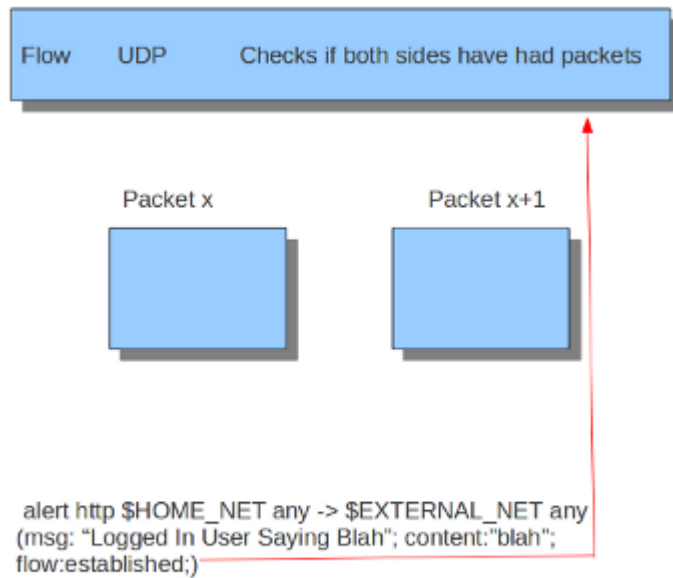
```
flow:to_client, established
flow:to_server, established, only_stream
flow:to_server, not_established, no_frag
```

The determination of *established* depends on the protocol:

- For TCP a connection will be established after a three way handshake.



- For other protocols (for example UDP), the connection will be considered established after seeing traffic from both sides of the connection.



8.11.3 flowint

Flowint allows storage and mathematical operations using variables. It operates much like flowbits but with the addition of mathematical capabilities and the fact that an integer can be stored and manipulated, not just a flag set. We can use this for a number of very useful things, such as counting occurrences, adding or subtracting occurrences, or doing thresholding within a stream in relation to multiple factors. This will be expanded to a global context very soon, so users can perform these operations between streams.

The syntax is as follows:

```
flowint: name, modifier[, value];
```

Define a var (not required), or check that one is set or not set.

```

flowint: name, < +,-,=,>,<,>=,<=,==, != >, value;
flowint: name, (isset|notset|isnotset);

```

Compare or alter a var. Add, subtract, compare greater than or less than, greater than or equal to, and less than or equal to are available. The item to compare with can be an integer or another variable.

For example, if you want to count how many times a username is seen in a particular stream and alert if it is over 5.

```

alert tcp any any -> any any (msg:"Counting Usernames"; content:"jonkman"; \
    flowint: usernamecount, +, 1; noalert;)

```

This will count each occurrence and increment the var usernamecount and not generate an alert for each.

Now say we want to generate an alert if there are more than five hits in the stream.

```
alert tcp any any -> any any (msg:"More than Five Usernames!"; content:"jonkman"; \
    flowint: usernamecount, +, 1; flowint:usernamecount, >, 5;)
```

So we'll get an alert ONLY if usernamecount is over five.

So now let's say we want to get an alert as above but NOT if there have been more occurrences of that username logging out. Assuming this particular protocol indicates a log out with "jonkman logout", let's try:

```
alert tcp any any -> any any (msg:"Username Logged out"; content:"logout jonkman"; \
    flowint: usernamecount, -, 1; flowint:usernamecount, >, 5;)
```

So now we'll get an alert ONLY if there are more than five active logins for this particular username.

This is a rather simplistic example, but I believe it shows the power of what such a simple function can do for rule writing. I see a lot of applications in things like login tracking, IRC state machines, malware tracking, and brute force login detection.

Let's say we're tracking a protocol that normally allows five login fails per connection, but we have vulnerability where an attacker can continue to login after that five attempts and we need to know about it.

```
alert tcp any any -> any any (msg:"Start a login count"; content:"login failed"; \
    flowint:loginfail, notset; flowint:loginfail, =, 1; noalert;)
```

So we detect the initial fail if the variable is not yet set and set it to 1 if so. Our first hit.

```
alert tcp any any -> any any (msg:"Counting Logins"; content:"login failed"; \
    flowint:loginfail, isset; flowint:loginfail, +, 1; noalert;)
```

We are now incrementing the counter if it's set.

```
alert tcp any any -> any any (msg:"More than Five login fails in a Stream"; \
    content:"login failed"; flowint:loginfail, isset; flowint:loginfail, >, 5;)
```

Now we'll generate an alert if we cross five login fails in the same stream.

But let's also say we also need alert if there are two successful logins and a failed login after that.

```
alert tcp any any -> any any (msg:"Counting Good Logins"; \
    content:"login successful"; flowint:loginsuccess, +, 1; noalert;)
```

Here we're counting good logins, so now we'll count good logins relevant to fails:

```
alert tcp any any -> any any (msg:"Login fail after two successes"; \
    content:"login failed"; flowint:loginsuccess, isset; \
    flowint:loginsuccess, =, 2;)
```

Here are some other general examples:

```
alert tcp any any -> any any (msg:"Setting a flowint counter"; content:"GET"; \
    flowint:myvar, notset; flowint:maxvar,notset; \
    flowint:myvar,=,1; flowint: maxvar,=,6;)
```

```
alert tcp any any -> any any (msg:"Adding to flowint counter"; \
    content:"Unauthorized"; flowint:myvar,isset; flowint: myvar,+,2;)
```

```
alert tcp any any -> any any (msg:"when flowint counter is 3 create new counter"; \
  content:"Unauthorized"; flowint:myvar, isset; flowint:myvar,==,3; \
  flowint:cntpackets,notset; flowint:cntpackets, =, 0;)
```

```
alert tcp any any -> any any (msg:"count the rest without generating alerts"; \
  flowint:cntpackets,isset; flowint:cntpackets, +, 1; noalert;)
```

```
alert tcp any any -> any any (msg:"fire this when it reach 6";
  flowint: cntpackets, isset;
  flowint: maxvar,isset; flowint: cntpackets, ==, maxvar;)
```

8.11.4 stream_size

The stream size option matches on traffic according to the registered amount of bytes by the sequence numbers. There are several modifiers to this keyword:

```
>      greater than
<      less than
=      equal
!=     not equal
>=     greater than or equal
<=     less than or equal
```

Format

```
stream_size:<server|client|both|either>, <modifier>, <number>;
```

Example of the stream-size keyword in a rule:

```
alert tcp any any -> any any (stream_size:both, >, 5000; sid:1;)
```

8.11.5 flow.age

Flow age in seconds (integer) This keyword does not wait for the end of the flow, but will be checked at each packet.

flow.age uses an *unsigned 32-bit integer*.

Syntax:

```
flow.age: [op]<number>
```

The time can be matched exactly, or compared using the `_op_` setting:

```
flow.age:3      # exactly 3
flow.age:<3     # smaller than 3 seconds
flow.age:>=2    # greater or equal than 2 seconds
```

Signature example:

```
alert tcp any any -> any any (msg:"Flow longer than one hour"; flow.age:>3600; flowbits:␣
  ␣isnotset, onehourflow; flowbits: onehourflow, name; sid:1; rev:1;)
```

In this example, we combine *flow.age* and *flowbits* to get an alert on the first packet after the flow's age is older than one hour.

8.11.6 flow.pkts

Flow number of packets (integer) This keyword does not wait for the end of the flow, but will be checked at each packet.

flow.pkts uses an *unsigned 32-bit integer* and supports following directions:

- toclient
- toserver
- either

Syntax:

```
flow.pkts:<direction>,[op]<number>
```

The number of packets can be matched exactly, or compared using the `_op_` setting:

```
flow.pkts:toclient,3    # exactly 3
flow.pkts:toserver,<3   # smaller than 3
flow.pkts:either,>=2    # greater than or equal to 2
```

Signature example:

```
alert ip any any -> any any (msg:"Flow has 20 packets in toclient dir"; flow.
->pkts:toclient,20; sid:1;)
```

Note

Suricata also supports `flow.pkts_toclient` and `flow.pkts_toserver` keywords for `flow.pkts:toclient` and `flow.pkts:toserver` respectively but that is not the preferred syntax.

8.11.7 flow.bytes

Flow number of bytes (integer) This keyword does not wait for the end of the flow, but will be checked at each packet.

flow.bytes uses an *unsigned 64-bit integer* and supports following directions:

- toclient
- toserver
- either

Syntax:

```
flow.bytes:<direction>,[op]<number>
```

The number of bytes can be matched exactly, or compared using the `_op_` setting:

```
flow.bytes:toclient,3    # exactly 3
flow.bytes:toserver,<3   # smaller than 3
flow.bytes:either,>=2    # greater than or equal to 2
```

Signature example:

```
alert ip any any -> any any (msg:"Flow has less than 2000 bytes in toserver dir"; flow.
->bytes:toserver,<2000; sid:1;)
```

Note

Suricata also supports `flow.bytes_toclient` and `flow.bytes_toserver` keywords for `flow.bytes:toclient` and `flow.bytes:toserver` respectively but that is not the preferred syntax.

8.12 Bypass Keyword

Suricata has a `bypass` keyword that can be used in signatures to exclude traffic from further evaluation.

The `bypass` keyword is useful in cases where there is a large flow expected (e.g. Netflix, Spotify, YouTube).

The `bypass` keyword is considered a post-match keyword.

8.12.1 bypass

Bypass a flow on matching http traffic.

```
alert http any any -> any any (http.host; content:"suricata.io"; bypass; sid:10001; rev:1;)
```

8.13 HTTP Keywords

Using the HTTP specific sticky buffers (see *Modifier Keywords*) provides a way to efficiently inspect the specific fields of HTTP protocol communications. After specifying a sticky buffer in a rule it should be followed by one or more *Payload Keywords* or using *pcr* (*Perl Compatible Regular Expressions*).

8.13.1 HTTP Primer

HTTP is considered a client-server or request-response protocol. A client requests resources from a server and a server responds to the request.

In versions of HTTP prior to version 2 a client request could look like:

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

Example signature that would alert on the above request.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Request Example"; flow:established,to_server;
http.method; content:"GET"; http.uri; content:"/index.html"; bsize:11; http.protocol; content:"HTTP/1.1"; bsize:8;
http.user_agent; content:"Mozilla/5.0"; bsize:11; http.host; content:"suricata.io"; bsize:11; classtype:bad-unknown;
sid:25; rev:1;)
```

In versions of HTTP prior to version 2 a server response could look like:

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 258
Date: Thu, 14 Dec 2023 20:22:41 GMT
Server: nginx/0.8.54
Connection: Close
```

Example signature that would alert on the above response.

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Stat Code Example"; flow:established,to_client;  
http.stat_code; content:"200"; bsize:8; http.content_type; content:"text/html"; bsize:9; classtype:bad-unknown; sid:30;  
rev:1;)
```

Request Keywords:

- *file.name*
- *http.accept*
- *http.accept_enc*
- *http.accept_lang*
- *http.host*
- *http.host.raw*
- *http.method*
- *http.referer*
- *http.request_body*
- *http.request_header*
- *http.request_line*
- *http.uri*
- *http.uri.raw*
- *http.user_agent*
- *urilen*

Response Keywords:

- *http.location*
- *http.response_body*
- *http.response_header*
- *http.response_line*
- *http.server*
- *http.stat_code*
- *http.stat_msg*

Request or Response Keywords:

- *file.data*
- *http.connection*
- *http.content_len*
- *http.content_type*
- *http.cookie*
- *http.header*
- *http.header.raw*
- *http.header_names*

- *http.protocol*
- *http.start*

8.13.2 Normalization

There are times when Suricata performs formatting/normalization changes to traffic that is seen.

Duplicate Header Names

If there are multiple values for the same header name, they are concatenated with a comma and space (" ") between each value. More information can be found in RFC 2616 <https://www.rfc-editor.org/rfc/rfc2616.html#section-4.2>

In the example below, notice that the User-Agent header, regardless of the letter casing is evaluated as the same header. The normalized header evaluation leads to the concatenated header values as described in the RFC above.

Example Duplicate HTTP Header:

```
GET / HTTP/1.1
Host: suricata.io
User-Agent: Mozilla/5.0
User-agent: Chrome/121.0.0
```

```
alert http $HOME_NET -> $EXTERNAL_NET (msg:"Example Duplicate Header"; flow:established,to_server;
http.user_agent; content:"Mozilla/5.0, Chrome/121.0.0"; classtype:bad-unknown; sid:103; rev:1;)
```

8.13.3 file.name

The `file.name` keyword can be used with HTTP requests.

It is possible to use any of the *Payload Keywords* with the `file.name` keyword.

Example HTTP Request:

```
GET /picture.jpg HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP file.name Example"; flow:established,to_client;
file.name; content:"picture.jpg"; classtype:bad-unknown; sid:129; rev:1;)
```

Note

Additional information can be found at *File Keywords*

8.13.4 http.accept

The `http.accept` keyword is used to match on the Accept field that can be present in HTTP request headers.

It is possible to use any of the *Payload Keywords* with the `http.accept` keyword.

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Accept: */*
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Accept Example"; flow:established,to_server;  
http.accept; content:"*/*"; bsize:3; classtype:bad-unknown; sid:91; rev:1;)
```

Note

`http.accept` does not include the leading space or trailing `\r\n`

Note

`http.accept` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.5 http.accept_enc

The `http.accept_enc` keyword is used to match on the Accept-Encoding field that can be present in HTTP request headers.

It is possible to use any of the [Payload Keywords](#) with the `http.accept_enc` keyword.

Example HTTP Request:

```
GET /index.html HTTP/1.1  
User-Agent: Mozilla/5.0  
Accept-Encoding: gzip, deflate  
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Accept-Encoding Example";  
flow:established,to_server; http.accept_enc; content:"gzip, deflate"; bsize:13; classtype:bad-unknown; sid:92;  
rev:1;)
```

Note

`http.accept_enc` does not include the leading space or trailing `\r\n`

Note

`http.accept_enc` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.6 http.accept_lang

The `http.accept_lang` keyword is used to match on the Accept-Language field that can be present in HTTP request headers.

It is possible to use any of the [Payload Keywords](#) with the `http.accept_lang` keyword.

Example HTTP Request:

```
GET /index.html HTTP/1.1  
User-Agent: Mozilla/5.0
```

(continues on next page)

(continued from previous page)

```
Accept-Language: en-US
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Accept-Encoding Example";
flow:established,to_server; http.accept_lang; content:"en-US"; bsize:5; classtype:bad-unknown; sid:93; rev:1;)
```

Note

`http.accept_lang` does not include the leading space or trailing `\r\n`

Note

`http.accept_lang` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.7 http.host

Matching on the HTTP host name has two options in Suricata, the `http.host` and the `http.host.raw` sticky buffers. It is possible to use any of the [Payload Keywords](#) with both `http.host` keywords.

Note

The `http.host` keyword normalizes the host header contents. If a host name has uppercase characters, those would be changed to lowercase.

Normalization Example:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: SuRiCaTa.Io
```

In the above example the host buffer would contain *suricata.io*.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Host Example"; flow:established,to_server;
http.host; content:"suricata.io"; bsize:11; classtype:bad-unknown; sid:123; rev:1;)
```

Note

The `nocase` keyword is no longer allowed since the host names are normalized to contain only lowercase letters.

Note

`http.host` does not contain the port associated with the host (i.e. *suricata.io:1234*). To match on the host and port or negate a host and port use [http.host.raw](#).

Note

`http.host` does not include the leading space or trailing `\r\n`

Note

The `http.host` and `http.host.raw` buffers are populated from either the URI (if the full URI is present in the request like in a proxy request) or the HTTP Host header. If both are present, the URI is used.

Note

`http.host` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.8 `http.host.raw`

The `http.host.raw` buffer matches on HTTP host content but does not have any normalization performed on the buffer contents (see [http.host](#))

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: SuRiCaTa.Io:8445
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Host Raw Example"; flow:established,to_server;
http.host.raw; content:"SuRiCaTa.Io|3a|8445"; bsize:16; classtype:bad-unknown; sid:124; rev:1;)
```

Note

`http.host.raw` does not include the leading space or trailing `\r\n`

Note

The `http.host` and `http.host.raw` buffers are populated from either the URI (if the full URI is present in the request like in a proxy request) or the HTTP Host header. If both are present, the URI is used.

Note

`http.host.raw` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.9 http.method

The `http.method` keyword matches on the method/verb used in an HTTP request. HTTP request methods can be any of the following:

- GET
- POST
- HEAD
- OPTIONS
- PUT
- DELETE
- TRACE
- CONNECT
- PATCH

It is possible to use any of the [Payload Keywords](#) with the `http.method` keyword.

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Request Example"; flow:established,to_server;
http.method; content:"GET"; classtype:bad-unknown; sid:2; rev:1;)
```

8.13.10 http.referer

The `http.referer` keyword is used to match on the Referer field that can be present in HTTP request headers.

It is possible to use any of the [Payload Keywords](#) with the `http.referer` keyword.

Example HTTP Request:

```
GET / HTTP/1.1
Host: suricata.io
Referer: https://suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Referer Example"; flow:established,to_server;
http.referer; content:"http|3a 2f 2fsuricata.io"; bsize:19; classtype:bad-unknown; sid:200; rev:1;)
```

Note

`http.referer` does not include the leading space or trailing `\r\n`

Note

`http.referer` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.11 http.request_body

The `http.request_body` keyword is used to match on the HTTP request body that can be present in an HTTP request. It is possible to use any of the [Payload Keywords](#) with the `http.request_body` keyword.

Example HTTP Request:

```
POST /suricata.php HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Host: suricata.io
Content-Length: 23
Connection: Keep-Alive
```

Suricata request body

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Request Body Example";
flow:established,to_server; http.request_body; content:"Suricata request body"; classtype:bad-unknown; sid:115;
rev:1;)
```

Note

How much of the request/client body is inspected is controlled in the [libhttp configuration section](#) via the `request-body-limit` setting.

Note

`http.request_body` replaces the previous keyword name, `http_client_body`. `http_client_body` can still be used but it is recommended that rules be converted to use `http.request_body`.

8.13.12 http.request_header

The `http.request_header` keyword is used to match on the name and value of a HTTP/1 or HTTP/2 request.

It is possible to use any of the [Payload Keywords](#) with the `http.request_header` keyword.

For HTTP/2, the header name and value get concatenated by ":" (colon and space). The colon and space are commonly noted with the hexadecimal format `|3a 20|` within signatures.

To detect if an HTTP/2 header name contains a ":" (colon), the keyword [http2.header_name](#) can be used.

Example HTTP/1 Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Request Example"; flow:established,to_server;
http.request_header; content:"Host|3a 20|suricata.io"; classtype:bad-unknown; sid:126; rev:1;)
```

Note

`http.request_header` does not include the trailing `\r\n`

8.13.13 `http.request_line`

The `http.request_line` keyword is used to match on the entire contents of the HTTP request line.

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Request Example"; flow:established,to_server;
http.request_line; content:"GET /index.html HTTP/1.1"; bsize:24; classtype:bad-unknown; sid:60; rev:1;)
```

Note

`http.request_line` does not include the trailing `\r\n`

8.13.14 `http.uri`

Matching on the HTTP URI buffer has two options in Suricata, the `http.uri` and the `http.uri.raw` sticky buffers.

It is possible to use any of the *Payload Keywords* with both `http.uri` keywords.

The `http.uri` keyword normalizes the URI buffer. For example, if a URI has two leading `//`, Suricata will normalize the URI to a single leading `/`.

Normalization Example:

```
GET //index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

In this case `//index.html` would be normalized to `/index.html`.

Normalized HTTP Request Example:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP URI Example"; flow:established,to_server;
http.uri; content:"/index.html"; bsize:11; classtype:bad-unknown; sid:3; rev:1;)
```

8.13.15 `http.uri.raw`

The `http.uri.raw` buffer matches on HTTP URI content but does not have any normalization performed on the buffer contents. (see *[http.uri](#)*)

Abnormal HTTP Request Example:

```
GET //index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP URI Raw Example"; flow:established,to_server;
http.uri.raw; content:"//index.html"; bsize:12; classtype:bad-unknown; sid:4; rev:1;)
```

Note

The `http.uri.raw` keyword/buffer does not allow for spaces.

Example Request:

```
GET /example spaces HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

`http.uri.raw` would be populated with `/example`

`http.protocol` would be populated with `spaces HTTP/1.1`

Reference: <https://redmine.openinfosecfoundation.org/issues/2881>

8.13.16 http.user_agent

The `http.user_agent` keyword is used to match on the User-Agent field that can be present in HTTP request headers.

It is possible to use any of the *Payload Keywords* with the `http.user_agent` keyword.

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Cookie: PHPSESSION=123
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP User-Agent Example";
flow:established,to_server; http.user_agent; content:"Mozilla/5.0"; bsize:11; classtype:bad-unknown; sid:90;
rev:1;)
```

Note

`http.user_agent` does not include the leading space or trailing `\r\n`

Note

Using the `http.user_agent` generally provides better performance than using `http.header`.

Note

`http.user_agent` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.17 urilen

The `urilen` keyword is used to match on the length of the normalized request URI. It is possible to use the `<` and `>` operators, which indicate respectively *less than* and *larger than*.

`urilen` uses an *unsigned 64-bit integer*.

The `urilen` keyword does not require a content match on the `http.uri` buffer or the `http.uri.raw` buffer.

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Request"; flow:established,to_server; urilen:11;
http.method; content:"GET"; classtype:bad-unknown; sid:40; rev:1;)
```

The above signature would match on any HTTP GET request that has a URI length of 11, regardless of the content or structure of the URI.

The following signatures would all alert on the example request above as well and show the different `urilen` options.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"urilen greater than 10"; flow:established,to_server;
urilen:>10; classtype:bad-unknown; sid:41; rev:1;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"urilen less than 12"; flow:established,to_server;
urilen:<12; classtype:bad-unknown; sid:42; rev:1;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"urilen greater/less than example";
flow:established,to_server; urilen:10<>12; classtype:bad-unknown; sid:43; rev:1;)
```

8.13.18 http.location

The `http.location` keyword is used to match on the HTTP response location header contents.

It is possible to use any of the [Payload Keywords](#) with the `http.location` keyword.

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
Location: suricata.io
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Location Example"; flow:established,to_client;
http.location; content:"suricata.io"; bsize:11; classtype:bad-unknown; sid:122; rev:1;)
```

Note

`http.location` does not include the leading space or trailing `\r\n`

Note

`http.location` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.19 http.response_body

The `http.response_body` keyword is used to match on the HTTP response body.

It is possible to use any of the [Payload Keywords](#) with the `http.response_body` keyword.

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
```

Server response body

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Response Body Example";
flow:established,to_client; http.response_body; content:"Server response body"; classtype:bad-unknown; sid:120;
rev:1;)
```

Note

`http.response_body` will match on gzip decoded data just like [file.data](#) does.

Note

How much of the response/server body is inspected is controlled in your [libhttp configuration section](#) via the `response-body-limit` setting.

Note

`http.response_body` replaces the previous keyword name, `http_server_body`. `http_server_body` can still be used but it is recommended that rules be converted to use `http.response_body`.

8.13.20 http.response_header

The `http.response_header` keyword is used to match on the name and value of an HTTP/1 or HTTP/2 request.

It is possible to use any of the [Payload Keywords](#) with the `http.response_header` keyword.

For HTTP/2, the header name and value get concatenated by ":" (colon and space). The colon and space are commonly noted with the hexadecimal format `|3a 20|` within signatures.

To detect if an HTTP/2 header name contains a ":" (colon), the keyword [http2.header_name](#) can be used.

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
```

(continues on next page)

(continued from previous page)

```
Server: nginx/0.8.54  
Location: suricata.io
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Response Example"; flow:established,to_client;  
http.response_header; content:"Location|3a 20|suricata.io"; classtype:bad-unknown; sid:127; rev:1;)
```

8.13.21 http.response_line

The `http.response_line` keyword is used to match on the entire HTTP response line.

It is possible to use any of the *Payload Keywords* with the `http.response_line` keyword.

Example HTTP Response:

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Server: nginx/0.8.54
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Response Line Example";  
flow:established,to_client; http.response_line; content:"HTTP/1.1 200 OK"; classtype:bad-unknown; sid:119;  
rev:1;)
```

Note

`http.response_line` does not include the trailing `\r\n`

8.13.22 http.server

The `http.server` keyword is used to match on the HTTP response server header contents.

It is possible to use any of the *Payload Keywords* with the `http.server` keyword.

Example HTTP Response:

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Server: nginx/0.8.54
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Server Example"; flow:established,to_client;  
http.server; content:"nginx/0.8.54"; bsize:12; classtype:bad-unknown; sid:121; rev:1;)
```

Note

`http.server` does not include the leading space or trailing `\r\n`

Note

`http.server` can have additional formatting/normalization applied to buffer contents, see *Normalization* for additional details.

8.13.23 http.stat_code

The `http.stat_code` keyword is used to match on the HTTP status code that can be present in an HTTP response. It is possible to use any of the *Payload Keywords* with the `http.stat_code` keyword.

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Stat Code Response Example";
flow:established,to_client; http.stat_code; content:"200"; classtype:bad-unknown; sid:117; rev:1;)
```

Note

`http.stat_code` does not include the leading or trailing space

8.13.24 http.stat_msg

The `http.stat_msg` keyword is used to match on the HTTP status message that can be present in an HTTP response. For HTTP/2, an empty buffer is returned by Suricata. See rfc 7540 section 8.1.2.4. about Response Pseudo-Header Fields.

It is possible to use any of the *Payload Keywords* with the `http.stat_msg` keyword.

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Stat Message Response Example";
flow:established,to_client; http.stat_msg; content:"OK"; classtype:bad-unknown; sid:118; rev:1;)
```

Note

`http.stat_msg` does not include the leading space or trailing `\r\n`

Note

`http.stat_msg` will always be empty when used with HTTP/2

8.13.25 file.data

With `file.data`, the HTTP response body is inspected, just like with `http.response_body`. `file.data` also works for HTTP request body and can be used in protocols other than HTTP.

It is possible to use any of the *Payload Keywords* with the `file.data` keyword.

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
```

Server response body

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP file.data Example"; flow:established,to_client;
file.data; content:"Server response body"; classtype:bad-unknown; sid:128; rev:1;)
```

The body of an HTTP response can be very large, therefore the response body is inspected in definable chunks.

How much of the response/server body is inspected is controlled in the [libhttp configuration section](#) via the response-body-limit setting.

Note

If the HTTP body is a flash file compressed with 'deflate' or 'lzma', it can be decompressed and `file.data` can match on the decompressed data. Flash decompression must be enabled under 'libhttp' configuration:

```
# Decompress SWF files.
# 2 types: 'deflate', 'lzma', 'both' will decompress deflate and lzma
# compress-depth:
# Specifies the maximum amount of data to decompress,
# set 0 for unlimited.
# decompress-depth:
# Specifies the maximum amount of decompressed data to obtain,
# set 0 for unlimited.
swf-decompression:
  enabled: yes
  type: both
  compress-depth: 0
  decompress-depth: 0
```

Note

`file.data` replaces the previous keyword name, `file_data`. `file_data` can still be used but it is recommended that rules be converted to use `file.data`.

Note

If an HTTP body is using gzip or deflate, `file.data` will match on the decompressed data.

Note

Negated matching is affected by the chunked inspection. E.g. `'content:!<html';` could not match on the first chunk, but would then possibly match on the 2nd. To avoid this, use a depth setting. The depth setting takes the body size into account. Assuming that the `response-body-minimal-inspect-size` is bigger than 1k, `'content:!<html'; depth:1024;` can only match if the pattern `<html` is absent from the first inspected chunk.

Note

Additional information can be found at [File Keywords](#)

Note

`file.data` supports multiple buffer matching, see [Multiple Buffer Matching](#).

8.13.26 http.connection

The `http.connection` keyword is used to match on the Connection field that can be present in HTTP request or response headers.

It is possible to use any of the [Payload Keywords](#) with the `http.connection` keyword.

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Accept-Language: en-US
Host: suricata.io
Connection: Keep-Alive
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Connection Example";
flow:established,to_server; http.connection; content:"Keep-Alive"; bsize:10; classtype:bad-unknown; sid:94;
rev:1;)
```

Note

`http.connection` does not include the leading space or trailing `\r\n`

Note

`http.connection` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.27 http.content_len

The `http.content_len` keyword is used to match on the Content-Length field that can be present in HTTP request or response headers. Use `flow:to_server` or `flow:to_client` to force inspection of the request or response respectively.

It is possible to use any of the [Payload Keywords](#) with the `http.content_len` keyword.

Example HTTP Request:

```
POST /suricata.php HTTP/1.1
Content-Type: multipart/form-data; boundary=-----123
Host: suricata.io
Content-Length: 100
Connection: Keep-Alive
```

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
Connection: Close
Content-Length: 20
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Content-Length Request Example";
flow:established,to_server; http.content_len; content:"100"; bsize:3; classtype:bad-unknown; sid:97; rev:1;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Content-Length Response Example";
flow:established,to_client; http.content_len; content:"20"; bsize:2; classtype:bad-unknown; sid:98; rev:1;)
```

To do numeric evaluation of the content length, *byte_test* can be used.

If we want to match on an HTTP request content length equal to and greater than 100 we could use the following signature.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Content-Length Request Byte Test Example";
flow:established,to_server; http.content_len; byte_test:0,>=,100,0,string,dec; classtype:bad-unknown; sid:99; rev:1;)
```

Note

`http.content_len` does not include the leading space or trailing `\r\n`

8.13.28 http.content_type

The `http.content_type` keyword is used to match on the Content-Type field that can be present in HTTP request or response headers. Use `flow:to_server` or `flow:to_client` to force inspection of the request or response respectively.

It is possible to use any of the *Payload Keywords* with the `http.content_type` keyword.

Example HTTP Request:

```
POST /suricata.php HTTP/1.1
Content-Type: multipart/form-data; boundary=-----123
Host: suricata.io
Content-Length: 100
Connection: Keep-Alive
```

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
Connection: Close
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Content-Type Request Example";
flow:established,to_server; http.content_type; content:"multipart/form-data[3b 20]"; startswith; classtype:bad-unknown; sid:95; rev:1;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Content-Type Response Example";
flow:established,to_client; http.content_type; content:"text/html"; bsize:9; classtype:bad-unknown; sid:96; rev:1;)
```

Note

`http.content_type` does not include the leading space or trailing `\r\n`

Note

`http.content_type` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.29 http.cookie

The `http.cookie` keyword is used to match on the cookie field that can be present in HTTP request (Cookie) or HTTP response (Set-Cookie) headers.

It is possible to use any of the [Payload Keywords](#) with both `http.header` keywords.

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Cookie: PHPSESSION=123
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Cookie Example"; flow:established,to_server;
http.cookie; content:"PHPSESSIONID=123"; bsize:14; classtype:bad-unknown; sid:80; rev:1;)
```

Note

Cookies are passed in HTTP headers but Suricata extracts the cookie data to `http.cookie` and will not match cookie content put in the [http.header](#) sticky buffer.

Note

`http.cookie` does not include the leading space or trailing `\r\n`

Note

`http.cookie` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.30 http.header

Matching on HTTP headers has two options in Suricata, the `http.header` and the `http.header.raw`.

It is possible to use any of the [Payload Keywords](#) with both `http.header` keywords.

The `http.header` keyword normalizes the header contents. For example if header contents contain trailing white-space or tab characters, those would be removed.

To match on non-normalized header data, use the [http.header.raw](#) keyword.

Normalization Example:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0      \r\n
Host: suricata.io
```

Would be normalized to Mozilla/5.0\r\n

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Example 1"; flow:established,to_server;
http.header; content:"User-Agent|3a 20|Mozilla/5.0|0d 0a|"; classtype:bad-unknown; sid:70; rev:1;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Example 2"; flow:established,to_server;
http.header; content:"Host|3a 20|suricata.io|0d 0a|"; classtype:bad-unknown; sid:71; rev:1;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Example 3"; flow:established,to_server;
http.header; content:"User-Agent|3a 20|Mozilla/5.0|0d 0a|"; startswith; content:"Host|3a 20|suricata.io|0d 0a|";
classtype:bad-unknown; sid:72; rev:1;)
```

Note

There are headers that will not be included in the `http.header` buffer, specifically the *http.cookie* buffer.

Note

`http.header` can have additional formatting/normalization applied to buffer contents, see *Normalization* for additional details.

8.13.31 http.header.raw

The `http.header.raw` buffer matches on HTTP header content but does not have any normalization performed on the buffer contents (see *http.header*)

Abnormal HTTP Header Example:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
User-Agent: Chrome
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Raw Example";
flow:established,to_server; http.header.raw; content:"User-Agent|3a 20|Mozilla/5.0|0d 0a|"; content:"User-Agent|3a
20|Chrome|0d 0a|"; classtype:bad-unknown; sid:73; rev:1;)
```

Note

`http.header.raw` can have additional formatting applied to buffer contents, see *Normalization* for additional details.

8.13.32 http.header_names

The `http.header_names` keyword is used to match on the names of the headers in an HTTP request or response. This is useful for checking for a header's presence, absence and/or header order. Use `flow:to_server` or `flow:to_client` to force inspection of the request or response respectively.

It is possible to use any of the *Payload Keywords* with the `http.header_names` keyword.

Example HTTP Request:

```
GET / HTTP/1.1
Host: suricata.io
Connection: Keep-Alive
```

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
```

Examples to match exactly on header order:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Names Request Example";
flow:established,to_server; http.header_names; content:"|0d 0a|Host|0d 0a|Connection|0d 0a 0d 0a|"; bsize:22;
classtype:bad-unknown; sid:110; rev:1;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Header Names Response Example";
flow:established,to_client; http.header_names; content:"|0d 0a|Content-Type|0d 0a|Server|0d 0a 0d a0|"; bsize:26;
classtype:bad-unknown; sid:111; rev:1;)
```

Examples to match on header existence:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Names Request Example 2";
flow:established,to_server; http.header_names; content:"|0d 0a|Host|0d 0a|"; classtype:bad-unknown; sid:112; rev:1;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Header Names Response Example 2";
flow:established,to_client; http.header_names; content:"|0d 0a|Content-Type|0d 0a|"; classtype:bad-unknown; sid:113;
rev:1;)
```

Examples to match on header absence:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Names Request Example 3";
flow:established,to_server; http.header_names; content:"!|0d 0a|User-Agent|0d 0a|"; classtype:bad-unknown; sid:114;
rev:1;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Header Names Response Example 3";
flow:established,to_client; http.header_names; content:"!|0d 0a|Date|0d 0a|"; classtype:bad-unknown; sid:115; rev:1;)
```

Example to check for the `User-Agent` header and that the `Host` header is after `User-Agent` but not necessarily directly after.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Header Names Request Example 4";
flow:established,to_server; http.header_names; content:"|0d 0a|Host|0d 0a|"; content:"User-Agent|0d 0a|"; distance:-
2; classtype:bad-unknown; sid:114; rev:1;)
```

Note

`http.header_names` starts with a `\r\n` and ends with an extra `\r\n`.

Note

`http.header_names` can have additional formatting/normalization applied to buffer contents, see [Normalization](#) for additional details.

8.13.33 http.protocol

The `http.protocol` keyword is used to match on the protocol field that is contained in HTTP requests and responses. For HTTP/2, the constant string "HTTP/2" is used. See rfc 7540 section 8.1.2.4. about Response Pseudo-Header Fields.

It is possible to use any of the [Payload Keywords](#) with the `http.protocol` keyword.

Note

`http.protocol` does not include the leading space or trailing `\r\n`

Example HTTP Request:

```
GET /index.html HTTP/1.1
User-Agent: Mozilla/5.0
Host: suricata.io
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Protocol Example"; flow:established,to_server;
http.protocol; content:"HTTP/1.1"; bsize:9; classtype:bad-unknown; sid:50; rev:1;)
```

8.13.34 http.start

The `http.start` keyword is used to match on the start of an HTTP request or response. This will contain the request/response line plus the request/response headers. Use `flow:to_server` or `flow:to_client` to force inspection of the request or response respectively.

It is possible to use any of the [Payload Keywords](#) with the `http.start` keyword.

Example HTTP Request:

```
GET / HTTP/1.1
Host: suricata.io
Connection: Keep-Alive
```

Example HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Server: nginx/0.8.54
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"HTTP Start Request Example";
flow:established,to_server; http.start; content:"POST / HTTP/1.1|0d 0a|Host|0d 0a|Connection|0d 0a 0d 0a|";
classtype:bad-unknown; sid:101; rev:1;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HTTP Start Response Example";
flow:established,to_client; http.start; content:"HTTP/1.1 200 OK|0d 0a|Content-Type|0d 0a|Server|0d 0a 0d a0|";
classtype:bad-unknown; sid:102; rev:1;)
```

Note

`http.start` contains the normalized headers and is terminated by an extra `\r\n` to indicate the end of the headers.

8.14 File Keywords

Suricata comes with several rule keywords to match on various file properties. They depend on properly configured *File Extraction*.

8.14.1 file.data

The `file.data` sticky buffer matches on contents of files that are seen in flows that Suricata evaluates. The various payload keywords can be used (e.g. `startswith`, `nocase` and `bsize`) with `file.data`.

Example:

```
alert smtp any any -> any any (msg:"smtp app layer file.data example"; \
file.data; content:"example file content"; sid:1; rev:1)

alert http any any -> any any (msg:"http app layer file.data example"; \
file.data; content:"example file content"; sid:2; rev:1)

alert http2 any any -> any any (msg:"http2 app layer file.data example"; \
file.data; content:"example file content"; sid:3; rev:1;)

alert nfs any any -> any any (msg:"nfs app layer file.data example"; \
file.data; content:" "; sid:5; rev:1)

alert ftp-data any any -> any any (msg:"ftp app layer file.data example"; \
file.data; content:"example file content"; sid:6; rev:1;)

alert tcp any any -> any any (msg:"tcp file.data example"; \
file.data; content:"example file content"; sid:4; rev:1)
```

Note `file_data` is the legacy notation but can still be used.

8.14.2 file.name

`file.name` is a sticky buffer that is used to look at filenames that are seen in flows that Suricata evaluates. The various payload keywords can be used (e.g. `startswith`, `nocase` and `bsize`) with `file.name`.

Example:

```
file.name; content:"examplefilename";
```

`file.name` supports multiple buffer matching, see *Multiple Buffer Matching*.

Note `filename` can still be used. A notable difference between `file.name` and `filename` is that `filename` assumes `nocase` by default. In the example below the two signatures are considered the same.

Example:

```
filename:"examplefilename";

file.name; content:"examplefilename"; nocase;
```

8.14.3 fileext

`fileext` is used to look at individual file extensions that are seen in flows that Suricata evaluates.

Example:

```
fileext:"pdf";
```

Note: `fileext` does not allow partial matches. For example, if a PDF file (.pdf) is seen by a Suricata signature with `fileext:"pd"`; the signature will not produce an alert.

Note: `fileext` assumes `nocase` by default. This means that a file with the extension .PDF will be seen the same as if the file had an extension of .pdf.

Note: `fileext` and `file.name` can both be used to match on file extensions. In the example below the two signatures are considered the same.

Example:

```
fileext:"pdf";

file.name; content:".pdf"; nocase; endswith;
```

Note: While `fileext` and `file.name` can both be used to match on file extensions, `file.name` allows for partial matching on file extensions. The following would match on a file with the extension of .pd as well as .pdf.

Example:

```
file.name; content:".pd";
```

8.14.4 file.magic

Matches on the information `libmagic` returns about a file.

Example:

```
file.magic; content:"executable for MS Windows";
```

Note `filemagic` can still be used. The only difference between `file.magic` and `file.magic` is that `filemagic` assumes `nocase` by default. In the example below the two signatures are considered the same.

Example:

```
filemagic:"executable for MS Windows";

file.magic; content:"executable for MS Windows"; nocase;
```

Note: Suricata currently uses its underlying operating systems version/implementation of `libmagic`. Different versions and implementations of `libmagic` do not return the same information. Additionally there are varying Suricata performance impacts based on the version and implementation of `libmagic`. Additional information about Suricata and `libmagic` can be found here: <https://redmine.openinfosecfoundation.org/issues/437>

`file.magic` supports multiple buffer matching, see *Multiple Buffer Matching*.

8.14.5 filestore

Stores files to disk if the signature matched.

Syntax:

```
filestore:<direction>,<scope>;
```

direction can be:

- request/to_server: store a file in the request / to_server direction
- response/to_client: store a file in the response / to_client direction
- both: store both directions

scope can be:

- file: only store the matching file (for filename,fileext,filemagic matches)
- tx: store all files from the matching HTTP transaction
- ssn/flow: store all files from the TCP session/flow.

If direction and scope are omitted, the direction will be the same as the rule and the scope will be per file.

8.14.6 filemd5

Match file *MD5* against list of MD5 checksums.

Syntax:

```
filemd5:[!]filename;
```

The filename is expanded to include the rule dir. In the default case it will become /etc/suricata/rules/filename. Use the exclamation mark to get a negated match. This allows for white listing.

Examples:

```
filemd5:md5-blacklist;  
filemd5:!md5-whitelist;
```

File format

The file format is simple. It's a text file with a single md5 per line, at the start of the line, in hex notation. If there is extra info on the line it is ignored.

Output from md5sum is fine:

```
2f8d0355f0032c3e6311c6408d7c2dc2  util-path.c  
b9cf5cf347a70e02fde975fc4e117760  util-pidfile.c  
02aaa6c3f4dbae65f5889eeb8f2bbb8d  util-pool.c  
dd5fc1ee7f2f96b5f12d1a854007a818  util-print.c
```

Just MD5's are good as well:

```
2f8d0355f0032c3e6311c6408d7c2dc2  
b9cf5cf347a70e02fde975fc4e117760  
02aaa6c3f4dbae65f5889eeb8f2bbb8d  
dd5fc1ee7f2f96b5f12d1a854007a818
```

Memory requirements

Each MD5 uses 16 bytes of memory. 20 Million MD5's use about 310 MiB of memory.

See also: <https://blog.inliniac.net/2012/06/09/suricata-md5-blacklisting/>

8.14.7 filesha1

Match file SHA1 against list of SHA1 checksums.

Syntax:

```
filesha1:[!]filename;
```

The filename is expanded to include the rule dir. In the default case it will become `/etc/suricata/rules/filename`. Use the exclamation mark to get a negated match. This allows for white listing.

Examples:

```
filesha1:sha1-blacklist;
filesha1:!sha1-whitelist;
```

File format

Same as md5 file format.

8.14.8 filesha256

Match file SHA256 against list of SHA256 checksums.

Syntax:

```
filesha256:[!]filename;
```

The filename is expanded to include the rule dir. In the default case it will become `/etc/suricata/rules/filename`. Use the exclamation mark to get a negated match. This allows for white listing.

Examples:

```
filesha256:sha256-blacklist;
filesha256:!sha256-whitelist;
```

File format

Same as md5 file format.

8.14.9 filesize

Match on the size of the file as it is being transferred.

filesize uses an *unsigned 64-bit integer*.

Syntax:

```
filesize:<value>;
```

Possible units are KB, MB and GB, without any unit the default is bytes.

Examples:

```
filesize:100; # exactly 100 bytes
filesize:100<>200; # greater than 100 and smaller than 200
filesize:>100MB; # greater than 100 megabytes
filesize:<100MB; # smaller than 100 megabytes
```

Note: For files that are not completely tracked because of packet loss or stream.reassembly.depth being reached on the "greater than" is checked. This is because Suricata can know a file is bigger than a value (it has seen some of it already), but it can't know if the final size would have been within a range, an exact value or smaller than a value.

8.15 DNS Keywords

Suricata supports sticky buffers as well as keywords for efficiently matching on specific fields in DNS messages.

Note that sticky buffers are expected to be followed by one or more *Payload Keywords*.

8.15.1 dns.opcode

This keyword matches on the **opcode** found in the DNS header flags.

dns.opcode uses an *unsigned 8-bit integer*.

Syntax

```
dns.opcode: [!] <number>
dns.opcode: [!] <number1>-<number2>
```

Examples

Match on DNS requests and responses with **opcode** 4:

```
dns.opcode:4;
```

Match on DNS requests where the **opcode** is NOT 0:

```
dns.opcode: !0;
```

Match on DNS requests where the **opcode** is between 7 and 15, exclusively:

```
dns.opcode:7-15;
```

Match on DNS requests where the **opcode** is not between 7 and 15:

```
dns.opcode:!7-15;
```

8.15.2 dns.rcode

This keyword matches on the **rcode** field found in the DNS header flags.

dns.rcode uses an *unsigned 8-bit integer*. It can also be specified by text from the enumeration.

Currently, Suricata only supports rcode values in the range [0-15], while the current DNS version supports rcode values from [0-23] as specified in [RFC 6895](#).

We plan to extend the rcode values supported by Suricata according to RFC 6895 as tracked by the ticket: <https://redmine.openinfosecfoundation.org/issues/6650>

Syntax

```
dns.rcode: [!] <number>
dns.rcode: [!] <number1>-<number2>
```

Examples

Match on DNS requests and responses with **rcode** 4:

```
dns.rcode:4;
```

Match on DNS requests and responses where the **rcode** is NOT 0:

```
dns.rcode:!0;
```

8.15.3 dns.rrtype

This keyword matches on the **rrtype** (integer) found in the DNS message.

`dns.rrtype` uses an *unsigned 16-bit integer*.

It can also be specified by text from the enumeration.

Syntax

```
dns.rrtype:[!]<number>
```

Examples

Match on DNS requests and responses with **rrtype** 4:

```
dns.rrtype:4;
```

Match on DNS requests and responses where the **rrtype** is NOT 0:

```
dns.rrtype:!0;
```

8.15.4 dns.query

`dns.query` is a sticky buffer that is used to inspect DNS query names in DNS request messages. Example:

```
alert dns any any -> any any (msg:"Test dns.query option"; dns.query; content:"google";  
  -nocase; sid:1;)
```

Being a sticky buffer, payload keywords such as `content` are to be used after `dns.query`:

```
dns_query; content: "abc"; pcre: /abc/;
```



The `dns.query` keyword affects all following contents, until `pkt_data` is used or it reaches the end of the rule.

Note

`dns.query` is equivalent to the older **`dns_query`**.

Note

`dns.query` will only match on DNS request messages, to also match on DNS response message, see [*dns.queries.rrname*](#).

`dns.queries.rrname` supports *Multiple Buffer Matching*.

Normalized Buffer

Buffer contains literal domain name

- <length> values (as seen in a raw DNS request) are literal '.' characters
- no leading <length> value
- No terminating NULL (0x00) byte (use a negated relative `isdataat` to match the end)

Example DNS request for "mail.google.com" (for readability, hex values are encoded between pipes):

DNS query on the wire (snippet):

```
|04|mail|06|google|03|com|00|
```

`dns.query` buffer:

```
mail.google.com
```

8.15.5 dns.queries.rrname

`dns.queries.rrname` is a sticky buffer that is used to look at the name field in DNS query (question) resource records. It is nearly identical to `dns.query` but supports both DNS requests and responses.

`dns.queries.rrname` will look at both requests and responses, so `flow` is recommended to confine to a specific direction.

The buffer being matched on contains the complete re-assembled resource name, for example "www.suricata.io".

`dns.queries.rrname` supports *Multiple Buffer Matching*.

`dns.queries.rrname` was introduced in Suricata 8.0.0.

8.15.6 dns.answers.rrname

`dns.answers.rrname` is a sticky buffer that is used to look at the name field in DNS answer resource records.

`dns.answers.rrname` will look at both requests and responses, so `flow` is recommended to confine to a specific direction.

The buffer being matched on contains the complete re-assembled resource name, for example "www.suricata.io".

`dns.answers.rrname` supports *Multiple Buffer Matching*.

`dns.answers.rrname` was introduced in Suricata 8.0.0.

8.15.7 dns.authorities.rrname

`dns.authorities.rrname` is a sticky buffer that is used to look at the `rrname` field in DNS authority resource records.

`dns.authorities.rrname` will look at both requests and responses, so `flow` is recommended to confine to a specific direction.

The buffer being matched on contains the complete re-assembled resource name, for example "www.suricata.io".

`dns.authorities.rname` supports *Multiple Buffer Matching*.

`dns.authorities.rname` was introduced in Suricata 8.0.0.

8.15.8 dns.additional.rname

`dns.additional.rname` is a sticky buffer that is used to look at the `rname` field in DNS additional resource records.

`dns.additional.rname` will look at both requests and responses, so `flow` is recommended to confine to a specific direction.

The buffer being matched on contains the complete re-assembled resource name, for example "www.suricata.io".

`dns.additional.rname` supports *Multiple Buffer Matching*.

`dns.additional.rname` was introduced in Suricata 8.0.0.

8.15.9 dns.response.rname

`dns.response.rname` is a sticky buffer that is used to look at all name and rdata fields of DNS response (answer) resource records that are represented as a resource name (hostname). It supports inspecting all DNS response sections. Example:

```
alert dns any any -> any any (msg:"Test dns.response.rname option"; \
    dns.response.rname; content:"google"; nocase; sid:1;)
```

rdata field matching supports a subset of types that contain domain name structured data, for example: "www.suricata.io". The list of types inspected is:

- CNAME
- PTR
- MX
- NS
- SOA (mname data: primary name server)

The buffer being matched on contains the complete re-assembled resource name, for example "www.suricata.io".

`dns.response.rname` supports *Multiple Buffer Matching*.

`dns.response.rname` was introduced in Suricata 8.0.0.

8.16 mDNS Keywords

Suricata supports sticky buffers for efficiently matching on specific fields in mDNS (Multicast DNS) messages.

Note that sticky buffers are expected to be followed by one or more *Payload Keywords*.

8.16.1 mdns.queries.rname

`mdns.queries.rname` is a sticky buffer that is used to look at the name field in mDNS query resource records.

The buffer being matched on contains the complete re-assembled resource name, for example "host.local".

`mdns.queries.rname` supports *Multiple Buffer Matching*.

Example:

```
alert udp any any -> any 5353 (msg:"mDNS query for .local domain"; \
  mdns.queries.rname; content:".local"; sid:1;)
```

8.16.2 mdns.answers.rname

mdns.answers.rname is a sticky buffer that is used to look at the name field in mDNS answer resource records.

The buffer being matched on contains the complete re-assembled resource name, for example "printer.local".

mdns.answers.rname supports *Multiple Buffer Matching*.

Example:

```
alert udp any 5353 -> any any (msg:"mDNS answer for printer.local"; \
  mdns.answers.rname; content:"printer.local"; sid:2;)
```

8.16.3 mdns.authorities.rname

mdns.authorities.rname is a sticky buffer that is used to look at the rname field in mDNS authority resource records.

The buffer being matched on contains the complete re-assembled resource name, for example "device.local".

mdns.authorities.rname supports *Multiple Buffer Matching*.

Example:

```
alert udp any 5353 -> any any (msg:"mDNS authority record check"; \
  mdns.authorities.rname; content:"auth.local"; sid:3;)
```

8.16.4 mdns.additionals.rname

mdns.additionals.rname is a sticky buffer that is used to look at the rname field in mDNS additional resource records.

The buffer being matched on contains the complete re-assembled resource name, for example "service.local".

mdns.additionals.rname supports *Multiple Buffer Matching*.

Example:

```
alert udp any any -> any 5353 (msg:"mDNS additional record check"; \
  mdns.additionals.rname; content:"_companion-link._tcp.local"; nocase; sid:4;)
```

8.16.5 mdns.response.rname

mdns.response.rname is a sticky buffer that is used to inspect all the rname fields in a response, in the queries, answers, additionals and authorities. Additionally it will also inspect rdata fields that have the same format as an rname (hostname).

rdata types that will be inspected are:

- CNAME
- PTR
- MX
- NS

- SOA

Example:

```
alert udp any 5353 -> any any (msg:"mDNS answer data match"; \
  mdns.response.rname; content:"Apple TV"; sid:5;)
```

8.17 SSL/TLS Keywords

Suricata comes with several rule keywords to match on various properties of TLS/SSL handshake. Matches are string inclusion matches.

8.17.1 `tls.cert_subject`

Match TLS/SSL certificate Subject field.

Examples:

```
tls.cert_subject; content:"CN=*.googleusercontent.com"; isdataat:!1,relative;
tls.cert_subject; content:"google.com"; nocase; pcre:"/google\.com$/";
```

`tls.cert_subject` is a 'sticky buffer'.

`tls.cert_subject` can be used as `fast_pattern`.

`tls.cert_subject` supports multiple buffer matching, see [Multiple Buffer Matching](#).

`tls.subject`

Legacy keyword to match TLS/SSL certificate Subject field.

example:

```
tls.subject:"CN=*.googleusercontent.com"
```

Case sensitive, can't use 'nocase', or other modifiers.

Note: `tls.cert_subject` replaces the following legacy keywords: `tls_cert_subject` and `tls.subject`. It's recommended that rules be converted to use the new one.

8.17.2 `tls.cert_issuer`

Match TLS/SSL certificate Issuer field.

Examples:

```
tls.cert_issuer; content:"WoSign"; nocase; isdataat:!1,relative;
tls.cert_issuer; content:"StartCom"; nocase; pcre:"/StartCom$/";
```

`tls.cert_issuer` is a 'sticky buffer'.

`tls.cert_issuer` can be used as `fast_pattern`.

`tls.issuerdn`

Legacy keyword to match TLS/SSL certificate IssuerDN field

example:

```
tls.issuerdn:! "CN=Google-Internet-Authority"
```

Case sensitive, can't use 'nocase', or other modifiers.

Note: `tls.cert_issuer` replaces the following legacy keywords: `tls_cert_issuer` and `tls.issuerdn`. It's recommended that rules be converted to use the new one.

8.17.3 `tls.cert_serial`

Match on the serial number in a certificate.

Example:

```
alert tls any any -> any any (msg:"match cert serial"; \
  tls.cert_serial; content:"5C:19:B7:B1:32:3B:1C:A1"; sid:200012;)
```

`tls.cert_serial` is a 'sticky buffer'.

`tls.cert_serial` can be used as `fast_pattern`.

`tls.cert_serial` replaces the previous keyword name: `tls_cert_serial`. You may continue to use the previous name, but it's recommended that rules be converted to use the new name.

8.17.4 `tls.cert_fingerprint`

Match on the SHA-1 fingerprint of the certificate.

Example:

```
alert tls any any -> any any (msg:"match cert fingerprint"; \
  tls.cert_fingerprint; \
  content:"4a:a3:66:76:82:cb:6b:23:bb:c3:58:47:23:a4:63:a7:78:a4:a1:18"; \
  sid:200023;)
```

`tls.cert_fingerprint` is a 'sticky buffer'.

`tls.cert_fingerprint` can be used as `fast_pattern`.

`tls.cert_fingerprint` replaces the previous keyword name: `tls_cert_fingerprint` may continue to use the previous name, but it's recommended that rules be converted to use the new name.

8.17.5 `tls.sni`

Match TLS/SSL Server Name Indication field.

Examples:

```
tls.sni; content:"oisf.net"; nocase; isdataat:!1,relative;
tls.sni; content:"oisf.net"; nocase; pcre:"/oisf.net$/";
```

`tls.sni` is a 'sticky buffer'.

`tls.sni` can be used as `fast_pattern`.

`tls.sni` replaces the previous keyword name: `tls_sni`. You may continue to use the previous name, but it's recommended that rules be converted to use the new name.

8.17.6 tls.subjectaltname

Match TLS/SSL Subject Alternative Name field.

Examples:

```
tls.subjectaltname; content:"|73 75 72 69 63 61 74 61 2e 69 6f|";
```

tls.subjectaltname is a 'sticky buffer'.

tls.subjectaltname can be used as fast_pattern.

tls.subjectaltname supports multiple buffer matching, see *Multiple Buffer Matching*.

8.17.7 tls_cert_notbefore

Match on the NotBefore field in a certificate.

Example:

```
alert tls any any -> any any (msg:"match cert NotBefore"; \
  tls_cert_notbefore:1998-05-01<>2008-05-01; sid:200005;)
```

8.17.8 tls_cert_notafter

Match on the NotAfter field in a certificate.

Example:

```
alert tls any any -> any any (msg:"match cert NotAfter"; \
  tls_cert_notafter:>2015; sid:200006;)
```

8.17.9 tls_cert_expired

Match returns true if certificate is expired. It evaluates the validity date from the certificate.

Usage:

```
tls_cert_expired;
```

8.17.10 tls_cert_valid

Match returns true if certificate is not expired. It only evaluates the validity date. It does *not* do cert chain validation. It is the opposite of tls_cert_expired.

Usage:

```
tls_cert_valid;
```

8.17.11 tls.certs

Do a "raw" match on each of the certificates in the TLS certificate chain.

Example:

```
alert tls any any -> any any (msg:"match bytes in TLS cert"; tls.certs; \
  content:"|06 09 2a 86|"; sid:200070;)
```

`tls.certs` is a 'sticky buffer'.

`tls.certs` can be used as `fast_pattern`.

`tls.certs` supports multiple buffer matching, see *Multiple Buffer Matching*.

8.17.12 `tls.version`

Match on negotiated TLS/SSL version.

Supported values: "1.0", "1.1", "1.2", "1.3"

It is also possible to match versions using a hex string.

Examples:

```
tls.version:1.2;
tls.version:0x7f12;
```

The first example matches TLSv1.2, whilst the last example matches TLSv1.3 draft 16.

8.17.13 `ssl_version`

Match version of SSL/TLS record.

Supported values "sslv2", "sslv3", "tls1.0", "tls1.1", "tls1.2", "tls1.3"

Example:

```
alert tls any any -> any any (msg:"match TLSv1.2"; \
  ssl_version:tls1.2; sid:200030;)
```

It is also possible to match on several versions at the same time.

Example:

```
alert tls any any -> any any (msg:"match SSLv2 and SSLv3"; \
  ssl_version:sslv2,sslv3; sid:200031;)
```

The list can be prefixed with `!` to match if version is different than all the versions listed in the signature. Such a negation does not match on a yet undetermined version.

8.17.14 `tls.fingerprint`

match TLS/SSL certificate SHA1 fingerprint

example:

```
tls.fingerprint:!"f3:40:21:48:70:2c:31:bc:b5:aa:22:ad:63:d6:bc:2e:b3:46:e2:5a"
```

Case sensitive, can't use 'nocase'.

The `tls.fingerprint` buffer is lower case so you must use lower case letters for this to match.

8.17.15 `tls.store`

store TLS/SSL certificate on disk. The location can be specified in the `output.tls-store.certs-log-dir` parameter of the yaml configuration file, cf *TLS parameters and certificates logging (tls.log)*..

8.17.16 ssl_state

The `ssl_state` keyword matches the state of the SSL connection. The possible states are `client_hello`, `server_hello`, `client_keyx`, `server_keyx` and `unknown`. You can specify several states with `|` (OR) to check for any of the specified states.

8.17.17 tls.random

Matches on the 32 bytes of the TLS random field from the client hello or server hello records.

Example:

```
alert tls any any -> any any (msg:"TLS random test"; \
  tls.random; content:"|9b ce 7a 5e 57 5d 77 02 07 c2 9d be 24 01 cc f0 5d cd e1 d2 a5_
  ↳86 9c 4a 3e ee 38 db 55 1a d9 bc|"; sid: 200074;)
```

`tls.random` is a sticky buffer.

8.17.18 tls.random_time

Matches on the first 4 bytes of the TLS random field from the client hello or server hello records.

Example:

```
alert tls any any -> any any (msg:"TLS random_time test"; \
  tls.random_time; content:"|9b ce 7a 5e|"; sid: 200075;)
```

`tls.random_time` is a sticky buffer.

8.17.19 tls.random_bytes

Matches on the last 28 bytes of the TLS random field from the client hello or server hello records.

Example:

```
alert tls any any -> any any (msg:"TLS random_bytes test"; \
  tls.random_bytes; content:"|57 5d 77 02 07 c2 9d be 24 01 cc f0 5d cd e1 d2 a5 86 9c_
  ↳4a 3e ee 38 db 55 1a d9 bc|"; sid: 200076;)
```

`tls.random_bytes` is a sticky buffer.

8.17.20 tls.cert_chain_len

Matches on the TLS certificate chain length.

`tls.cert_chain_len` uses an *unsigned 32-bit integer*.

`tls.cert_chain_len` supports `<`, `>`, `<>`, `!` and using an exact value.

Example:

```
alert tls any any -> any any (msg:"cert chain exact value"; \
tls.cert_chain_len:1; classtype:misc-activity; sid:1; rev:1;)

alert tls any any -> any any (msg:"cert chain less than value"; \
tls.cert_chain_len:<2; classtype:misc-activity; sid:2; rev:1;)

alert tls any any -> any any (msg:"cert chain greater than value"; \
```

(continues on next page)

(continued from previous page)

```
tls.cert_chain_len:>0; classtype:misc-activity; sid:2; rev:1;)

alert tls any any -> any any (msg:"cert chain greater than less than value";\
tls.cert_chain_len:0<>2; classtype:misc-activity; sid:3; rev:1;)

alert tls any any -> any any (msg:"cert chain not value"; \
tls.cert_chain_len:12; classtype:misc-activity; sid:4; rev:1;)
```

8.17.21 tls.alpn

Matches on the ALPN buffers.

Example:

```
alert tls any any -> any any (msg:"TLS ALPN test"; \
tls.alpn; content:"http/1.1"; sid:1;)
```

tls.alpn is a sticky buffer.

8.18 SSH Keywords

Suricata has several rule keywords to match on different elements of SSH connections.

8.18.1 Hooks

The available hooks for SSH are:

Request (to_server) side:

- request_in_progress
- request_banner_wait_eol
- request_banner_done
- request_finished

Response (to_client) side:

- response_in_progress
- response_banner_wait_eol
- response_banner_done
- response_finished

8.18.2 Frames

The SSH parser supports the following frames:

- ssh.record_hdr
- ssh.record_data
- ssh.record_pdu

These are header + data = pdu for SSH records, after the banner and before encryption. The SSH record header is 6 bytes long : 4 bytes length, 1 byte passing, 1 byte message code.

Example:

```
alert ssh any any -> any any (msg:"hdr frame new keys"; frame:ssh.record.hdr; content: "|15|"; endswith; bsize: 6; sid:2;)
```

This rule matches like Wireshark `ssh.message_code == 0x15`.

8.18.3 ssh.proto

Match on the version of the SSH protocol used. `ssh.proto` is a sticky buffer, and can be used as a fast pattern. `ssh.proto` replaces the previous buffer name: `ssh_proto`. You may continue to use the previous name, but it's recommended that existing rules be converted to use the new name.

Format:

```
ssh.proto;
```

Example:

```
alert ssh any any -> any any (msg:"match SSH protocol version"; ssh.proto; content:"2.0"; sid:1000010;)
```

The example above matches on SSH connections with SSH version 2.0.

8.18.4 ssh.software

Match on the software string from the SSH banner. `ssh.software` is a sticky buffer, and can be used as fast pattern.

Format:

```
ssh.software;
```

Example:

```
alert ssh any any -> any any (msg:"match SSH software string"; ssh.software; content:"openssh"; nocase; sid:1000020;)
```

The example above matches on SSH connections where the software string contains "openssh".

8.18.5 ssh.hassh

Match on hassh (md5 of hassh algorithms of client).

Example:

```
alert ssh any any -> any any (msg:"match hassh"; \
  ssh.hassh; content:"ec7378c1a92f5a8dde7e8b7a1ddf33d1"; \
  sid:1000010;)
```

`ssh.hassh` is a 'sticky buffer'.

`ssh.hassh` can be used as `fast_pattern`.

8.18.6 ssh.hassh.string

Match on Hassh string (hassh algorithms of client).

Example:

```
alert ssh any any -> any any (msg:"match hassh-string"; \
  ssh.hassh.string; content:"none,zlib@openssh.com,zlib"; \
  sid:1000030;)
```

`ssh.hassh.string` is a 'sticky buffer'.

`ssh.hassh.string` can be used as `fast_pattern`.

8.18.7 ssh.hassh.server

Match on `hassh` (md5 of `hassh` algorithms of server).

Example:

```
alert ssh any any -> any any (msg:"match SSH hash-server"; \
  ssh.hassh.server; content:"b12d2871a1189eff20364cf5333619ee"; \
  sid:1000020;)
```

`ssh.hassh.server` is a 'sticky buffer'.

`ssh.hassh.server` can be used as `fast_pattern`.

8.18.8 ssh.hassh.server.string

Match on `hassh` string (hash algorithms of server).

Example:

```
alert ssh any any -> any any (msg:"match SSH hash-server-string"; \
  ssh.hassh.server.string; content:"umac-64-etm@openssh.com,umac-128-etm@openssh.com"; \
  sid:1000040;)
```

`ssh.hassh.server.string` is a 'sticky buffer'.

`ssh.hassh.server.string` can be used as `fast_pattern`.

8.19 JA3/JA4 Keywords

Suricata comes with JA3 (<https://github.com/salesforce/ja3>) and JA4 (<https://github.com/FoxIO-LLC/ja4>) integration. JA3 and JA4 are used to fingerprint TLS and QUIC clients.

Support must be enabled in the Suricata config file (set `app-layer.protocols.tls.ja{3,4}-fingerprints` to `yes`). If it is not explicitly disabled (`no`), it will be enabled if a loaded rule requires it. Note that JA3/JA4 support can also be disabled at compile time; it is possible to use the `requires: feature ja{3,4};` keyword to skip rules if no JA3/JA4 support is present.

8.19.1 ja3.hash

Match on JA3 hash (md5).

Example:

```
alert tls any any -> any any (msg:"match JA3 hash"; \
  ja3.hash; content:"e7eca2baf4458d095b7f45da28c16c34"; \
  sid:100001;)
```

`ja3.hash` is a 'sticky buffer'.

`ja3.hash` can be used as `fast_pattern`.

`ja3.hash` replaces the previous keyword name: `ja3_hash`. You may continue to use the previous name, but it's recommended that rules be converted to use the new name.

8.19.2 ja3.string

Match on JA3 string.

Example:

```
alert tls any any -> any any (msg:"match JA3 string"; \
  ja3.string; content:"19-20-21-22"; \
  sid:100002;)
```

ja3.string is a 'sticky buffer'.

ja3.string can be used as fast_pattern.

ja3.string replaces the previous keyword name: ja3_string. You may continue to use the previous name, but it's recommended that rules be converted to use the new name.

8.19.3 ja3s.hash

Match on JA3S hash (md5).

Example:

```
alert tls any any -> any any (msg:"match JA3S hash"; \
  ja3s.hash; content:"b26c652e0a402a24b5ca2a660e84f9d5"; \
  sid:100003;)
```

ja3s.hash is a 'sticky buffer'.

ja3s.hash can be used as fast_pattern.

8.19.4 ja3s.string

Match on JA3S string.

Example:

```
alert tls any any -> any any (msg:"match on JA3S string"; \
  ja3s.string; content:"771,23-35"; sid:100004;)
```

ja3s.string is a 'sticky buffer'.

ja3s.string can be used as fast_pattern.

8.19.5 ja4.hash

Match on JA4 hash (e.g. q13d0310h3_55b375c5d22e_cd85d2d88918).

Example:

```
alert quic any any -> any any (msg:"match JA4 hash"; \
  ja4.hash; content:"q13d0310h3_55b375c5d22e_cd85d2d88918"; \
  sid:100001;)
```

ja4.hash is a 'sticky buffer'.

ja4.hash can be used as fast_pattern.

8.20 Modbus Keyword

The modbus keyword can be used for matching on various properties of Modbus requests.

There are three ways of using this keyword:

- matching on functions properties with the setting "function";
- matching on directly on data access with the setting "access";
- matching on unit identifier with the setting "unit" only or with the previous setting "function" or "access".

With the setting **function**, you can match on:

- an action based on a function code field and a sub-function code when applicable;
- one of three categories of Modbus functions;
- public functions that are publicly defined (setting "public")
- user-defined functions (setting "user")
- reserved functions that are dedicated to proprietary extensions of Modbus (keyword "reserved")
- one of the two sub-groups of public functions:
 - assigned functions whose definition is already given in the Modbus specification (keyword "assigned");
 - unassigned functions, which are reserved for future use (keyword "unassigned").

Syntax:

```
modbus: function <value>
modbus: function <value>, subfunction <value>
modbus: function [!] <assigned | unassigned | public | user | reserved | all>
```

Sign '!' is negation

Examples:

```
modbus: function 21           # Write File record function
modbus: function 4, subfunction 4 # Force Listen Only Mode (Diagnostics) function
modbus: function assigned     # defined by Modbus Application Protocol
↪ Specification V1.1b3
modbus: function public       # validated by the Modbus.org community
modbus: function user         # internal use and not supported by the specification
modbus: function reserved    # used by some companies for legacy products and not
↪ available for public use
modbus: function !reserved    # every function but reserved function
```

With the **access** setting, you can match on:

- a type of data access (read or write);
- one of primary tables access (Discretes Input, Coils, Input Registers and Holding Registers);
- a range of addresses access;
- a written value.

Syntax:

```

modbus: access <read | write>
modbus: access read <discretes | coils | input | holding>
modbus: access read <discretes | coils | input | holding>, address <value>
modbus: access write < coils | holding>
modbus: access write < coils | holding>, address <value>
modbus: access write < coils | holding>, address <value>, value <value>

```

With `_<value>_` setting matches on the address or value as it is being accessed or written as follows:

```

address 100      # exactly address 100
address 100<>200 # greater than address 100 and smaller than address 200
address >100     # greater than address 100
address <100     # smaller than address 100

```

Examples:

```

modbus: access read                                # Read access
modbus: access write                               # Write access
modbus: access read input                          # Read access to Discretes Input
↳table
modbus: access write coils                         # Write access to Coils table
modbus: access read discretes, address <100        # Read access at address smaller
↳than 100 of Discretes Input table
modbus: access write holding, address 500, value >200 # Write value greater than 200 at
↳address 500 of Holding Registers table

```

With the setting **unit**, you can match on:

- a MODBUS slave address of a remote device connected on the sub-network behind a bridge or a gateway. The destination IP address identifies the bridge itself and the bridge uses the MODBUS unit identifier to forward the request to the right slave device.

Syntax:

```

modbus: unit <value>
modbus: unit <value>, function <value>
modbus: unit <value>, function <value>, subfunction <value>
modbus: unit <value>, function [!] <assigned | unassigned | public | user | reserved |
↳all>
modbus: unit <value>, access <read | write>
modbus: unit <value>, access read <discretes | coils | input | holding>
modbus: unit <value>, access read <discretes | coils | input | holding>, address <value>
modbus: unit <value>, access write < coils | holding>
modbus: unit <value>, access write < coils | holding>, address <value>
modbus: unit <value>, access write < coils | holding>, address <value>, value <value>

```

With `_<value>_` setting matches on the address or value as it is being accessed or written as follows:

```

unit 10      # exactly unit identifier 10
unit 10<>20  # greater than unit identifier 10 and smaller than unit identifier 20
unit >10     # greater than unit identifier 10
unit <10     # smaller than unit identifier 10

```

Examples:

```

modbus: unit 10                                     # Unit identifier
↪ 10
modbus: unit 10, function 21                         # Unit identifier
↪ 10 and write File record function
modbus: unit 10, function 4, subfunction 4           # Unit identifier
↪ 10 and force Listen Only Mode (Diagnostics) function
modbus: unit 10, function assigned                   # Unit identifier
↪ 10 and assigned function
modbus: unit 10, function !reserved                  # Unit identifier
↪ 10 and every function but reserved function
modbus: unit 10, access read                         # Unit identifier
↪ 10 and Read access
modbus: unit 10, access write coils                  # Unit identifier
↪ 10 and Write access to Coils table
modbus: unit >10, access read discretes, address <100 # Greater than
↪ unit identifier 10 and Read access at address smaller than 100 of Discretes Input table
modbus: unit 10<>20, access write holding, address 500, value >200 # Greater than
↪ unit identifier 10 and smaller than unit identifier 20 and Write value greater than
↪ 200 at address 500 of Holding Registers table

```

(cf. http://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf)

Note: Address of read and write are starting at 1. So if your system is using a start at 0, you need to add 1 the address values.

Note: According to MODBUS Messaging on TCP/IP Implementation Guide V1.0b, it is recommended to keep the TCP connection opened with a remote device and not to open and close it for each MODBUS/TCP transaction. In that case, it is important to set the depth of the stream reassembling as unlimited (stream.reassembly.depth: 0)

Note: According to MODBUS Messaging on TCP/IP Implementation Guide V1.0b, the MODBUS slave device addresses on serial line are assigned from 1 to 247 (decimal). Address 0 is used as broadcast address.

(cf. http://www.modbus.org/docs/Modbus_Messaging_Implementation_Guide_V1_0b.pdf)

Paper and presentation (in french) on Modbus support are available : <http://www.ssi.gouv.fr/agence/publication/detection-dintrusion-dans-les-systemes-industriels-suricata-et-le-cas-modbus/>

8.21 DCERPC Keywords

Following keywords can be used for matching on fields in headers and payloads of DCERPC packets over UDP, TCP and SMB.

8.21.1 dcerpc.iface

Match on the value of the interface UUID in a DCERPC header. If *any_frag* option is given, the match shall be done on all fragments. If it's not, the match shall only happen on the first fragment.

The format of the keyword:

```

dcerpc.iface:<uuid>;
dcerpc.iface:<uuid>,[>,<,!]=]<iface_version>;
dcerpc.iface:<uuid>,any_frag;
dcerpc.iface:<uuid>,[>,<,!]=]<iface_version>,any_frag;

```

Examples:

```
dcerpc.iface:367abb81-9844-35f1-ad32-98f038001003;
dcerpc.iface:367abb81-9844-35f1-ad32-98f038001003,!10;
dcerpc.iface:367abb81-9844-35f1-ad32-98f038001003,any_frag;
dcerpc.iface:367abb81-9844-35f1-ad32-98f038001003,>1,any_frag;
```

ET Open rule example:

```
alert tcp any any -> $HOME_NET any (msg:"ET NETBIOS DCERPC WMI Remote Process Execution";
flow:to_server,established; dce_iface:00000143-0000-0000-c000-000000000046; classtype:bad-unknown;
sid:2027167; rev:1; metadata:affected_product Windows_XP_Vista_7_8_10_Server_32_64_Bit, attack_target
Client_Endpoint, created_at 2019_04_09, deployment Internal, former_category NETBIOS, signature_severity
Informational, updated_at 2019_04_09;)
```

8.21.2 dcerpc.opnum

Match on one or many operation numbers and/or operation number range within the interface in a DCERPC header.

The format of the keyword:

```
dcerpc.opnum:<u16>;
dcerpc.opnum:<u16>,<u16>,<u16>....;
dcerpc.opnum:<u16>-<u16>;
```

Examples:

```
dcerpc.opnum:15;
dcerpc.opnum:12,24,62,61;
dcerpc.opnum:12,18-24,5;
dcerpc.opnum:12-14,12,121,62-78;
```

Note

earlier versions of the documentation incorrectly stated that greater than, less than, etc. notation was supported. This may be added later, see ticket #8179).

8.21.3 dcerpc.stub_data

Match on the stub data in a given DCERPC packet. It is a 'sticky buffer'.

Example:

```
dcerpc.stub_data; content:"123456";
```

8.21.4 Additional information

More information on the protocol can be found here:

- DCERPC: <https://pubs.opengroup.org/onlinepubs/9629399/chap1.htm>

8.22 DHCP keywords

8.22.1 dhcp.lease_time

DHCP lease time (integer).

dhcp.lease_time uses an *unsigned 64-bit integer*.

Syntax:

```
dhcp.lease_time:[op]<number>
```

The time can be matched exactly, or compared using the `_op_` setting:

```
dhcp.lease_time:3      # exactly 3
dhcp.lease_time:<3     # smaller than 3
dhcp.lease_time:>=2    # greater or equal than 2
```

Signature example:

```
alert dhcp any any -> any any (msg:"small DHCP lease time (<3)"; dhcp.lease_time:<3; sid:1; rev:1;)
```

8.22.2 dhcp.rebinding_time

DHCP rebinding time (integer).

dhcp.rebinding_time uses an *unsigned 64-bit integer*.

Syntax:

```
dhcp.rebinding_time:[op]<number>
```

The time can be matched exactly, or compared using the `_op_` setting:

```
dhcp.rebinding_time:3      # exactly 3
dhcp.rebinding_time:<3     # smaller than 3
dhcp.rebinding_time:>=2    # greater or equal than 2
```

Signature example:

```
alert dhcp any any -> any any (msg:"small DHCP rebinding time (<3)"; dhcp.rebinding_time:<3; sid:1; rev:1;)
```

8.22.3 dhcp.renewal_time

DHCP renewal time (integer).

dhcp.renewal_time uses an *unsigned 64-bit integer*.

Syntax:

```
dhcp.renewal_time:[op]<number>
```

The time can be matched exactly, or compared using the `_op_` setting:

```
dhcp.renewal_time:3      # exactly 3
dhcp.renewal_time:<3     # smaller than 3
dhcp.renewal_time:>=2    # greater or equal than 2
```

Signature example:

```
alert dhcp any any -> any any (msg:"small DHCP renewal time (<3)"; dhcp.renewal_time:<3;sid:1; rev:1;)
```

8.23 DNP3 Keywords

The DNP3 keywords can be used to match on fields in decoded DNP3 messages. The keywords are based on Snort's DNP3 keywords and aim to be 100% compatible.

8.23.1 dnp3_func

This keyword will match on the application function code found in DNP3 request and responses. It can be specified as the integer value or the symbolic name of the function code.

Syntax

```
dnp3_func:<value>;
```

Where value is one of:

- An integer value between 0 and 255 inclusive.
- Function code name:
 - confirm
 - read
 - write
 - select
 - operate
 - direct_operate
 - direct_operate_nr
 - immed_freeze
 - immed_freeze_nr
 - freeze_clear
 - freeze_clear_nr
 - freeze_at_time
 - freeze_at_time_nr
 - cold_restart
 - warm_restart
 - initialize_data
 - initialize_appl
 - start_appl
 - stop_appl
 - save_config
 - enable_unsolicited

- disable_unsolicited
- assign_class
- delay_measure
- record_current_time
- open_file
- close_file
- delete_file
- get_file_info
- authenticate_file
- abort_file
- activate_config
- authenticate_req
- authenticate_err
- response
- unsolicited_response
- authenticate_resp

8.23.2 dnp3_ind

This keyword matches on the DNP3 internal indicator flags in the response application header.

Syntax

`dnp3_ind:<flag>{,<flag>...}`

Where flag is the name of the internal indicator:

- all_stations
- class_1_events
- class_2_events
- class_3_events
- need_time
- local_control
- device_trouble
- device_restart
- no_func_code_support
- object_unknown
- parameter_error
- event_buffer_overflow
- already_executing
- config_corrupt

- reserved_2
- reserved_1

This keyword will match if any of the flags listed are set. To match on multiple flags (AND type match), use `dnp3_ind` for each flag that must be set.

Examples

```
dnp3_ind:all_stations;
```

```
dnp3_ind:class_1_events,class_2_events;
```

8.23.3 dnp3_obj

This keyword matches on the DNP3 application data objects.

Syntax

```
dnp3_obj:<group>,<variation>
```

Where `<group>` and `<variation>` are integer values between 0 and 255 inclusive.

8.23.4 dnp3_data

This keyword will cause the following content options to match on the re-assembled application buffer. The reassembled application buffer is a DNP3 fragment with CRCs removed (which occur every 16 bytes), and will be the complete fragment, possibly reassembled from multiple DNP3 link layer frames.

Syntax

```
dnp3_data;
```

Example

```
dnp3_data; content:"|c3 06|";
```

8.24 ENIP/CIP Keywords

8.24.1 enip_command

For the ENIP command, we are matching against the command field found in the ENIP encapsulation.

Examples:

```
enip_command:99;
enip_command:list_identity;
```

`enip_command` uses an *unsigned 16-bits integer*. It can also be specified by text from the enumeration.

8.24.2 cip_service

For the CIP Service, we use a maximum of 3 comma separated values representing the Service, Class and Attribute. These values are described in the CIP specification. CIP Classes are associated with their Service, and CIP Attributes are associated with their Service. If you only need to match up until the Service, then only provide the Service value. If you want to match to the CIP Attribute, then you must provide all 3 values.

Examples:

```
cip_service:75  
cip_service:16,246,6
```

(cf. <http://read.pudn.com/downloads166/ebook/763211/EIP-CIP-V1-1.0.pdf>)

Information on the protocol can be found here: http://literature.rockwellautomation.com/idc/groups/literature/documents/wp/enet-wp001_-en-p.pdf

8.24.3 enip.status

For the ENIP status, we are matching against the status field found in the ENIP encapsulation. It uses a 32-bit unsigned integer as value.

enip.status uses an *unsigned 32-bits integer*. It can also be specified by text from the enumeration.

Examples:

```
enip.status:100;  
enip.status:>106;  
enip.status:invalid_cmd;
```

8.24.4 enip.protocol_version

Match on the protocol version in different messages. It uses a 16-bit unsigned integer as value.

enip.protocol_version uses an *unsigned 16-bits integer*.

Examples:

```
enip.protocol_version:1;  
enip.protocol_version:>1;
```

8.24.5 enip.cip_attribute

Match on the cip attribute in different messages. It uses a 32-bit unsigned integer as value.

This allows to match without needing to match on cip.service.

enip.cip_attribute uses an *unsigned 32-bits integer*.

Examples:

```
enip.cip_attribute:1;  
enip.cip_attribute:>1;
```

8.24.6 enip.cip_instance

Match on the cip instance in CIP request path. It uses a 32-bit unsigned integer as value.

enip.cip_instance uses an *unsigned 32-bits integer*.

Examples:

```
enip.cip_instance:1;  
enip.cip_instance:>1;
```

8.24.7 enip.cip_class

Match on the cip class in CIP request path. It uses a 32-bit unsigned integer as value.

enip.cip_class uses an *unsigned 32-bits integer*.

This allows to match without needing to match on cip.service.

Examples:

```
enip.cip_class:1;  
enip.cip_class:>1;
```

8.24.8 enip.cip_extendedstatus

Match on the cip extended status, if any is present. For multiple service packet, will match on any of the seen statuses. It uses a 16-bit unsigned integer as value.

enip.cip_extendedstatus uses an *unsigned 16-bits integer*.

Examples:

```
enip.cip_extendedstatus:1;  
enip.cip_extendedstatus:>1;
```

8.24.9 enip.revision

Match on the revision in identity message. It uses a 16-bit unsigned integer as value.

enip.revision uses an *unsigned 16-bits integer*.

Examples:

```
enip.revision:1;  
enip.revision:>1;
```

8.24.10 enip.identity_status

Match on the status in identity message (not in ENIP header). It uses a 16-bit unsigned integer as value.

enip.identity_status uses an *unsigned 16-bits integer*.

Examples:

```
enip.identity_status:1;  
enip.identity_status:>1;
```

8.24.11 enip.state

Match on the state in identity message. It uses an 8-bit unsigned integer as value.

enip.state uses an *unsigned 8-bits integer*.

Examples:

```
enip.state:1;  
enip.state:>1;
```

8.24.12 enip.serial

Match on the serial in identity message. It uses a 32-bit unsigned integer as value.

enip.serial uses an *unsigned 32-bits integer*.

Examples:

```
enip.serial:1;  
enip.serial:>1;
```

8.24.13 enip.product_code

Match on the product code in identity message. It uses a 16-bit unsigned integer as value.

enip.product_code uses an *unsigned 16-bits integer*.

Examples:

```
enip.product_code:1;  
enip.product_code:>1;
```

8.24.14 enip.device_type

Match on the device type in identity message. It uses a 16-bit unsigned integer as value.

enip.device_type uses an *unsigned 16-bits integer*.

Examples:

```
enip.device_type:1;  
enip.device_type:>1;
```

8.24.15 enip.vendor_id

Match on the vendor id in identity message. It uses a 16-bit unsigned integer as value.

enip.vendor_id uses an *unsigned 16-bits integer*.

Examples:

```
enip.vendor_id:1;  
enip.vendor_id:>1;
```

8.24.16 enip.product_name

Match on the product name in identity message.

Examples:

```
enip.product_name; pcre:"/^123[0-9]*/";
enip.product_name; content:"swordfish";
```

enip.product_name is a 'sticky buffer' and can be used as fast_pattern.

8.24.17 enip.service_name

Match on the service name in list services message.

Examples:

```
enip.service_name; pcre:"/^123[0-9]*/";
enip.service_name; content:"swordfish";
```

enip.service_name is a 'sticky buffer' and can be used as fast_pattern.

8.24.18 enip.capabilities

Match on the capabilities in list services message. It uses a 16-bit unsigned integer as value.

enip.capabilities uses an *unsigned 16-bits integer*.

Examples:

```
enip.capabilities:1;
enip.capabilities:>1;
```

8.24.19 enip.cip_status

Match on the cip status (one of them in case of multiple service packet). It uses an 8-bit unsigned integer as value.

enip.cip_status uses an *unsigned 8-bits integer*.

Examples:

```
enip.cip_status:1;
enip.cip_status:>1;
```

8.25 FTP/FTP-DATA Keywords

8.25.1 ftpdata_command

Filter ftp-data channel based on command used on the FTP command channel. Currently supported commands are RETR (get on a file) and STOR (put on a file).

Syntax:

```
ftpdata_command:(retr|stor)
```

Signature Example:

```
alert ftp-data any any -> any any (msg:"FTP store password"; filestore; filename:"password"; ftpdata_command:stor;
sid:3; rev:1;)
```

8.25.2 ftpbounce

Detect FTP bounce attacks.

Syntax:

```
ftpbounce
```

8.25.3 file.name

The `file.name` keyword can be used at the FTP application level.

Signature Example:

```
alert ftp-data any any -> any any (msg:"FTP file.name usage"; file.name; content:"file.txt"; classtype:bad-unknown; sid:1; rev:1;)
```

For additional information on the `file.name` keyword, see [File Keywords](#).

8.25.4 ftp.command

This keyword matches on the command name from an FTP client request. `ftp.command` is a sticky buffer and can be used as a fast pattern.

Syntax:

```
ftp.command; content: <command>;
```

Signature Example:

```
alert ftp any any -> any any (ftp.command; content:"PASS"; sid: 1;)
```

Examples of commands are:

- USER
- PASS
- PORT
- EPRT
- PASV
- RETR

8.25.5 ftp.command_data

This keyword matches on the command data from a FTP client request. `ftp.command_data` is a sticky buffer and can be used as a fast pattern.

Syntax:

```
ftp.command_data; content: <command_data>;
```

Signature Example:

```
alert ftp any any -> any any (ftp.command_data; content:"anonymous"; sid: 1;)
```

The `ftp.command_data` matches the data associated with an FTP command. Consider the following FTP command examples:

```
USER anonymous
RETR temp.txt
PORT 192,168,0,13,234,10
```

Example rules for each of the preceding FTP commands and command data.

```
alert ftp any any -> any any (ftp.command; content: "USER"; ftp.command_data; content:"anonymous"; sid: 1;)
alert ftp any any -> any any (ftp.command_data; content:"anonymous"; sid: 1;)
alert ftp any any -> any any (ftp.command_data; content:"temp.txt"; sid: 2;)
alert ftp any any -> any any (ftp.command_data; content:"192,168,0,13,234,10"; sid: 3;)
```

8.25.6 ftp.completion_code

This keyword matches on an FTP completion code string. Note that there may be multiple reply strings for an FTP command and hence, multiple completion code values to check. `ftp.completion_code` is a sticky buffer and can be used as a fast pattern. Do not include the response string in the *content* to match upon (see examples).

Syntax:

```
ftp.completion_code; content: <quoted-completion-code>;
```

Signature Example:

```
alert ftp any any -> any any (ftp.completion_code; content:"226"; sid: 1;)
```

Note

FTP commands can return multiple reply strings. Specify a single completion code for each `ftp.completion_code` keyword.

This example shows an FTP command (RETR) followed by an FTP reply with multiple response strings.

```
RETR temp.txt
150 Opening BINARY mode data connection for temp.txt (1164 bytes).
226 Transfer complete.
```

Because there are multiple completion codes and responses, the rule can match on `ftp.reply` and the `ftp.completion_code`. Suricata cannot guarantee that these come from the *same response*, however.

Signature Examples:

```
alert ftp any any -> any any (ftp.reply; content:"Opening BINARY mode data connection for temp.";
ftp.completion_code; content: "150"; sid: 1;)
alert ftp any any -> any any (ftp.completion_code; content: "226"; sid: 2;)
alert ftp any any -> any any ( ftp.reply; content: "Transfer complete."; ftp.completion_code; content: "226"; sid: 3;)
```

8.25.7 ftp.dynamic_port

This keyword matches on the dynamic port negotiated during an FTP session with the following FTP commands:

- IPv4: PORT and EPRT
- IPv6: PASV and EPSV

Syntax:

```
ftp.dynamic_port: <port-spec>;
```

port-spec can be one of the following:

- > (greater than)
- < (less than)
- >= (greater than or equal)
- <= (less than or equal)
- arg1-arg2 (exclusive range)

Signature Example:

```
alert ftp any any -> any any (ftp.dynamic_port: 59914; sid: 1;)
```

These rules will also alert on port 59914:

```
alert ftp any any -> any any (ftp.dynamic_port: 59913-59915; sid: 1;)
```

```
alert ftp any any -> any any (ftp.dynamic_port: =59914; sid: 1;)
```

Example rules combining ftp.dynamic_port with ftp.command

```
alert ftp any any -> any any (ftp.command; content: "PORT"; ftp.dynamic_port: 59914; sid: 1;)
```

```
alert ftp any any -> any any (ftp.command; content: "EPSV"; ftp.dynamic_port: 58612; sid: 1;)
```

8.25.8 ftp.mode

This keyword matches on whether the FTP session is dynamic or passive. In *active* mode sessions, the server establishes the data channel. In *passive* mode, the client establishes the data channel. Active mode sessions are established in part with the PORT (EPRT for IPv6) command; passive mode sessions use PASV (EPSV for IPv6).

Syntax:

```
ftp.mode: active|passive;
```

Signature Example:

```
alert ftp any any -> any any (ftp.mode: active; sid: 1;)
```

```
alert ftp any any -> any any (ftp.mode: passive; sid: 1;)
```

Example rules combining ftp.command with ftp.mode

```
alert ftp any any -> any any (ftp.command; content: "PORT"; ftp.mode: active; sid:1;)
```

```
alert ftp any any -> any any (ftp.command; content: "PASV"; ftp.mode: passive; sid:1;)
```

8.25.9 ftp.reply

This keyword matches on an FTP reply string. Note that there may be multiple reply strings for an FTP command. ftp.reply is a sticky buffer and can be used as a fast pattern. Do not include the completion code in the *content* to match upon (see examples).

Syntax:

```
ftp.reply; content: <reply-string>;
```

Note

FTP commands can return multiple reply strings. Specify a single reply for each `ftp.reply` keyword.

This example shows an FTP command (RETR) followed by an FTP reply with multiple response strings.

```
RETR temp.txt
150 Opening BINARY mode data connection for temp.txt (1164 bytes).
226 Transfer complete.
```

Signature Example:

```
alert ftp any any -> any any (ftp.reply; content:"Please specify the password."; sid: 1;)
```

```
alert ftp any any -> any any (ftp.reply; content:"Opening BINARY mode data connection for temp."; sid: 1;)
```

```
alert ftp any any -> any any (ftp.reply; content:"Transfer complete."; sid: 2;)
```

8.25.10 ftp.reply_received

This keyword matches on whether an FTP reply string was received. EVE logs with the FTP event_type include a field named `reply_received`. Use this keyword to alert when a reply is (is not) received. `ftp.reply_received` is not a sticky buffer and uses a different syntax to express its value.

Note

Specify the match value without using quotes, e.g., use `yes` instead of `"yes"`.

Syntax:

```
ftp.reply_received: yes|on|true|1|no|off|false|0;
```

Signature Example:

```
alert ftp any any -> any any (ftp.reply_received: yes; sid: 1;)
```

```
alert ftp any any -> any any (ftp.reply_received: no; sid: 1;)
```

8.26 Kerberos Keywords

8.26.1 krb5_msg_type

This keyword allows to match the Kerberos messages by its type (integer). It is possible to specify the following values defined in RFC4120:

- 10 (AS-REQ)
- 11 (AS-REP)
- 12 (TGS-REQ)
- 13 (TGS-REP)
- 30 (ERROR)

Syntax:

```
krb5_msg_type:<number>
```

Signature examples:

```
alert krb5 any any -> any any (msg:"Kerberos 5 AS-REQ message"; krb5_msg_type:10; sid:3;
  ↳rev:1;)
alert krb5 any any -> any any (msg:"Kerberos 5 AS-REP message"; krb5_msg_type:11; sid:4;
  ↳rev:1;)
alert krb5 any any -> any any (msg:"Kerberos 5 TGS-REQ message"; krb5_msg_type:12; sid:5;
  ↳rev:1;)
alert krb5 any any -> any any (msg:"Kerberos 5 TGS-REP message"; krb5_msg_type:13; sid:6;
  ↳rev:1;)
alert krb5 any any -> any any (msg:"Kerberos 5 ERROR message"; krb5_msg_type:30; sid:7;
  ↳rev:1;)
```

Note

AP-REQ and AP-REP are not currently supported since those messages are embedded in other application protocols.

8.26.2 krb5_cname

Kerberos client name, provided in the ticket (for AS-REQ and TGS-REQ messages).

If the client name from the Kerberos message is composed of several parts, the name is compared to each part and the match will succeed if any is identical.

Comparison is case-sensitive.

Syntax:

```
krb5_cname; content:"name";
```

Signature example:

```
alert krb5 any any -> any any (msg:"Kerberos 5 des server name"; krb5_cname; content:"des
  ↳"; sid:4; rev:1;)
```

krb5_cname is a 'sticky buffer'.

krb5_cname can be used as `fast_pattern`.

krb5.cname supports multiple buffer matching, see *Multiple Buffer Matching*.

8.26.3 krb5_sname

Kerberos server name, provided in the ticket (for AS-REQ and TGS-REQ messages) or in the error message.

If the server name from the Kerberos message is composed of several parts, the name is compared to each part and the match will succeed if any is identical.

Comparison is case-sensitive.

Syntax:

```
krb5_sname; content:"name";
```

Signature example:

```
alert krb5 any any -> any any (msg:"Kerberos 5 krbtgt server name"; krb5_sname; content:
  ↳"krbtgt"; sid:5; rev:1;)
```

krb5_sname is a 'sticky buffer'.

krb5_sname can be used as fast_pattern.

krb5.sname supports multiple buffer matching, see *Multiple Buffer Matching*.

8.26.4 krb5_err_code

Kerberos error code (integer). This field is matched in Kerberos error messages only.

For a list of error codes, refer to RFC4120 section 7.5.9.

Syntax:

```
krb5_err_code:<number>
```

Signature example:

```
alert krb5 any any -> any any (msg:"Kerberos 5 error C_PRINCIPAL_UNKNOWN"; krb5_err_
  ↳code:6; sid:6; rev:1;)
```

8.26.5 krb5.weak_encryption (event)

Event raised if the encryption parameters selected by the server are weak or deprecated. For example, using a key size smaller than 128, or using deprecated ciphers like DES.

Syntax:

```
app-layer-event:krb5.weak_encryption
```

Signature example:

```
alert krb5 any any -> any any (msg:"SURICATA Kerberos 5 weak encryption parameters";
  ↳flow:to_client; app-layer-event:krb5.weak_encryption; classtype:protocol-command-
  ↳decode; sid:22260001; rev:1;)
```

8.26.6 krb5.malformed_data (event)

Event raised in case of a protocol decoding error.

Syntax:

```
app-layer-event:krb5.malformed_data
```

Signature example:

```
alert krb5 any any -> any any (msg:"SURICATA Kerberos 5 malformed request data"; flow:to_
  ↳server; app-layer-event:krb5.malformed_data; classtype:protocol-command-decode;
  ↳sid:22260000; rev:1;)
```

8.26.7 krb5.ticket_encryption

Kerberos ticket encryption (enumeration).

For a list of encryption types, refer to RFC3961 section 8.

Syntax:

```
krb5.ticket_encryption: (!)"weak" or (space or comma)-separated list of integer or ↵  
↵string values for an encryption type
```

Signature example:

```
alert krb5 any any -> any any (krb5.ticket_encryption: weak; sid:1;)  
alert krb5 any any -> any any (krb5.ticket_encryption: 23; sid:2;)  
alert krb5 any any -> any any (krb5.ticket_encryption: rc4-hmac,rc4-hmac-exp; sid:3;)
```

8.27 SMB Keywords

SMB keywords used in both SMB1 and SMB2 protocols.

8.27.1 smb.named_pipe

Match on SMB named pipe in tree connect.

Examples:

```
smb.named_pipe; content:"IPC"; endswith;  
smb.named_pipe; content:"strange"; nocase; pcre:"/really$/";
```

smb.named_pipe is a 'sticky buffer'.

smb.named_pipe can be used as fast_pattern.

8.27.2 smb.share

Match on SMB share name in tree connect.

Examples:

```
smb.share; content:"shared"; endswith;  
smb.share; content:"strange"; nocase; pcre:"/really$/";
```

smb.share is a 'sticky buffer'.

smb.share can be used as fast_pattern.

8.27.3 smb.ntlmssp_user

Match on SMB ntlmssp user in session setup.

Examples:

```
smb.ntlmssp_user; content:"doe"; endswith;  
smb.ntlmssp_user; content:"doe"; nocase; pcre:"/j(ohn|ane).*doe$/";
```

smb.ntlmssp_user is a 'sticky buffer'.

smb.ntlmssp_user can be used as fast_pattern.

8.27.4 smb.ntlmssp_domain

Match on SMB ntlmssp domain in session setup.

Examples:

```
smb.ntlmssp_domain; content:"home"; endswith;
smb.ntlmssp_domain; content:"home"; nocase; pcre:"/home(sweet)*$/";
```

smb.ntlmssp_domain is a 'sticky buffer'.

smb.ntlmssp_domain can be used as fast_pattern.

8.27.5 smb.version

Keyword to match on the SMB version seen in an SMB transaction.

Signature Example:

```
alert smb $HOME_NET any -> any any (msg:"SMBv1 version rule"; smb.version:1; sid:1;)
```

```
alert smb $HOME_NET any -> any any (msg:"SMBv2 version rule"; smb.version:2; sid:2;)
```

Matching in transition from SMBv1 to SMBv2

In the initial protocol negotiation request, a client supporting SMBv1 and SMBv2 can send an initial SMBv1 request and receive a SMBv2 response from server, indicating that SMBv2 will be used.

This first SMBv2 response made by the server will match as SMBv1, since the entire transaction will be considered a SMBv1 transaction.

Will smb.version match SMBv3 traffic?

Yes, it will match SMBv3 messages using *smb.version:2;*, which will match SMBv2 and SMBv3, since they use the same version identifier in the SMB header.

This keyword will use the Protocol ID specified in SMB header to determine the version. Here is a summary of the Protocol ID codes:

- 0xffSMB is SMBv1 header
- 0xfeSMB is SMBv2 normal header (can be sync or async)
- 0xfdSMB is SMBv2 transform header. This is only valid for the SMB 3.x dialect family.
- 0xfcSMB is SMBv2 transform compression header (can be chained or unchained). These ones require the use of the 3.1.1 dialect.

The Protocol ID in the header distinguishes only SMBv1 and SMBv2 since they are completely different protocols with entirely different message formats, types and implementations.

On the other hand, SMBv3 is more like an extension of SMBv2. When using SMBv2 we can select one of the following dialects for the conversation between client and server:

- 2.0.2
- 2.1
- 3.0
- 3.0.2
- 3.1.1

We say we are using SMBv3 when we select a 3.x dialect for the conversation, so you can use SMB 3.0, SMB 3.0.2 or SMB 3.1.1. The higher you choose, the more capabilities you have, but the message syntax and message command number remains the same.

SMB version and dialect are separate components. In the case of SMBv3 for instance, the SMB version will be 2 but the dialect will be 3.x. Dialect specification is not available currently via keyword.

8.27.6 file.name

The `file.name` keyword can be used at the SMB application level.

Signature Example:

```
alert smb any any -> any any (msg:"SMB file.name usage"; file.name; content:"file.txt"; classtype:bad-unknown; sid:1; rev:1;)
```

For additional information on the `file.name` keyword, see [File Keywords](#).

8.28 SNMP keywords

8.28.1 snmp.version

SNMP protocol version (integer). Expected values are 1, 2 (for version 2c) or 3.

`snmp.version` uses an, :ref:` unsigned 32-bits integer <rules-integer-keywords>`.

Syntax:

```
snmp.version: [op] <number>
```

The version can be matched exactly, or compared using the `_op_` setting:

```
snmp.version:3      # exactly 3
snmp.version:<3     # smaller than 3
snmp.version:>=2    # greater or equal than 2
```

Signature example:

```
alert snmp any any -> any any (msg:"old SNMP version (<3)"; snmp.version:<3; sid:1; rev:1;)
```

8.28.2 snmp.community

SNMP community strings are like passwords for SNMP messages in version 1 and 2c. In version 3, the community string is likely to be encrypted. This keyword will not match if the value is not accessible.

The default value for the read-only community string is often "public", and "private" for the read-write community string.

Comparison is case-sensitive.

Syntax:

```
snmp.community; content:"private";
```

Signature example:

```
alert snmp any any -> any any (msg:"SNMP community private"; snmp.community; content:"private"; sid:2; rev:1;)
```

`snmp.community` is a 'sticky buffer'.

`snmp.community` can be used as `fast_pattern`.

8.28.3 snmp.usm

SNMP User-based Security Model (USM) is used in version 3. It corresponds to the user name.

Comparison is case-sensitive.

Syntax:

```
snmp.usm; content:"admin";
```

Signature example:

```
alert snmp any any -> any any (msg:"SNMP usm admin"; snmp.usm; content:"admin"; sid:2; rev:1;)
```

`snmp.usm` is a 'sticky buffer'.

`snmp.usm` can be used as `fast_pattern`.

8.28.4 snmp.pdu_type

SNMP PDU type (integer).

`snmp.pdu_type` uses an, :ref:`unsigned 32-bits integer <rules-integer-keywords>`.

Common values are:

- 0: GetRequest
- 1: GetNextRequest
- 2: Response
- 3: SetRequest
- 4: TrapV1 (obsolete, was the old Trap-PDU in SNMPv1)
- 5: GetBulkRequest
- 6: InformRequest
- 7: TrapV2
- 8: Report

This keyword will not match if the value is not accessible within (for ex, an encrypted SNMP v3 message).

Syntax:

```
snmp.pdu_type:<number>
```

Signature example:

```
alert snmp any any -> any any (msg:"SNMP response"; snmp.pdu_type:2; sid:3; rev:1;)
```

8.28.5 snmp.trap_type

SNMP Trap type (integer).

snmp.trap_type uses an, :ref:` unsigned 8-bits integer <rules-integer-keywords>`.

You can specify the value as an integer or a string:

- 0: coldstart
- 1: warmstart
- 2: linkdown
- 3: linkup
- 4: authenticationfailure
- 5: egpneighborloss
- 6: enterprisespecific

Syntax:

```
snmp.trap_type:(mode) <number or string>
```

Signature example:

```
alert snmp any any -> any 162 (msg:"SNMP trap cold start"; snmp.pdu_type:trap_v1; snmp.  
->trap_type:coldstart; sid:3; rev:1;)
```

8.29 NTP Keywords

8.29.1 ntp.mode

NTP mode. This keyword accepts either an integer or one of the known mode names.

ntp.mode uses an *unsigned 8-bit integer*.

Syntax:

```
ntp.mode:[op]<number>  
ntp.mode:[!]reserved  
ntp.mode:[!]symmetric_active  
ntp.mode:[!]symmetric_passive  
ntp.mode:[!]client  
ntp.mode:[!]server  
ntp.mode:[!]broadcast  
ntp.mode:[!]control  
ntp.mode:[!]private
```

Examples:

```
ntp.mode:3  
ntp.mode:>3  
ntp.mode:client  
ntp.mode:!server
```

Signature Example:

```
alert ntp any any -> any any (msg:"NTP client mode"; ntp.mode:client; sid:1; rev:1;)
```

8.29.2 ntp.reference_id

`ntp.reference_id` is a sticky buffer exposing the 4-byte NTP reference ID.

Examples:

```
ntp.reference_id; content:"RATE";
ntp.reference_id; content:"|0a 00 00 01|";
```

Signature Example:

```
alert ntp any any -> any any (msg:"NTP reference ID RATE"; ntp.reference_id; content:"RATE"; sid:4; rev:1;)
```

8.29.3 ntp.stratum

NTP stratum (integer).

`ntp.stratum` uses an *unsigned 8-bit integer*.

Syntax:

```
ntp.stratum: [op] <number>
```

The stratum can be matched exactly, or compared using the `op` setting:

```
ntp.stratum:2      # exactly 2
ntp.stratum:<16    # smaller than 16
ntp.stratum:>=1    # greater or equal than 1
```

Signature Example:

```
alert ntp any any -> any any (msg:"NTP stratum 2"; ntp.stratum:2; sid:2; rev:1;)
```

8.29.4 ntp.version

NTP protocol version (integer). Expected values are 3 and 4.

`ntp.version` uses an *unsigned 8-bit integer*.

Syntax:

```
ntp.version: [op] <number>
```

The version can be matched exactly, or compared using the `op` setting:

```
ntp.version:4      # exactly 4
ntp.version:<4      # smaller than 4
ntp.version:>=3     # greater or equal than 3
```

Signature Example:

```
alert ntp any any -> any any (msg:"NTP version 4"; ntp.version:4; sid:3; rev:1;)
```

8.30 Base64 keywords

Suricata supports decoding base64 encoded data from buffers and matching on the decoded data.

This is achieved by using two keywords, `base64_decode` and `base64_data`. Both keywords must be used in order to generate an alert.

8.30.1 base64_decode

Decodes base64 data from a buffer and makes it available for the `base64_data` function.

We recommend using the base64 transform instead -- see *from_base64*.

Syntax:

```
base64_decode:bytes <value>, offset <value>, relative;
```

The `bytes` option specifies how many bytes Suricata should decode and make available for `base64_data`. This number is limited to 64KiB. The decoding will stop at the end of the buffer.

The `offset` option specifies how many bytes Suricata should skip before decoding. Bytes are skipped relative to the start of the payload buffer if the `relative` is not set.

The `relative` option makes the decoding start relative to the previous content match. Default behavior is to start at the beginning of the buffer. This option makes `offset` skip bytes relative to the previous match.

Note

Regarding `relative` and `base64_decode`:

The content match that you want to decode relative to must be the first match in the stream.

Note

`base64_decode` follows RFC 4648 by default i.e. encounter with any character that is not found in the base64 alphabet leads to rejection of that character and the rest of the string.

See Redmine Bug 5223: <https://redmine.openinfosecfoundation.org/issues/5223> and RFC 4648: <https://www.rfc-editor.org/rfc/rfc4648#section-3.3>

8.30.2 base64_data

`base64_data` is a sticky buffer.

Enables content matching on the data previously decoded by `base64_decode`.

8.30.3 Example

Here is an example of a rule matching on the base64 encoded string "test" that is found inside the `http_uri` buffer.

It starts decoding relative to the known string "somestring" with the known offset of 1. This must be the first occurrence of "somestring" in the buffer.

Example:

Buffer content:

```
http_uri = "GET /en/somestring&dGVzdAo=&not_base64"
```

Rule:

```
alert http any any -> any any (msg:"Example"; http.uri; content:"somestring"; \
  base64_decode:bytes 8, offset 1, relative; \
  base64_data; content:"test"; sid:10001; rev:1;)
```

(continues on next page)

(continued from previous page)

Buffer content:

`http_uri = "GET /en/somestring&dGVzdAo=¬_base64"`

Rule:

```

alert http any any -> any any (msg:"Example"; content:"somestring"; http_uri; \
    base64_decode:bytes 8, offset 1, relative; \
    base64_data; content:"test"; sid:10001; rev:1;)

```

Note

base64_data cannot be used with fast_pattern and will result in a rule load error.

8.31 SIP Keywords

The SIP keywords are implemented as sticky buffers and can be used to match on fields in SIP messages.

As described in RFC3261, common header field names can be represented in a short form. In such cases, the header name is normalized to its regular form to be matched by its corresponding sticky buffer.

Keyword	Direction
sip.method	Request
sip.uri	Request
sip.request_line	Request
sip.stat_code	Response
sip.stat_msg	Response
sip.response_line	Response
sip.protocol	Both
sip.from	Both
sip.to	Both
sip.via	Both
sip.user_agent	Both
sip.content_type	Both
sip.content_length	Both

8.31.1 sip.method

This keyword matches on the method found in a SIP request.

Syntax

```
sip.method; content:<method>;
```

Examples of methods are:

- INVITE
- BYE
- REGISTER
- CANCEL

- ACK
- OPTIONS

Examples

```
sip.method; content:"INVITE";
```

8.31.2 sip.uri

This keyword matches on the uri found in a SIP request.

Syntax

```
sip.uri; content:<uri>;
```

Where <uri> is an uri that follows the SIP URI scheme.

Examples

```
sip.uri; content:"sip:sip.url.org";
```

8.31.3 sip.request_line

This keyword forces the whole SIP request line to be inspected.

Syntax

```
sip.request_line; content:<request_line>;
```

Where <request_line> is a partial or full line.

Examples

```
sip.request_line; content:"REGISTER sip:sip.url.org SIP/2.0"
```

8.31.4 sip.stat_code

This keyword matches on the status code found in a SIP response.

Syntax

```
sip.stat_code; content:<stat_code>
```

Where <status_code> belongs to one of the following groups of codes:

- 1xx - Provisional Responses
- 2xx - Successful Responses
- 3xx - Redirection Responses
- 4xx - Client Failure Responses
- 5xx - Server Failure Responses
- 6xx - Global Failure Responses

Examples

```
sip.stat_code; content:"100";
```

8.31.5 sip.stat_msg

This keyword matches on the status message found in a SIP response.

Syntax

```
sip.stat_msg; content:<stat_msg>
```

Where <stat_msg> is a reason phrase associated to a status code.

Examples

```
sip.stat_msg; content:"Trying";
```

8.31.6 sip.response_line

This keyword forces the whole SIP response line to be inspected.

Syntax

```
sip.response_line; content:<response_line>;
```

Where <response_line> is a partial or full line.

Examples

```
sip.response_line; content:"SIP/2.0 100 OK"
```

8.31.7 sip.protocol

This keyword matches the protocol field from a SIP request or response line.

If the response line is 'SIP/2.0 100 OK', then this buffer will contain 'SIP/2.0'

Syntax

```
sip.protocol; content:<protocol>
```

Where <protocol> is the SIP protocol version.

Example

```
sip.protocol; content:"SIP/2.0"
```

8.31.8 sip.from

This keyword matches on the From field that can be present in SIP headers. It matches both the regular and short forms, though it cannot distinguish between them.

Syntax

```
sip.from; content:<from>
```

Where <from> is the value of the From header.

Example

```
sip.from; content:"user"
```

8.31.9 sip.to

This keyword matches on the To field that can be present in SIP headers. It matches both the regular and short forms, though it cannot distinguish between them.

Syntax

```
sip.to; content:<to>
```

Where <to> is the value of the To header.

Example

```
sip.to; content:"user"
```

8.31.10 sip.via

This keyword matches on the Via field that can be present in SIP headers. It matches both the regular and short forms, though it cannot distinguish between them.

Syntax

```
sip.via; content:<via>
```

Where <via> is the value of the Via header.

Example

```
sip.via; content:"SIP/2.0/UDP"
```

8.31.11 sip.user_agent

This keyword matches on the User-Agent field that can be present in SIP headers.

Syntax

```
sip.user_agent; content:<user_agent>
```

Where <user_agent> is the value of the User-Agent header.

Example

```
sip.user_agent; content:"Asterisk"
```

8.31.12 sip.content_type

This keyword matches on the Content-Type field that can be present in SIP headers. It matches both the regular and short forms, though it cannot distinguish between them.

Syntax

```
sip.content_type; content:<content_type>
```

Where <content_type> is the value of the Content-Type header.

Example

```
sip.content_type; content:"application/sdp"
```

8.31.13 sip.content_length

This keyword matches on the Content-Length field that can be present in SIP headers. It matches both the regular and short forms, though it cannot distinguish between them.

Syntax

```
sip.content_length; content:<content_length>
```

Where <content_length> is the value of the Content-Length header.

Example

```
sip.content_length; content:"200"
```

8.32 SDP Keywords

The SDP keywords are implemented as sticky buffers and can be used to match on fields in SDP messages.

Keyword	Direction
sdp.origin	Both
sdp.session_name	Both
sdp.session_info	Both
sdp.uri	Both
sdp.email	Both
sdp.connection_data	Both
sdp.bandwidth	Both
sdp.time	Both
sdp.repeat_time	Both
sdp.timezone	Both
sdp.encryption_key	Both
sdp.attribute	Both
sdp.media.media	Both
sdp.media.session_info	Both
sdp.media.connection_data	Both
sdp.media.encryption_key	Both

8.32.1 sdp.origin

This keyword matches on the originator found in an SDP request or response.

Syntax

```
sdp.origin; content:<origin>;
```

Where <origin> is an originator that follows the SDP Origin (o=) scheme.

Examples

```
sdp.origin; content:"SIPPS 105015165 105015162 IN IP4 192.168.1.2";
```

8.32.2 sdp.session_name

This keyword matches on the session name found in an SDP request or response.

Syntax

```
sdp.session_name; content:<session_name>;
```

Where <session_name> is a name that follows the SDP Session name (s=) scheme.

Examples

```
sdp.session_name; content:"SIP call";
```

8.32.3 sdp.session_info

This keyword matches on the session information found in an SDP request or response.

Syntax

```
sdp.session_info; content:<session_info>;
```

Where <session_info> is a description that follows the SDP Session information (i=) scheme.

Examples

```
sdp.session_info; content:"Session Description Protocol";
```

8.32.4 sdp.uri

This keyword matches on the URI found in an SDP request or response.

Syntax

```
sdp.uri; content:<uri>;
```

Where <uri> is a URI (u=) that follows the SDP scheme.

Examples

```
sdp.uri; content:"https://www.sdp.proto"
```

8.32.5 sdp.email

This keyword matches on the email found in an SDP request or response.

Syntax

```
sdp.email; content:<email>
```

Where <email> is an email address (e=) that follows the SDP scheme.

Examples

```
sdp.email; content:"j.doe@example.com (Jane Doe)";
```

8.32.6 sdp.phone_number

This keyword matches on the phone number found in an SDP request or response.

Syntax

```
sdp.phone_number; content:<phone_number>
```

Where <phone_number> is a phone number (p=) that follows the SDP scheme.

Examples

```
sdp.phone_number; content:"+1 617 555-6011 (Jane Doe)";
```

8.32.7 sdp.connection_data

This keyword matches on the connection found in an SDP request or response.

Syntax

```
sdp.connection_data; content:<connection_data>;
```

Where <connection_data> is a connection (c=) that follows the SDP scheme.

Examples

```
sdp.connection_data; content:"IN IP4 192.168.1.2"
```

8.32.8 sdp.bandwidth

This keyword matches on the bandwidths found in an SDP request or response.

Syntax

```
sdp.bandwidth; content:<bandwidth>
```

Where <bandwidth> is a bandwidth (b=) that follows the SDP scheme.

Example

```
sdp.bandwidth; content:"AS:64"
```

8.32.9 sdp.time

This keyword matches on the time found in an SDP request or response.

Syntax

```
sdp.time; content:<time>
```

Where <time> is a time (t=) that follows the SDP scheme.

Example

```
sdp.time; content:"3034423619 3042462419"
```

8.32.10 sdp.repeat_time

This keyword matches on the repeat time found in an SDP request or response.

Syntax

```
sdp.repeat_time; content:<repeat_time>
```

Where <repeat_time> is a repeat time (r=) that follows the SDP scheme.

Example

```
sdp.repeat_time; content:"604800 3600 0 90000"
```

8.32.11 sdp.timezone

This keyword matches on the timezone found in an SDP request or response.

Syntax

```
sdp.timezone; content:<timezone>
```

Where <timezone> is a timezone (z=) that follows the SDP scheme.

Example

```
sdp.timezone; content:"2882844526 -1h 2898848070 0"
```

8.32.12 sdp.encryption_key

This keyword matches on the encryption key found in an SDP request or response.

Syntax

```
sdp.encryption_key; content:<encryption_key>
```

Where <encryption_key> is a key (k=) that follows the SDP scheme.

Example

```
sdp.encryption_key; content:"prompt"
```

8.32.13 sdp.attribute

This keyword matches on the attributes found in an SDP request or response.

Syntax

```
sdp.attribute; content:<attribute>
```

Where <attribute> is an attribute (a=) that follows the SDP scheme.

Example

```
sdp.attribute; content:"sendrecv"
```

8.32.14 sdp.media.media

This keyword matches on the Media subfield of a Media description field found in an SDP request or response.

Syntax

```
sdp.media.media; content:<media>
```

Where <media> is a media (m=) that follows the SDP scheme.

Example

```
sdp.media.media; content:"audio 30000 RTP/AVP 0 8 97 2 3"
```

8.32.15 sdp.media.session_info

This keyword matches on the Session information subfield of a Media description field found in an SDP request or response.

Syntax

```
sdp.media.session_info; content:<session_info>
```

Where <session_info> is a description (i=) that follows the SDP scheme.

Example

```
sdp.media.session_info; content:"Session Description Protocol"
```

8.32.16 sdp.media.connection_data

This keyword matches on the Connection data subfield of a Media description field found in an SDP request or response.

Syntax

```
sdp.media.connection_data; content:<connection_data>
```

Where <connection_data> is a connection (c=) that follows the SDP scheme.

Example

```
sdp.media.connection_data; content:"IN IP4 192.168.1.2"
```

8.32.17 sdp.media.encryption_key

This keyword matches on the Encryption key subfield of a Media description field found in an SDP request or response.

Syntax

```
sdp.media.encryption_key; content:<encryption_key>
```

Where <encryption_key> is a key (k=) that follows the SDP scheme.

Example

```
sdp.media.encryption_key; content:"prompt"
```

8.33 RFB Keywords

The `rfb.name` and `rfb.sectype` keywords can be used for matching on various properties of RFB (Remote Framebuffer, i.e. VNC) handshakes.

8.33.1 rfb.name

Match on the value of the RFB desktop name field.

Examples:

```
rfb.name; content:"Alice's desktop";
rfb.name; pcre:"/.* \(\screen [0-9]\)\$/";
```

`rfb.name` is a 'sticky buffer'.

`rfb.name` can be used as `fast_pattern`.

8.33.2 rfb.secrestult

Match on the value of the RFB security result, e.g. `ok`, `fail`, `toomany` or `unknown`.

`rfb.secrestult` uses an *unsigned 32-bit integer*.

Examples:

```
rfb.secrestult: ok;
rfb.secrestult: !0;
rfb.secrestult: unknown;
```

8.33.3 rfb.sectype

Match on the value of the RFB security type field, e.g. 2 for VNC challenge-response authentication, 0 for no authentication, and 30 for Apple's custom Remote Desktop authentication.

`rfb.sectype` uses an *unsigned 32-bit integer*.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- `>` (greater than)
- `<` (less than)
- `>=` (greater than or equal)
- `<=` (less than or equal)

Examples:

```
rfb.sectype:2;
rfb.sectype:>=3;
```

8.33.4 Additional information

More information on the protocol can be found here: <https://tools.ietf.org/html/rfc6143>

8.34 MQTT Keywords

Various keywords can be used for matching on fields in fixed and variable headers of MQTT messages as well as payload values.

8.34.1 mqtt.protocol_version

Match on the value of the MQTT protocol version field in the fixed header.

mqtt.protocol_version uses an *unsigned 8-bit integer*.

The format of the keyword:

```
mqtt.protocol_version:<min>-<max>;  
mqtt.protocol_version:[<|>]<number>;  
mqtt.protocol_version:<value>;
```

Examples:

```
mqtt.protocol_version:5;
```

8.34.2 mqtt.type

Match on the MQTT message type (also: control packet type). Valid values are :

- CONNECT
- CONNACK
- PUBLISH
- PUBACK
- PUBREC
- PUBREL
- PUBCOMP
- SUBSCRIBE
- SUBACK
- UNSUBSCRIBE
- UNSUBACK
- PINGREQ
- PINGRESP
- DISCONNECT
- AUTH
- UNASSIGNED

where UNASSIGNED refers to message type code 0.

mqtt.type uses an *unsigned 8-bits integer*.

Examples:

```
mqtt.type:CONNECT;
mqtt.type:PUBLISH;
mqtt.type:2;
```

8.34.3 mqtt.flags

Match on a combination of MQTT header flags, separated by commas (,). Flags may be prefixed by ! to indicate negation, i.e. a flag prefixed by ! must *not* be set to match.

mqtt.flags uses an *unsigned 8-bits integer*

Valid flags are:

- dup (duplicate message)
- retain (message should be retained on the broker)

Examples:

```
mqtt.flags:dup,!retain;
mqtt.flags:retain;
```

8.34.4 mqtt.qos

Match on the Quality of Service request code in the MQTT fixed header. Valid values are:

- 0 (fire and forget)
- 1 (at least one delivery)
- 2 (exactly one delivery)

Examples:

```
mqtt.qos:0;
mqtt.qos:2;
```

8.34.5 mqtt.reason_code

Match on the numeric value of the reason code that is used in MQTT 5.0 for some message types. Please refer to the specification for the meaning of these values, which are often specific to the message type in question.

mqtt.reason_code uses an *unsigned 8-bits integer*.

Examples:

```
# match on attempts to unsubscribe from a non-subscribed topic
mqtt.type:UNSUBACK; mqtt.reason_code:17;

# match on publications that were accepted but there were no subscribers
mqtt.type:PUBACK; mqtt.reason_code:16;

# match on connection attempts by banned clients
mqtt.CONNACK; mqtt.reason_code:138;

# match on failed connection attempts due to bad credentials
```

(continues on next page)

(continued from previous page)

```
mqtt.CONNACK; mqtt.reason_code:134;  
  
# match on connections terminated by server shutdowns  
mqtt.DISCONNECT; mqtt.reason_code:139;
```

This keyword is also available under the alias `mqtt.connack.return_code` for completeness.

8.34.6 mqtt.connack.session_present

Match on the MQTT CONNACK `session_present` flag. Values can be yes, true, no or false.

Examples:

```
mqtt.CONNACK; mqtt.connack.session_present:true;
```

8.34.7 mqtt.connect.clientid

Match on the self-assigned client ID in the MQTT CONNECT message.

Examples:

```
mqtt.connect.clientid; pcre:"/^mosq.*"/;  
mqtt.connect.clientid; content:"myclient";
```

`mqtt.connect.clientid` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.8 mqtt.connect.flags

Match on a combination of MQTT CONNECT flags, separated by commas (,). Flags may be prefixed by ! to indicate negation, i.e. a flag prefixed by ! must *not* be set to match.

`mqtt.connect.flags` uses an *unsigned 8-bits integer*

Valid flags are:

- `username` (message contains a username)
- `password` (message contains a password)
- `will` (message contains a will definition)
- `will_retain` (will should be retained on broker)
- `clean_session` (start with a clean session)

Examples:

```
mqtt.connect.flags:username,password,!will;  
mqtt.connect.flags:username,!password;  
mqtt.connect.flags:clean_session;
```

8.34.9 mqtt.connect.password

Match on the password credential in the MQTT CONNECT message.

Examples:

```
mqtt.connect.password; pcre:"/^123[0-9]*/";
mqtt.connect.password; content:"swordfish";
```

`mqtt.connect.password` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.10 mqtt.connect.protocol_string

Match on the protocol string in the MQTT CONNECT message. In contrast to `mqtt.protocol_version` this is a property that is only really relevant in the initial CONNECT communication and never used again; hence it is organized under `mqtt.connect`.

Examples:

```
mqtt.connect.protocol_string; content:"MQTT";
mqtt.connect.protocol_string; content:"MQIsdp";
```

`mqtt.connect.protocol_string` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.11 mqtt.connect.username

Match on the username credential in the MQTT CONNECT message.

Examples:

```
mqtt.connect.username; content:"benson";
```

`mqtt.connect.username` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.12 mqtt.connect.willmessage

Match on the will message in the MQTT CONNECT message, if a will is defined.

Examples:

```
mqtt.connect.willmessage; pcre:"/^fooba[rz]"/";
mqtt.connect.willmessage; content:"hunter2";
```

`mqtt.connect.willmessage` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.13 mqtt.connect.willtopic

Match on the will topic in the MQTT CONNECT message, if a will is defined.

Examples:

```
mqtt.connect.willtopic; pcre:"/^hunter[0-9]"/;
```

`mqtt.connect.willtopic` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.14 mqtt.publish.message

Match on the payload to be published in the MQTT PUBLISH message.

Examples:

```
mqtt.type:PUBLISH; mqtt.publish.message; pcre:"/uid=[0-9]+/";  
# match on published JPEG images  
mqtt.type:PUBLISH; mqtt.publish.message; content:"|FF D8 FF E0|"; startswith;
```

`mqtt.publish.message` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.15 mqtt.publish.topic

Match on the topic to be published to in the MQTT PUBLISH message.

Examples:

```
mqtt.publish.topic; content:"mytopic";
```

`mqtt.publish.topic` is a 'sticky buffer' and can be used as `fast_pattern`.

8.34.16 mqtt.subscribe.topic

Match on any of the topics subscribed to in a MQTT SUBSCRIBE message.

Examples:

```
mqtt.subscribe.topic; content:"mytopic";
```

`mqtt.subscribe.topic` is a 'sticky buffer' and can be used as `fast_pattern`.

`mqtt.subscribe.topic` supports multiple buffer matching, see *Multiple Buffer Matching*.

8.34.17 mqtt.unsubscribe.topic

Match on any of the topics unsubscribed from in a MQTT UNSUBSCRIBE message.

Examples:

```
mqtt.unsubscribe.topic; content:"mytopic";
```

`mqtt.unsubscribe.topic` is a 'sticky buffer' and can be used as `fast_pattern`.

`mqtt.unsubscribe.topic` supports multiple buffer matching, see *Multiple Buffer Matching*.

8.34.18 Additional information

More information on the protocol can be found here:

- MQTT 3.1: <https://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/mqtt-v3r1.html>
- MQTT 3.1.1: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- MQTT 5.0: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

8.35 IKE Keywords

The keywords

- `ike.init_spi`
- `ike.resp_spi`
- `ike.chosen_sa_attribute`

- `ike.exctype`
- `ike.vendor`
- `ike.key_exchange_payload`
- `ike.key_exchange_payload_length`
- `ike.nonce_payload`
- `ike.nonce_payload_length`

can be used for matching on various properties of IKE connections.

8.35.1 `ike.init_spi`, `ike.resp_spi`

Match on an exact value of the Security Parameter Index (SPI) for the Initiator or Responder.

Examples:

```
ike.init_spi; content:"18fe9b731f9f8034";
ike.resp_spi; content:"a00b8ef0902bb8ec";
```

`ike.init_spi` and `ike.resp_spi` are 'sticky buffer'.

`ike.init_spi` and `ike.resp_spi` can be used as `fast_pattern`.

8.35.2 `ike.chosen_sa_attribute`

Match on an attribute value of the chosen Security Association (SA) by the Responder. Supported for IKEv1 are: `alg_enc`, `alg_hash`, `alg_auth`, `alg_dh`, `alg_prf`, `sa_group_type`, `sa_life_type`, `sa_life_duration`, `sa_key_length` and `sa_field_size`. IKEv2 supports `alg_enc`, `alg_auth`, `alg_prf` and `alg_dh`.

If there is more than one chosen SA the event `MultipleServerProposal` is set. The attributes of the first SA are used for this keyword.

Examples:

```
ike.chosen_sa_attribute:alg_hash=2;
ike.chosen_sa_attribute:sa_key_length=128;
```

8.35.3 `ike.exctype`

Match on the value of the Exchange Type.

`ike.exctype` uses an *unsigned 8-bit integer*.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- `>` (greater than)
- `<` (less than)
- `>=` (greater than or equal)
- `<=` (less than or equal)
- `arg1-arg2` (range)

Examples:

```
ike.exctype:5;
ike.exctype:>=2;
```

8.35.4 ike.vendor

Match a vendor ID against the list of collected vendor IDs.

Examples:

```
ike.vendor:4a131c81070358455c5728f20e95452f;
```

`ike.vendor` supports multiple buffer matching, see *Multiple Buffer Matching*.

8.35.5 ike.key_exchange_payload

Match against the public key exchange payload (e.g. Diffie-Hellman) of the server or client.

Examples:

```
ike.key_exchange_payload; content:"|6d026d5616c45be05e5b898411e9|"
```

`ike.key_exchange_payload` is a 'sticky buffer'.

`ike.key_exchange_payload` can be used as `fast_pattern`.

8.35.6 ike.key_exchange_payload_length

Match against the length of the public key exchange payload (e.g. Diffie-Hellman) of the server or client.

`ike.key_exchange_payload_length` uses an *unsigned 32-bit integer*.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- > (greater than)
- < (less than)
- >= (greater than or equal)
- <= (less than or equal)
- arg1-arg2 (range)

Examples:

```
ike.key_exchange_payload_length:>132
```

8.35.7 ike.nonce_payload

Match against the nonce of the server or client.

Examples:

```
ike.nonce_payload; content:"|6d026d5616c45be05e5b898411e9|"
```

`ike.nonce_payload` is a 'sticky buffer'.

`ike.nonce_payload` can be used as `fast_pattern`.

8.35.8 ike.nonce_payload_length

Match against the length of the nonce of the server or client.

`ike.nonce_payload_length` uses an *unsigned 32-bit integer*.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- > (greater than)
- < (less than)
- >= (greater than or equal)
- <= (less than or equal)
- arg1-arg2 (range)

Examples:

```
ike.nonce_payload_length:132
ike.nonce_payload_length:>132
```

8.35.9 Additional information

More information on the protocol and the data contained in it can be found here: <https://tools.ietf.org/html/rfc2409>

8.36 HTTP2 Keywords

HTTP2 frames are grouped into transactions based on the stream identifier it is not 0. For frames with stream identifier 0, whose effects are global for the connection, a transaction is created for each frame.

8.36.1 Frames

The HTTP2 parser supports the following frames (as defined by Suricata) which are created for each HTTP2 frame (as defined by the HTTP2 RFC) :

- http2.hdr
- http2.data
- http2.pdu

8.36.2 http2.frame_type

Match on the frame type present in a transaction.

Examples:

```
http2.frame_type:GOAWAY;
```

8.36.3 http2.errorcode

Match on the error code in a GOWAY or RST_STREAM frame

Examples:

```
http2.errorcode: NO_ERROR;
http2.errorcode: INADEQUATE_SECURITY;
```

8.36.4 http2.priority

Match on the value of the HTTP2 priority field present in a PRIORITY or HEADERS frame.

http2.priority uses an *unsigned 8-bit integer*.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- > (greater than)
- < (less than)
- x-y (range between values x and y)

Examples:

```
http2.priority:2;  
http2.priority:>100;  
http2.priority:32-64;
```

8.36.5 http2.window

Match on the value of the HTTP2 value field present in a WINDOWUPDATE frame.

http2.window uses an *unsigned 32-bit integer*.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- > (greater than)
- < (less than)
- x-y (range between values x and y)

Examples:

```
http2.window:1;  
http2.window:<1000000;
```

8.36.6 http2.size_update

Match on the size of the HTTP2 Dynamic Headers Table. More information on the protocol can be found here: <https://tools.ietf.org/html/rfc7541#section-6.3>

http2.size_update uses an *unsigned 64-bit integer*.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- > (greater than)
- < (less than)
- x-y (range between values x and y)

Examples:

```
http2.size_update:1234;  
http2.size_update:>4096;
```

8.36.7 http2.settings

Match on the name and value of a HTTP2 setting from a SETTINGS frame.

This keyword takes a numeric argument after a colon and supports additional qualifiers, such as:

- > (greater than)
- < (less than)
- x-y (range between values x and y)

Examples:

```
http2.settings:SETTINGS_ENABLE_PUSH=0;
http2.settings:SETTINGS_HEADER_TABLE_SIZE>4096;
```

8.36.8 http2.header_name

Match on the name of a HTTP2 header from a HEADER frame (or PUSH_PROMISE or CONTINUATION).

Examples:

```
http2.header_name; content:"agent";
```

http2.header_name is a 'sticky buffer'.

http2.header_name can be used as fast_pattern.

http2.header_name supports multiple buffer matching, see *Multiple Buffer Matching*.

8.36.9 Additional information

More information on the protocol can be found here: <https://tools.ietf.org/html/rfc7540>

8.37 Quic Keywords

Suricata implements initial support for Quic by parsing the Quic version.

Suricata also derives a CYU hash for earlier versions of Quic.

Quic app-layer parsing must be enabled in the Suricata config file (set 'app-layer.protocols.quic.enabled' to 'yes').

8.37.1 quic.cyu.hash

Match on the CYU hash

Examples:

```
alert quic any any -> any any (msg:"QUIC CYU HASH"; \
  quic.cyu.hash; content:"7b3ceb1adc974ad360cfa634e8d0a730"; \
  sid:1;)
```

quic.cyu.hash supports multiple buffer matching, see *Multiple Buffer Matching*.

8.37.2 quic.cyu.string

Match on the CYU string

Examples:

```
alert quic any any -> any any (msg:"QUIC CYU STRING"; \
  quic.cyu.string; content:"46,PAD-SNI-VER-CCS-UAID-TCID-PDMD-SMHL-ICSL-NONP-MIDS-SCLS-
  CSCT-COPT-IRTT-CFCW-SFCW"; \
  sid:2;)
```

quic.cyu.string supports multiple buffer matching, see *Multiple Buffer Matching*.

8.37.3 quic.version

Sticky buffer for matching on the Quic header version in long headers.

Examples:

```
alert quic any any -> any any (msg:"QUIC VERSION"; \
  quic.version; content:"Q046"; \
  sid:3;)
```

8.37.4 Additional information

More information on CYU Hash can be found here: <https://engineering.salesforce.com/gquic-protocol-analysis-and-fingerprinting-in-zeek-a4178855d75f>

More information on the protocol can be found here: <https://datatracker.ietf.org/doc/html/draft-ietf-quic-transport-17>

8.38 NFS Keywords

8.38.1 file.name

The `file.name` keyword can be used at the NFS application level.

Signature Example:

```
alert nfs any any -> any any (msg:"NFS file.name usage"; file.name; content:"file.txt"; classtype:bad-unknown; sid:1; rev:1;)
```

For additional information on the `file.name` keyword, see *File Keywords*.

8.39 SMTP Keywords

8.39.1 file.name

The `file.name` keyword can be used at the SMTP application level.

Signature Example:

```
alert smtp any any -> any any (msg:"SMTP file.name usage"; file.name; content:"winmail.dat"; classtype:bad-unknown; sid:1; rev:1;)
```

For additional information on the `file.name` keyword, see *File Keywords*.

8.39.2 smtp.helo

SMTP helo is the parameter passed to the first HELO command from the client. This keyword matches per transaction, so it can match more than once per flow, even if the helo occurred only once at the beginning of the flow.

Syntax:

```
smtp.helo; content:"localhost";
```

Signature example:

```
alert smtp any any -> any any (msg:"SMTP helo localhost"; smtp.helo; content:"localhost";
  -> sid:2; rev:1;)
```

smtp.helo is a 'sticky buffer'.

smtp.helo can be used as fast_pattern.

This keyword maps to the eve.json log field smtp.helo

8.39.3 smtp.mail_from

SMTP mail from is the parameter passed to the first MAIL FROM command from the client.

Syntax:

```
smtp.mail_from; content:"spam";
```

Signature example:

```
alert smtp any any -> any any (msg:"SMTP mail from spam"; smtp.mail_from; content:"spam";
  ↳ sid:2; rev:1;)
```

smtp.mail_from is a 'sticky buffer'.

smtp.mail_from can be used as fast_pattern.

This keyword maps to the eve.json log field smtp.mail_from

8.39.4 smtp.rcpt_to

SMTP rcpt to is the one of the parameters passed to one RCPT TO command from the client.

Syntax:

```
smtp.rcpt_to; content:"sensitive@target";
```

Signature example:

```
alert smtp any any -> any any (msg:"SMTP rcpt to sensitive"; smtp.rcpt_to; content:
  ↳ "sensitive@target"; sid:2; rev:1;)
```

smtp.rcpt_to is a 'sticky buffer'.

smtp.rcpt_to is a 'multi buffer'.

smtp.rcpt_to can be used as fast_pattern.

This keyword maps to the eve.json log field smtp.rcpt_to[]

8.39.5 Frames

The SMTP parser supports the following frames:

- smtp.command_line
- smtp.response_line
- smtp.data
- smtp.stream

smtp.command_line

A single line from the client to the server. Multi-line commands will have a frame per line. Lines part of the SMTP DATA transfer are excluded.

```
alert smtp any any -> any any ( frame:smtp.command_line; content:"MAIL|20|FROM: "; startswith; sid:1;)
```

smtp.response_line

A single line from the server to the client. Multi-line commands will have a frame per line.

```
alert smtp any any -> any any ( frame:smtp.response_line; content:"354 go ahead"; startswith; sid:1;)
```

smtp.data

A streaming buffer containing the DATA bytes sent from client to server.

```
alert smtp any any -> any any ( frame:smtp.data; content:"Reply-To: "; startswith; content:"Subject "; distance:0; sid:1;)
```

smtp.stream

Streaming buffer of the entire TCP data for the SMTP session.

```
alert smtp any any -> any any ( flow:to_client; frame:smtp.stream; content:"250 ok|0d 0a|354 go ahead"; sid:1;)
```

8.40 WebSocket Keywords

8.40.1 websocket.payload

A sticky buffer on the unmasked payload, limited by suricata.yaml config value `websocket.max-payload-size`.

Examples:

```
websocket.payload; pcre:"/^123[0-9]*/";  
websocket.payload content:"swordfish";
```

`websocket.payload` is a 'sticky buffer' and can be used as `fast_pattern`.

8.40.2 websocket.flags

Matches on the websocket flags. It uses a 8-bit unsigned integer as value. Only the four upper bits are used.

The value can also be a list of strings (comma-separated), where each string is the name of a specific bit like *fin* and *comp*, and can be prefixed by *!* for negation.

`websocket.flags` uses an *unsigned 8-bits integer*

Examples:

```
websocket.flags:128;  
websocket.flags:&0x40=0x40;  
websocket.flags:fin,!comp;
```

8.40.3 websocket.mask

Matches on the websocket mask if any. It uses a 32-bit unsigned integer as value (big-endian).

`websocket.mask` uses an *unsigned 32-bits integer*

Examples:

```
websocket.mask:123456;
websocket.mask:>0;
```

8.40.4 websocket.opcode

Matches on the websocket opcode. It uses a 8-bit unsigned integer as value. Only 16 values are relevant. It can also be specified by text from the enumeration

websocket.opcode uses an *unsigned 8-bits integer*

Examples:

```
websocket.opcode:1;
websocket.opcode:>8;
websocket.opcode:ping;
```

8.41 Generic App Layer Keywords

8.41.1 app-layer-protocol

Match on the detected app-layer protocol.

Syntax:

```
app-layer-protocol:[!]<protocol>(,<mode>);
```

Examples:

```
app-layer-protocol:ssh;
app-layer-protocol:!tls;
app-layer-protocol:failed;
app-layer-protocol:http,final;
app-layer-protocol:http,to_server; app-layer-protocol:tls,to_client;
app-layer-protocol:http2,final; app-layer-protocol:http1,original;
app-layer-protocol:unknown;
```

A special value 'failed' can be used for matching on flows in which protocol detection failed. This can happen if Suricata doesn't know the protocol or when certain 'bail out' conditions happen.

A special value 'unknown' can be used to match on a protocol being not yet known. It can not be negated.

The different modes are * direction : protocol recognized on the direction of the current packet * to_server : protocol recognized in the direction to server * to_client : protocol recognized in the direction to client * either : tries to match protocols found on both directions * final : final protocol chosen by Suricata for parsing * original : original protocol (in case of protocol change)

By default, (if no mode is specified), the mode is direction.

Note

when negation is used, like !http, it will not match on the "unknown" state in the flow.

Here is an example of a rule matching non-http traffic on port 80:

```
alert tcp any any -> any 80 (msg:"non-HTTP traffic over HTTP standard port"; flow:to_server; app-layer-protocol:!http,final; sid:1; )
```

Bail out conditions

Protocol detection gives up in several cases:

- both sides are inspected and no match was found
- side A detection failed, side B has no traffic at all (e.g. FTP data channel)
- side A detection failed, side B has so little data detection is inconclusive

In these last 2 cases the `app-layer-event:applayer_proto_detection_skipped` is set.

8.41.2 app-layer-event

Match on events generated by the App Layer Parsers and the protocol detection engine.

Syntax:

```
app-layer-event:<event name>;
```

Examples:

```
app-layer-event:applayer_mismatch_protocol_both_directions;  
app-layer-event:http.gzip_decompression_failed;
```

Protocol Detection

`applayer_mismatch_protocol_both_directions`

The toserver and toclient directions have different protocols. For example a client talking HTTP to a SSH server.

`applayer_wrong_direction_first_data`

Some protocol implementations in Suricata have a requirement with regards to the first data direction. The HTTP parser is an example of this.

<https://redmine.openinfosecfoundation.org/issues/993>

`applayer_detect_protocol_only_one_direction`

Protocol detection only succeeded in one direction. For FTP and SMTP this is expected.

`applayer_proto_detection_skipped`

Protocol detection was skipped because of *Bail out conditions*.

8.41.3 app-layer-state

Match on the detected app-layer protocol transaction state.

Syntax:

```
app-layer-state:[<>]<state>;
```

Examples:

```
app-layer-state:request_headers;
app-layer-state:>request_body;
```

8.42 Generic Decode Layer Keywords

8.42.1 decode-event

Match on events generated by the decode layer. Decode events are generated during the packet decoding phase that indicate structural or invalid values for the Ethernet and layer 2 and layer 3 protocol data.

Syntax:

```
decode-event:<event name>;
```

Examples:

```
decode-event:ipv4.opt_duplicate
decode-event:ethernet.unknown_ether_type
```

Decode Events

ethernet.unknown_ether_type

The ether_type value was not recognized by Suricata. Suricata recognizes the following ether_type values:

```
ETHERNET_TYPE_IP
ETHERNET_TYPE_IPV6
ETHERNET_TYPE_VLAN
ETHERNET_TYPE_8021QINQ
ETHERNET_TYPE_8021AD
ETHERNET_TYPE_8021AH
ETHERNET_TYPE_ARP
ETHERNET_TYPE_MPLS_UNICAST
ETHERNET_TYPE_MPLS_MULTICAST
ETHERNET_TYPE_DCE
ETHERNET_TYPE_VNTAG
ETHERNET_TYPE_NSH
ETHERNET_TYPE_PPOE_SESS
ETHERNET_TYPE_PPOE_DISC
```

8.43 Xbits Keyword

Set, unset, toggle and check for bits stored per host, ip_pair or tx.

Syntax:

```
xbits:<set|unset|isset|isnotset|toggle>,<name>,track <ip_src|ip_dst|ip_pair>;
xbits:<set|unset|isset|toggle>,<name>,track <ip_src|ip_dst|ip_pair> \
    [,expire <seconds>];
xbits:<set|unset|isset|toggle>,<name>,track <ip_src|ip_dst|ip_pair> \
    [,expire <seconds>];
xbits:<set|isset>,<name>,track tx;
```

8.43.1 Transaction support (*tx*)

Bits stored per app-layer transaction. What counts as a transaction protocol is implementation dependent.

8.43.2 Notes

- No difference between using `hostbits` and `xbits` with `track ip_<src|dst>`
- If you set on a client request and use `track ip_dst`, if you want to match on the server response, you check it (`isset`) with `track ip_src`.
- To not alert, use `noalert;`
- the `toggle` option will flip the value of the `xbits`.
- See also:
 - <https://blog.inliniac.net/2014/12/21/crossing-the-streams-in-suricata/>
 - <http://www.cipherdyne.org/blog/2013/07/crossing-the-streams-in-ids-signature-languages.html>

YAML settings

Bits that are stored per host are stored in the Host table. This means that host table settings affect `hostbits` and `xbits` per host.

Bits that are stored per IP pair are stored in the IPPair table. This means that `ippair` table settings, especially `memcap`, affect `xbits` per `ip_pair`.

Threading

Due to subtle timing issues between threads the order of sets and checks can be slightly unpredictable.

Unix Socket

Hostbits can be added, removed and listed through the unix socket.

Add:

```
suricatasc -c "add-hostbit <ip> <bit name> <expire in seconds>"
suricatasc -c "add-hostbit 1.2.3.4 blacklist 3600"
```

If a hostbit is added for an existing hostbit, it's expiry timer is updated.

Remove:

```
suricatasc -c "remove-hostbit <ip> <bit name>"
suricatasc -c "remove-hostbit 1.2.3.4 blacklist"
```

List:

```
suricatasc -c "list-hostbit <ip>"
suricatasc -c "list-hostbit 1.2.3.4"
```

This results in:

```
{
  "message":
  {
    "count": 1,
```

(continues on next page)

(continued from previous page)

```

    "hostbits":
      [{
        "expire": 89,
        "name": "blacklist"
      }]
  },
  "return": "OK"
}

```

Examples

Creating a SSH blacklist

Below is an example of rules incoming to a SSH server.

The first 2 rules match on a SSH software version often used in bots. They drop the traffic and create an 'xbit' 'badssh' for the source ip. It expires in an hour:

```

drop ssh any any -> $MYSERVER 22 (msg:"DROP libssh incoming"; \
  flow:to_server,established; ssh.software; content:"libssh"; \
  xbits:set, badssh, track ip_src, expire 3600; sid:4000000005;)
drop ssh any any -> $MYSERVER 22 (msg:"DROP PUTTY incoming"; \
  flow:to_server,established; ssh.software; content:"PUTTY"; \
  xbits:set, badssh, track ip_src, expire 3600; sid:4000000007;)

```

Then the following rule simply drops any incoming traffic to that server that is on that 'badssh' list:

```

drop ssh any any -> $MYSERVER 22 (msg:"DROP BLACKLISTED"; \
  xbits:isset, badssh, track ip_src; sid:4000000006;)

```

8.44 Alert Keywords

In addition to the action, alerting behavior can be controlled in the rule body using the `noalert` and `alert` keywords. Additionally, alerting behavior is controlled by *Thresholding Keywords*.

8.44.1 noalert

A rule that specifies `noalert` will not generate an alert when it matches, but rule actions will still be performed.

`noalert` is often used in rules that set a `flowbit` for common patterns.

`noalert` is meant for use with rule actions `alert`, `drop`, `reject` that all explicitly or implicitly include `alert`.

```

alert http any any -> any any (http.user_agent; content:"Mozilla/5.0"; startwith; endswith; flowbits:set,mozilla-ua;
noalert; sid:1;)

```

This example sets a flowbit "mozilla-ua" on matching, but does not generate an alert due to the presence of `noalert`.

Note

this option is also used as `flowbits:noalert`; see *Flow Keywords*

8.44.2 alert

A rule that specifies `alert` will generate an alert, even if the rule action doesn't imply alerting.

This keyword can be used to implement an "alert then pass"-logic.

```
pass http any any -> any any (http.user_agent; content:"Mozilla/5.0"; startwith; endswith; alert; sid:1;)
```

This example would pass the rest of the HTTP flow with the Mozilla/5.0 user-agent, generating an alert for the "pass" event.

8.45 Thresholding Keywords

Thresholding can be configured per rule and also globally, see [Global-Thresholds](#).

Thresholds are tracked in a hash table that is sized according to configuration, see: [Thresholding Settings](#).

IMPORTANT for both `threshold` and `detection_filter` keywords

Note

Rules that contain `flowbits`, `flowints`, etc will still have those actions performed when the rule contains one of the `threshold` keywords. Those actions are not subject to the threshold limits.

Rule actions `drop` (IPS mode) and `reject` are applied to each packet (not only the one that meets the limit condition).

8.45.1 threshold

The `threshold` keyword can be used to control the rule's alert frequency. There are four threshold modes:

1. `threshold`
2. `limit`
3. `both`
4. `backoff`

Syntax:

```
threshold: type <threshold|limit|both|backoff>, track <by_src|by_dst|by_rule|by_both|by_flow>, count <N>, <seconds <T>|multiplier <M>>
```

Specify `seconds` to control the number of alerts per time period.

type "threshold"

This type sets a minimum threshold for a rule before it generates alerts.

A threshold setting with a `count` value of `C` will generate an alert the `C`th time the alert matches. If `seconds` is specified, an alert is generated when `count` matches have occurred within `N` seconds.

Syntax:

```
threshold: type threshold, track by_flow, count <C>, seconds <N>;
```

Example:

```
alert tcp !$HOME_NET any -> $HOME_NET 25 (msg:"ET POLICY Inbound Frequent Emails - Possible Spambot Inbound"; flow:established; content:"mail from|3a|"; nocase; threshold: type threshold, track by_src, count 10, seconds 60; reference:url,doc.emergingthreats.net/2002087; classtype:misc-activity; sid:2002087; rev:10;)
```

This signature generates an alert if there are 10 or more inbound emails from the same server within one minute.

type "limit"

The `limit` type prevents a flood of alerts by limiting the number of alerts. A limit with a count of `N` won't generate more than `N` alerts.

Limit the number of alerts per time period by specifying `seconds` with `count`.

Syntax:

```
threshold: type limit, track by_dst, count <C>, seconds <N>;
```

Example:

```
alert http $HOME_NET any -> any any (msg:"ET INFO Internet Explorer 6 in use - Significant Security Risk"; flow:established,to_server; http.user_agent; content:"Mozilla/4.0 (compatible|3b| MSIE 6.0|3b|"; threshold: type limit, track by_src, seconds 180, count 1; classtype:policy-violation; sid:2010706; rev:10; metadata:created_at 2010_07_30, updated_at 2024_03_16;)
```

In this example, at most 1 alert is generated per host within a period of 3 minutes if "MSIE 6.0" is detected.

type "both"

This type combines `threshold` and `limit` to control when alerts are generated.

Syntax:

```
threshold: type both, track by_flow, count <C>, multiplier <M>;
```

Example:

```
alert tcp $HOME_NET 5060 -> $EXTERNAL_NET any (msg:"ET VOIP Multiple Unauthorized SIP Responses TCP"; flow:established,from_server; content:"SIP/2.0 401 Unauthorized"; depth:24; threshold: type both, track by_src, count 5, seconds 360; reference:url,doc.emergingthreats.net/2003194; classtype:attempted-dos; sid:2003194; rev:6;)
```

This rule will generate at most one alert every 6 minutes if there have been 5 or more occurrences of "SIP2.0 401 Unauthorized" responses.

The `type backoff` section describes the `multiplier` keyword.

type "backoff"

This type limits the alert output by using a backoff algorithm between alerts.

Note

`backoff` can only be used with `track by_flow`

Syntax:

```
threshold: type backoff, track by_flow, count <C>, multiplier <M>;
```

track: backoff is only supported for **by_flow** **count:** number of alerts before the first match generates an alert.
multiplier: value to multiply count with each time the next value is reached

A count of 1 with a multiplier of 10 would generate alerts for matching packets:

1, 10, 100, 1000, 10000, 100000, etc.

A count of 1 with a multiplier of 2 would generate alerts for matching packets:

1, 2, 4, 8, 16, 32, 64, etc.

A count of 5 with multiplier 5 would generate alerts for matching packets:

5, 25, 125, 625, 3125, 15625, etc

In the following example, the `pkt_invalid_ack` would only lead to alerts the 1st, 10th, 100th, etc.

alert tcp any any -> any any (stream-event:pkt_invalid_ack; threshold:type backoff, track by_flow, count 1, multiplier 10; sid:2210045; rev:2;)

track

Option	Tracks By
by_src	source IP
by_dst	destination IP
by_both	pair of src IP and dst IP
by_rule	signature id
by_flow	flow

8.45.2 detection_filter

The `detection_filter` keyword can be used to alert on every match after an initial threshold has been reached. It differs from `threshold` with type `threshold` in that it generates an alert for each rule match after the initial threshold has been reached, where the latter will reset its internal counter and alert each time the threshold has been reached.

Syntax:

`detection_filter: track <by_src|by_dst|by_rule|by_both|by_flow>, count <N>, seconds <T>`

Example:

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"ET WEB_SERVER WebResource.axd access
without t (time) parameter - possible ASP padding-oracle exploit"; flow:established,to_server; content:"GET";
http_method; content:"WebResource.axd"; http_uri; nocase; content:"!&t="; http_uri; nocase; content:"!&|t=";
http_uri; nocase; detection_filter:track by_src,count 15,seconds 2; reference:url,netifera.com/research/; ref-
erence:url,www.microsoft.com/technet/security/advisory/2416728.mspx; classtype:web-application-attack;
sid:2011807; rev:5;)
```

This rule will generate alerts after 15 or more matches have occurred within 2 seconds.

8.46 IP Reputation Keyword

IP Reputation can be used in rules through a new rule keyword "iprep".

For more information about IP Reputation see *IP Reputation Config* and *IP Reputation Format*.

8.46.1 iprep

The iprep directive matches on the IP reputation information for a host.

```
iprep:<side to check>,<category>,<operator>,<reputation score>
```

side to check: <any|src|dst|both>

category: the category short name

operator: <, <=, >, >=, =

reputation score: 0-127

Example:

```
alert ip $HOME_NET any -> any any (msg:"IPREP internal host talking to CnC server";  
  ⤵flow:to_server; iprep:dst,CnC,>,30; sid:1; rev:1;)
```

This rule will alert when a system in \$HOME_NET acts as a client while communicating with any IP in the CnC category that has a reputation score set to greater than 30.

isset and isnotset

isset and isnotset can be used to test reputation "membership"

```
iprep:<side to check>,<category>,<isset|isnotset>
```

side to check: <any|src|dst|both>

category: the category short name

To test whether an IP is part of an iprep set at all, the isset can be used. It acts as a >=, 0 statement.

```
drop ip $HOME_NET any -> any any (iprep:src,known-bad-hosts,isset; sid:1;)
```

In this example traffic to any IP with a score in known-bad-hosts would be blocked.

isnotset can be used to test if an IP is not a part of the set.

```
drop ip $HOME_NET any -> any any (iprep:src,trusted-hosts,isnotset; sid:1;)
```

In this example traffic for a host w/o a trust score would be blocked.

Compatibility with IP-only

The "iprep" keyword is compatible with "IP-only" rules. This means that a rule like:

```
alert ip any any -> any any (msg:"IPREP High Value CnC"; iprep:src,CnC,>,100; sid:1;  
  ⤵rev:1;)
```

will only be checked once per flow-direction.

8.47 IP Addresses Match

Matching on IP addresses can be done via the IP tuple parameters or via the iprep keywords (see *IP Reputation Keyword*). Some keywords providing interaction with datasets are also available.

8.47.1 ip.src

The *ip.src* keyword is a sticky buffer to match on source IP address. It matches on the binary representation and is compatible with datasets of types *ip* and *ipv4*.

Example:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Inbound bad list"; flow:to_server; ip.  
↪src; dataset:isset,badips,type ip,load badips.list; sid:1; rev:1;)
```

8.47.2 ip.dst

The *ip.dst* keyword is a sticky buffer to match on destination IP address. It matches on the binary representation and is compatible with the dataset of type *ip* and *ipv4*.

Example:

```
alert tcp $HOME_NET any -> any any (msg:"Outbound bad list"; flow:to_server; ip.dst;↪  
↪dataset:isset,badips,type ip,load badips.list; sid:1; rev:1;)
```

8.48 Config Rules

Config rules are rules that when matching, will change the configuration of Suricata for a flow, transaction, packet or other unit.

Example:

```
config dns any any -> any any (dns.query; content:"suricata"; config: logging disable,↪  
↪type tx, scope tx; sid:1;)
```

This example will detect if a DNS query contains the string *suricata* and if so disable the DNS transaction logging. This means that *eve.json* records, but also Lua output, will not be generated/triggered for this DNS transaction.

Example:

```
config tcp:pre_flow any any <> any 666 (config: tracking disable, type flow, scope↪  
↪packet; sid:1;)
```

This example skips flow tracking for any packet from or to tcp port 666.

8.48.1 Keyword

The *config* rule keyword provides the setting and the scope of the change.

Syntax:

```
config:<subsys> <action>, type <type>, scope <scope>;
```

subsys can be set to:

- *logging* setting affects logging.
- *tracking* setting affects tracking.

type can be set to:

- *tx* sub type of the *subsys*. If *subsys* is set to *logging*, setting the *type* to *tx* means transaction logging is affected.
- *flow* sub type of the *subsys*. If *subsys* is set to *flow*, setting the *type* to *flow* means flow tracking is disabled.

scope can be set to:

- *tx* setting affects the matching transaction.
- *packet* setting affects the matching packet.

The *action* in `<subsys>` is currently limited to *disable*.

8.48.2 Action

Config rules can, but don't have to, use the *config* rule action. The *config* rule action won't generate an alert when the rule matches, but the rule actions will still be applied. It is equivalent to *alert ... (noalert; ...)*.

8.49 Datasets

Using the *dataset* and *datarep* keyword it is possible to match on large amounts of data against any sticky buffer.

For example, to match against a DNS black list called `dns-bl`:

```
dns.query; dataset:isset,dns-bl;
```

These keywords are aware of transforms. So to look up a DNS query against a MD5 black list:

```
dns.query; to_md5; dataset:isset,dns-bl;
```

8.49.1 Global config (optional)

Datasets can optionally be defined in the main config. Sets can also be declared from the rule syntax.

Example of sets for tracking unique values:

```
datasets:
  ua-seen:
    type: string
    state: ua-seen.lst
  dns-sha256-seen:
    type: sha256
    state: dns-sha256-seen.lst
```

Rules to go with the above:

```
alert dns any any -> any any (msg:"dns list test"; dns.query; to_sha256; dataset:isset,dns-sha256-seen; sid:123; rev:1;)
```

```
alert http any any -> any any (msg: "http user-agent test"; http.user_agent; dataset:set,ua-seen; sid:234; rev:1;)
```

It is also possible to optionally define global default memcap and hashsize.

Example:

```
datasets:
  defaults:
    memcap: 100mb
    hashsize: 2048
  ua-seen:
    type: string
    load: ua-seen.lst
```

or define memcap and hashsize per dataset.

Example:

```
datasets:
  ua-seen:
    type: string
    load: ua-seen.lst
    memcap: 10mb
    hashsize: 1024
```

Note

The *hashsize* should be close to the amount of entries in the dataset to avoid collisions. If it's set too low, this could result in rather long startup time.

8.49.2 Rule keywords

dataset

Datasets are binary: something is in the set or it's not.

Syntax:

```
dataset:<cmd>,<name>,<options>;

dataset:<set|unset|isset|isnotset>,<name> \
    [, type <string|md5|sha256|ipv4|ip>, save <file name>, load <file name>, state <file_
↪name>, memcap <size>, hashsize <size>
    , format <csv|json|ndjson>, context_key <output_key>, value_key <json_key>, array_
↪key <json_path>,
    remove_key];
```

type <type>

the data type: string, md5, sha256, ipv4, ip

load <file name>

file name for load the data when Suricata starts up

state

sets file name for loading and saving a dataset

save <file name>

advanced option to set the file name for saving the in-memory data when Suricata exits.

memcap <size>

maximum memory limit for the respective dataset

hashsize <size>

allowed size of the hash for the respective dataset

format <type>

the format of the file: csv, json. Default to csv. See *dataset with json format* for json and ndjson option

context_key <key>

the key to use for the enrichment of the alert event for json format

value_key <key>

the key to use for the value of the alert for json format

array_key <key>

the key to use for the array of the alert for json format

remove_key

if set, the JSON object pointed by value key will be removed from the alert event

Note

'type' is mandatory and needs to be set.

Note

'load' and 'state' or 'save' and 'state' cannot be mixed.

Example rules could look like:

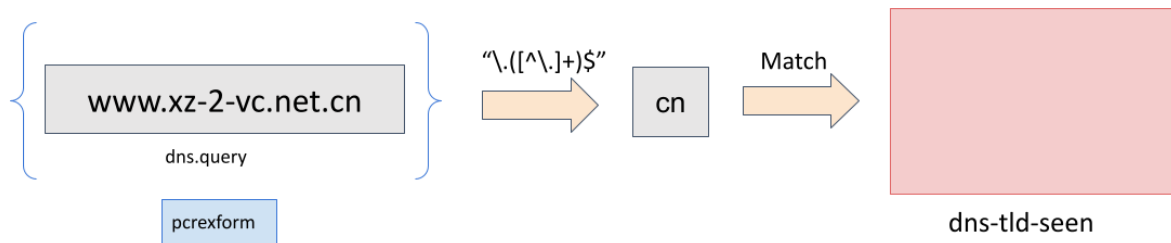
1. Detect unique User-Agents:

```
alert http any any -> any any (msg:"LOCAL HTTP new UA"; http.user_agent; dataset:set,http-ua-seen, type string,
state http-ua-seen.csv; sid:8000001; rev:1;)
```

2. Detect unique TLDs:

```
alert dns $HOME_NET any -> any any (msg:"LOCAL DNS unique TLD"; dns.query; pcrcxform:"\.(^[^\.]+)$";
dataset:set,dns-tld-seen, type string, state dns-tld-seen.csv; sid:8000002; rev:1;)
```

Following image is a pictorial representation of how the `pcrcxform` works on domain names to find TLDs in the dataset `dns-tld-seen`:



Notice how it is not possible to do certain operations alone with datasets (example 2 above), but, it is possible to use a combination of other rule keywords. Keep in mind the cost of additional keywords though e.g. in the second example rule above, negative performance impact can be expected due to `pcrcxform`.

datarep

Data Reputation allows matching data against a reputation list.

Syntax:

```
datarep:<name>,<operator>,<value>,\
[, load <file name>, type <string|md5|sha256|ipv4|ip>, memcap <size>, hashsize <size>
->];
```

Example rules could look like:

```

alert dns any any -> any any (dns.query; to_md5; datarep:dns_md5, >, 200, load dns_md5.
↳rep, type md5, memcap 100mb, hashsize 2048; sid:1;)
alert dns any any -> any any (dns.query; to_sha256; datarep:dns_sha256, >, 200, load dns_
↳sha256.rep, type sha256; sid:2;)
alert dns any any -> any any (dns.query; datarep:dns_string, >, 200, load dns_string.rep,
↳ type string; sid:3;)

```

In these examples the DNS query string is checked against three different reputation lists. A MD5 list, a SHA256 list, and a raw string (buffer) list. The rules will only match if the data is in the list and the reputation value is higher than 200.

dataset with JSON

Dataset with JSON allows matching data against a set and output data attached to the matching value in the event.

There are two formats supported: `json` and `ndjson`. The difference is that `json` format is a single JSON object, while `ndjson` is handling file with one JSON object per line. The `ndjson` format is useful for large files as the parsing is done line by line.

Syntax:

```

dataset:<cmd>,<name>,<options>;

dataset:<isset|isnotset>,<name> \
    [, type <string|md5|sha256|ipv4|ip>, load <file name>, format <json|ndjson>, memcap
↳<size>, hashsize <size>, context_key <json_key> \
    , value_key <json_key>, array_key <json_path>];

```

Example rules could look like:

```

alert http any any -> any any (msg:"IP match"; ip.dst; dataset:isset,bad_ips, type ip,
↳load bad_ips.json, format json, context_key bad_ones, value_key ip; sid:80000001;)

```

In this example, the match will occur if the destination IP is in the set and the alert will have an `alert.content.bad_ones` subobject that will contain the JSON data associated to the value (`bad_ones` coming from `context_key` option).

When format is `json` or `ndjson`, the `value_key` is used to get the value in the line (`ndjson` format) or in the array (`json` format). At least one single element needs to have the `value_key` present in the data file to have a successful load. If `array_key` is present, Suricata will extract the corresponding subobject that has to be a JSON array and search for element to add to the set in this array. This is only valid for `json` format.

If you don't want to have the `value_key` in the alert, you can use the `remove_key` option. This will remove the key from the alert event.

See [Dataset with JSON format](#) for more information.

8.49.3 Rule Reloads

Sets that are defined in the `yaml`, or sets that only use `state` or `save`, are considered *dynamic* sets. These are not reloaded during rule reloads.

Sets that are defined in rules using only `load` are considered *static* tests. These are not expected to change during runtime. During rule reloads these are reloaded from disk. This reload is effective when the complete rule reload process is complete.

8.49.4 Unix Socket

dataset-add

Unix Socket command to add data to a set. On success, the addition becomes active instantly.

Syntax:

```
dataset-add <set name> <set type> <data>
```

set name

Name of an already defined dataset

type

Data type: string, md5, sha256, ipv4, ip

data

Data to add in serialized form (base64 for string, hex notation for md5/sha256, string representation for ipv4/ip)

Example adding 'google.com' to set 'myset':

```
dataset-add myset string Z29vZ2x1LmNvbQ==
```

dataset-remove

Unix Socket command to remove data from a set. On success, the removal becomes active instantly.

Syntax:

```
dataset-remove <set name> <set type> <data>
```

set name

Name of an already defined dataset

type

Data type: string, md5, sha256, ipv4, ip

data

Data to remove in serialized form (base64 for string, hex notation for md5/sha256, string representation for ipv4/ip)

dataset-clear

Unix Socket command to remove all data from a set. On success, the removal becomes active instantly.

Syntax:

```
dataset-clear <set name> <set type>
```

set name

Name of an already defined dataset

type

Data type: string, md5, sha256, ipv4, ip

dataset-lookup

Unix Socket command to test if data is in a set.

Syntax:

```
dataset-lookup <set name> <set type> <data>
```

set name

Name of an already defined dataset

type

Data type: string, md5, sha256, ipv4, ip

data

Data to test in serialized form (base64 for string, hex notation for md5/sha256, string notation for ipv4/ip)

Example testing if 'google.com' is in the set 'myset':

```
dataset-lookup myset string Z29vZ2x1LmNvbQ==
```

dataset-dump

Unix socket command to trigger a dump of datasets to disk.

Syntax:

```
dataset-dump
```

dataset-add-json

Unix Socket command to add data to a set. On success, the addition becomes active instantly.

Syntax:

```
dataset-add-json <set name> <set type> <data> <json_info>
```

set name

Name of an already defined dataset

type

Data type: string, md5, sha256, ipv4, ip

data

Data to add in serialized form (base64 for string, hex notation for md5/sha256, string representation for ipv4/ip)

Example adding 'google.com' to set 'myset':

```
dataset-add-json myset string Z29vZ2x1LmNvbQ== {"city":"Mountain View"}
```

8.49.5 File formats

Datasets use a simple CSV format where data is per line in the file.

data types**string**

in the file as base64 encoded string

md5

in the file as hex encoded string

sha256

in the file as hex encoded string

ipv4

in the file as string

ip

in the file as string, it can be IPv6 or IPv4 address (standard notation or IPv4 in IPv6 one)

dataset

Datasets have a simple structure, where there is one piece of data per line in the file.

Syntax:

```
<data>
```

e.g. for ua-seen with type string:

```
TW96aWxsYS80LjAgKGNvbXBhdGlibGU7ICk=
```

which when piped to `base64 -d` reveals its value:

```
Mozilla/4.0 (compatible; )
```

datarep

The datarep format follows the dataset, except that there are 1 more CSV field:

Syntax:

```
<data>,<value>
```

dataset with JSON enrichment

If `format json` is used in the parameters of a dataset keyword, then the loaded file has to contain a valid JSON object.

If `value_key`` option is present then the file has to contain a valid JSON object containing an array where the key equal to `value_key` value is present.

For example, if the file `file.json` is like the following example (typical of return of REST API call)

```
{
  "time": "2024-12-21",
  "response": {
    "threats":
      [
        {"host": "toto.com", "origin": "japan"},
        {"host": "grenouille.com", "origin": "french"}
      ]
  }
}
```

then the match to check the list of threats using dataset with JSON can be defined as

```
http.host; dataset:isset,threats,load file.json, context_key threat, value_key host,␣
↪array_key response.threats;
```

If the signature matches, it will result in an alert with the following

```
{
  "alert": {
    "context": {
      "threat": {
        "host": "toto.com",
        "origin": "japan"
      }
    }
  }
}
```

8.49.6 File Locations

Dataset filenames configured in the `suricata.yaml` can exist anywhere on your filesystem.

When a dataset filename is specified in rule, the following *rules* are applied:

- For `load`, the filename is opened relative to the rule file containing the rule. Absolute filenames and parent directory traversals are allowed.
- For `save` and `state` the filename is relative to `$LOCALSTATEDIR/suricata/data`. On many installs this will be `/var/lib/suricata/data`, but run `suricata --build-info` and check the value of `--localstatedir` to verify this location on your installation.
 - Absolute filenames, or filenames containing parent directory traversal (`..`) are not allowed unless the configuration parameter `datasets.allow-absolute-filenames` is set to `true`.

8.49.7 Security

As datasets potentially allow a rule distributor write access to your system with `save` and `state` dataset rules, the locations allowed are strict by default, however there are two dataset options to tune the security of rules utilizing dataset filenames:

```
datasets:
  rules:
    # Set to true to allow absolute filenames and filenames that use
    # ".." components to reference parent directories in rules that specify
    # their filenames.
    allow-absolute-filenames: false

    # Allow datasets in rules write access for "save" and
    # "state". This is enabled by default, however write access is
    # limited to the data directory.
    allow-write: true
```

By setting `datasets.rules.allow-write` to `false`, all `save` and `state` rules will fail to load. This option is enabled by default to preserve compatibility with previous 6.0 Suricata releases, however may change in a future major release.

Pre-Suricata 6.0.13 behavior can be restored by setting `datasets.rules.allow-absolute-filenames` to `true`, however allowing so will allow any rule to overwrite any file on your system that Suricata has write access to.

8.50 Lua Scripting for Detection

There are 2 ways that Lua can be used with detection. These are

- lua rule keyword.
- luaxform transform.

Note

As of Suricata 8.0, Lua rules are enabled by default and run in a sandboxed environment. See [Lua Sandbox and Available functions](#).

8.50.1 Lua Rule Keyword

Syntax:

```
lua:[!]<scriptfilename>;
```

The script filename will be appended to your default rules location.

A Lua rule script has 2 required functions, an `init` function and `match` function, discussed below.

Additionally, the script will run in a limited sandbox by default.

Init function

```
function init (args)
    return {}
end
```

Most Lua rule scripts can simply return an empty table in their `init` method. To hook into specific protocols states, *Explicit rule hook (states)* may be used. However, some buffers do require explicit initialization:

```
* ja3
* ja3s
* packet
* payload
* stream
```

To request these buffers, use an `init` method like:

```
function init (args)
    return {packet = true}
end
```

Match function

```
local http = require("suricata.http")

function match(args)
    local tx = http.get_tx()
    a = tx:request_line()
    if #a > 0 then
```

(continues on next page)

(continued from previous page)

```

    if a:find("^POST%s+/.%*.php%s+HTTP/1.0$") then
        return 1
    end
end

return 0
end

```

The script can return 1 or 0. It should return 1 if the condition(s) it checks for match, 0 if not.

8.50.2 Lua Transform: `luaxform`

More details in *luaxform*.

8.50.3 Lua Sandbox and Available functions

Lua rule scripts are run in a sandbox environment the applies the following restrictions:

- reduced libraries
- only allowed functions available
- instruction count limit
- memory allocation limit

The following table lists the library and functions available:

Pack- age Name	Functions
base	assert, ipairs, next, pairs, print, rawequal, rawlen, select, tonumber, tostring, type, warn, rawget, rawset, error
table	concat, insert, move, pack, remove, sort, unpack
string	byte, char, dump, find, format, gmatch, gsub, len, lower, match, pack, packsize, rep, reverse, sub, unpack, upper
math	abs, acos, asin, atan, atan2, ceil, cos, cosh, deg, exp, floor, fmod, frexp, ldexp, log, log10, max, min, modf, pow, rad, random, randomseed, sin, sinh, sqrt, tan, tanh, tointeger, type, ult
utf8	offset, len, codes, char, codepoint

Of note, the following standard libraries are not available:

- coroutine
- package
- input and output
- operating system facilities
- debug

This behavior can be modified via the `security.lua` section of *Lua*

Note

Suricata 8.0 has moved to Lua 5.4 and now has builtin support for bitwise and utf8 operations.

A comprehensive list of existing lua functions - with examples - can be found at [Lua functions](#) (some of them, however, work only for the lua-output functionality).

8.51 Differences From Snort

This document is intended to highlight the major differences between Suricata and Snort that apply to rules and rule writing.

Where not specified, the statements below apply to Suricata. In general, references to Snort refer to the version 2.9 branch.

8.51.1 Automatic Protocol Detection

- Suricata does automatic protocol detection of the following application layer protocols:
 - dcerpc
 - dnp3
 - dns
 - http
 - imap (detection only by default; no parsing)
 - pop3 (detection only by default; no parsing)
 - ftp
 - modbus (disabled by default; minimalist probe parser; can lead to false positives)
 - smb
 - smb2 (disabled internally inside the engine)
 - smtp
 - ssh
 - tls (SSLv2, SSLv3, TLSv1, TLSv1.1 and TLSv1.2)
- In Suricata, protocol detection is port agnostic (in most cases). In Snort, in order for the `http_inspect` and other preprocessors to be applied to traffic, it has to be over a configured port.
 - Some configurations for app-layer in the Suricata yaml can/do by default specify specific destination ports (e.g. DNS)
 - **You can look on 'any' port without worrying about the performance impact that you would have to be concerned about with Snort.**
- If the traffic is detected as HTTP by Suricata, the `http_*` buffers are populated and can be used, regardless of port(s) specified in the rule.
- You don't have to check for the http protocol (i.e. `alert http ...`) to use the `http_*` buffers although it is recommended.

- If you are trying to detect legitimate (supported) application layer protocol traffic and don't want to look on specific port(s), the rule should be written as `alert <protocol> ...` with `any` in place of the usual protocol port(s). For example, when you want to detect HTTP traffic and don't want to limit detection to a particular port or list of ports, the rules should be written as `alert http ...` with `any` in place of `$HTTP_PORTS`.

– You can also use `app-layer-protocol:<protocol>;` inside the rule instead.

So, instead of this Snort rule:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS ...
```

Do this for Suricata:

```
alert http $HOME_NET -> $EXTERNAL_NET any ...
```

Or:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET any (app-layer-protocol:http; ...
```

8.51.2 urilen Keyword

- Ranges given in the `urilen` keyword are inclusive for Snort but not inclusive for Suricata.

Example: `urilen:2<>10`

- Snort interprets this as, "the URI length must be **greater than or equal to 2**, and **less than or equal to 10**".
- Suricata interprets this as "the URI length must be **greater than 2** and **less than 10**".
- There is a request to have Suricata behave like Snort in future versions – <https://redmine.openinfosecfoundation.org/issues/1416>
- * Currently on hold

- By default, with *Suricata*, `urilen` applies to the **normalized** buffer
 - Use `,raw` for raw buffer
 - e.g. `urilen:>20,raw;`
- By default, with *Snort*, `urilen` applies to the **raw** buffer
 - Use `,norm` for normalized buffer
 - e.g. `urilen:>20,norm;`

8.51.3 http_uri Buffer

- In Snort, the `http_uri` buffer normalizes '+' characters (0x2B) to spaces (0x20).
 - Suricata can do this as well but you have to explicitly set `query-plusspace-decode: yes` in the `libhttp` section of Suricata's `yaml` file.
- <https://redmine.openinfosecfoundation.org/issues/1035>
- <https://github.com/inliniac/suricata/pull/620>

8.51.4 http_header Buffer

- In Snort, the `http_header` buffer includes the CRLF CRLF (0x0D 0x0A 0x0D 0x0A) that separates the end of the last HTTP header from the beginning of the HTTP body. Suricata includes a CRLF after the last header in the `http_header` buffer but not an extra one like Snort does. If you want to match the end of the buffer, use

either the `http_raw_header` buffer, a relative `isdataat` (e.g. `isdataat:!1,relative`) or a PCRE (although PCRE will be worse on performance).

- Suricata *will* include CRLF CRLF at the end of the `http_raw_header` buffer like Snort does.
- Snort will include a *leading* CRLF in the `http_header` buffer of *server responses* (but not client requests). Suricata does not have the leading CRLF in the `http_header` buffer of the server response or client request.
- In the `http_header` buffer, Suricata will normalize HTTP header lines such that there is a single space (0x20) after the colon (':') that separates the header name from the header value; this single space replaces zero or more whitespace characters (including tabs) that may be present in the raw HTTP header line immediately after the colon. If the extra whitespace (or lack thereof) is important for matching, use the `http_raw_header` buffer instead of the `http_header` buffer.
- Snort will also normalize superfluous whitespace between the header name and header value like Suricata does but only if there is at least one space character (0x20 only so not 0x90) immediately after the colon. This means that, unlike Suricata, if there is no space (or if there is a tab) immediately after the colon before the header value, the content of the header line will remain unchanged in the `http_header` buffer.
- When there are duplicate HTTP headers (referring to the header name only, not the value), the normalized buffer (`http_header`) will concatenate the values in the order seen (from top to bottom), with a comma and space (", ") between each of them. If this hinders detection, use the `http_raw_header` buffer instead.

Example request:

```
GET /test.html HTTP/1.1
Content-Length: 44
Accept: */*
Content-Length: 55
```

The Content-Length header line becomes this in the `http_header` buffer:

```
Content-Length: 44, 55
```

- The HTTP 'Cookie' and 'Set-Cookie' headers are **NOT** included in the `http_header` buffer; instead they are extracted and put into their own buffer – `http_cookie`. See the [http_cookie Buffer](#) section.
- The HTTP 'Cookie' and 'Set-Cookie' headers **ARE** included in the `http_raw_header` buffer so if you are trying to match on something like particular header ordering involving (or not involving) the HTTP Cookie headers, use the `http_raw_header` buffer.
- If 'enable_cookie' is set for Snort, the HTTP Cookie header names and trailing CRLF (i.e. "Cookie: \r\n" and "Set-Cooke \r\n") are kept in the `http_header` buffer. This is not the case for Suricata which removes the entire "Cookie" or "Set-Cookie" line from the `http_header` buffer.
- Other HTTP headers that have their own buffer (`http_user_agent`, `http_host`) are not removed from the `http_header` buffer like the Cookie headers are.
- When inspecting server responses and `file_data` is used, content matches in `http_*` buffers should come before `file_data` unless you use `pkt_data` to reset the cursor before matching in `http_*` buffers. Snort will not complain if you use `http_*` buffers after `file_data` is set.

8.51.5 http_cookie Buffer

- The `http_cookie` buffer will NOT include the header name, colon, or leading whitespace. i.e. it will not include "Cookie: " or "Set-Cookie: ".
- The `http_cookie` buffer does not include a CRLF (0x0D 0x0A) at the end. If you want to match the end of the buffer, use a relative `isdataat` or a PCRE (although PCRE will be worse on performance).

- There is no `http_raw_cookie` buffer in Suricata. Use `http_raw_header` instead.
- You do not have to configure anything special to use the 'http_cookie' buffer in Suricata. This is different from Snort where you have to set `enable_cookie` in the `http_inspect_server` preprocessor config in order to have the `http_cookie` buffer treated separate from the `http_header` buffer.
- If Snort has 'enable_cookie' set and multiple "Cookie" or "Set-Cookie" headers are seen, it will concatenate them together (with no separator between them) in the order seen from top to bottom.
- If a request contains multiple "Cookie" or "Set-Cookie" headers, the values will be concatenated in the Suricata `http_cookie` buffer, in the order seen from top to bottom, with a comma and space (", ") between each of them.

Example request:

```
GET /test.html HTTP/1.1
Cookie: monster
Accept: */*
Cookie: elmo
```

Suricata `http_cookie` buffer contents:

```
monster, elmo
```

Snort `http_cookie` buffer contents:

```
monsterelmo
```

- Corresponding PCRE modifier: C (same as Snort)

8.51.6 New HTTP keywords

Suricata supports several HTTP keywords that Snort doesn't have.

Examples are `http_user_agent`, `http_host` and `http_content_type`.

See [HTTP Keywords](#) for all HTTP keywords.

8.51.7 byte_extract Keyword

- Suricata supports `byte_extract` from `http_*` buffers, including `http_header` which does not always work as expected in Snort.
- In Suricata, variables extracted using `byte_extract` must be used in the same buffer, otherwise they will have the value "0" (zero). Snort does allow cross-buffer byte extraction and usage.
- Be sure to always positively and negatively test Suricata rules that use `byte_extract` and `byte_test` to verify that they work as expected.

8.51.8 byte_jump Keyword

- Suricata allows a variable name from `byte_extract` or `byte_math` to be specified for the `nbytes` value. The value of `nbytes` must adhere to the same constraints as if it were supplied directly in the rule.

8.51.9 byte_math Keyword

- Suricata accepts `dce` as an endian value or as a separate keyword. `endian dce` or `dce` are equivalent.
- Suricata's rule parser rejects rules that repeat keywords in a single rule. E.g., `byte_math: endian big, endian little`.

- Suricata's rule parser accepts `rvalue` values of 0 to the maximum uint32 value. Snort rejects `rvalue` values of 0 and requires values to be between `[1..max-uint32 value]`.
- Suricata will never match if there's a zero divisor. Division by 0 is undefined.

8.51.10 `byte_test` Keyword

- Suricata allows a variable name from `byte_extract` or `byte_math` to be specified for the `nbytes` value. The value of `nbytes` must adhere to the same constraints as though a value was directly supplied by the rule.
- Suricata allows a variable name from `byte_extract` to be specified for the `nbytes` value. The value of `nbytes` must adhere to the same constraints as if it were supplied directly in the rule.

8.51.11 `isdataat` Keyword

- The `rawbytes` keyword is supported in the Suricata syntax but doesn't actually do anything.
- Absolute `isdataat` checks will succeed if the offset used is **less than** the size of the inspection buffer. This is true for Suricata and Snort.
- For *relative* `isdataat` checks, there is a **1 byte difference** in the way Snort and Suricata do the comparisons.
 - Suricata will succeed if the relative offset is **less than or equal to** the size of the inspection buffer. This is different from absolute `isdataat` checks.
 - Snort will succeed if the relative offset is **less than** the size of the inspection buffer, just like absolute `isdataat` checks.
 - Example - to check that there is no data in the inspection buffer after the last content match:
 - * Snort: `isdataat:!0,relative;`
 - * Suricata: `isdataat:!1,relative;`
- With Snort, the "inspection buffer" used when checking an `isdataat` keyword is generally the packet/segment with some exceptions:
 - With PAF enabled the PDU is examined instead of the packet/segment. When `file_data` or `base64_data` has been set, it is those buffers (unless `rawbytes` is set).
 - With some preprocessors - `modbus`, `gtp`, `sip`, `dce2`, and `dnp3` - the buffer can be particular portions of those protocols (unless `rawbytes` is set).
 - With some preprocessors - `rpc_decode`, `ftp_telnet`, `smtp`, and `dnp3` - the buffer can be particular *decoded* portions of those protocols (unless `rawbytes` is set).
- With Suricata, the "inspection buffer" used when checking an absolute `isdataat` keyword is the packet/segment if looking at a packet (e.g. `alert tcp-pkt...`) or the reassembled stream segments.
- In Suricata, a *relative* `isdataat` keyword **will apply to the buffer of the previous content match**. So if the previous content match is a `http_*` buffer, the relative `isdataat` applies to that buffer, starting from the end of the previous content match in that buffer. *Snort does not behave like this!*
- For example, this Suricata rule looks for the string ".exe" at the end of the URI; to do the same thing in the normalized URI buffer in Snort you would have to use a PCRE – `pcre:"/\x2Eexe$/U"`;

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:".EXE File Download Request";
↪flow:established,to_server; content:"GET"; http_method; content:".exe"; http_uri;
↪isdataat:!1,relative; priority:3; sid:18332111;)
```

- If you are unclear about behavior in a particular instance, you are encouraged to positively and negatively test your rules that use an `isdataat` keyword.

8.51.12 Relative PCRE

- You can do relative PCRE matches in normalized/special buffers with Suricata. Example:

```
content:".php?sign="; http_uri; pcre:"/^[a-zA-Z0-9]{8}$/UR";
```

- With Snort you can't combine the "relative" PCRE option ('R') with other buffer options like normalized URI ('U') – you get a syntax error.

8.51.13 tls* Keywords

In addition to TLS protocol identification, Suricata supports the storing of certificates to disk, verifying the validity dates on certificates, matching against the calculated SHA1 fingerprint of certificates, and matching on certain TLS/SSL certificate fields including the following:

- Negotiated TLS/SSL version.
- Certificate Subject field.
- Certificate Issuer field.
- Certificate SNI Field

For details see *SSL/TLS Keywords*.

8.51.14 dns_query Keyword

- Sets the detection pointer to the DNS query.
- Works like `file_data` does ("sticky buffer") but for a DNS request query.
- Use `pkt_data` to reset the detection pointer to the beginning of the packet payload.
- See *DNS Keywords* for details.

8.51.15 IP Reputation and iprep Keyword

- Snort has the "reputation" preprocessor that can be used to define whitelist and blacklist files of IPs which are used generate GID 136 alerts as well as block/drop/pass traffic from listed IPs depending on how it is configured.
- Suricata also has the concept of files with IPs in them but provides the ability to assign them:
 - Categories
 - Reputation score
- Suricata rules can leverage these IP lists with the `iprep` keyword that can be configured to match on:
 - Direction
 - Category
 - Value (reputation score)
- *Reputation*
- *IP Reputation Config*
- *IP Reputation Keyword*
- *IP Reputation Format*
- <https://blog.inliniac.net/2012/11/21/ip-reputation-in-suricata/>

8.51.16 Flowbits

- Suricata fully supports the setting and checking of flowbits (including the same flowbit) on the same packet/stream. Snort does not always allow for this.
- In Suricata, `flowbits:isset` is checked after the fast pattern match but before other content matches. In Snort, `flowbits:isset` is checked in the order it appears in the rule, from left to right.
- If there is a chain of flowbits where multiple rules set flowbits and they are dependent on each other, then the order of the rules or the sid values can make a difference in the rules being evaluated in the proper order and generating alerts as expected. See bug 1399 - <https://redmine.openinfosecfoundation.org/issues/1399>.
- *Flow Keywords*

8.51.17 flowbits:noalert;

A common pattern in existing rules is to use `flowbits:noalert;` to make sure a rule doesn't generate an alert if it matches.

Suricata allows using just `noalert;` as well. Both have an identical meaning in Suricata.

8.51.18 Negated Content Match Special Case

- For Snort, a *negated* content match where the starting point for searching is at or beyond the end of the inspection buffer will never return true.
 - For negated matches, you want it to return true if the content is not found.
 - This is believed to be a Snort bug rather than an engine difference but it was reported to Sourcefire and acknowledged many years ago indicating that perhaps it is by design.
 - This is not the case for Suricata which behaves as expected.

Example HTTP request:

```
POST /test.php HTTP/1.1
Content-Length: 9

user=suri
```

This rule snippet will never return true in Snort but will in Suricata:

```
content:!"snort"; offset:10; http_client_body;
```

8.51.19 File Extraction

- Suricata has the ability to match on files from FTP, HTTP and SMTP streams and log them to disk.
- Snort has the "file" preprocessor that can do something similar but it is experimental, development of it has been stagnant for years, and it is not something that should be used in a production environment.
- Files can be matched on using a number of keywords including:
 - `filename`
 - `fileext`
 - `filemagic`
 - `filesize`
 - `filemd5`

- filesha1
 - filesha256
 - filesize
 - See *File Keywords* for a full list.
- The `filestore` keyword tells Suricata to save the file to disk.
 - Extracted files are logged to disk with meta data that includes things like timestamp, src/dst IP, protocol, src/dst port, HTTP URI, HTTP Host, HTTP Referer, filename, file magic, md5sum, size, etc.
 - There are a number of configuration options and considerations (such as stream reassembly depth and libhttp body-limit) that should be understood if you want fully utilize file extraction in Suricata.
 - *File Keywords*
 - *File Extraction*
 - <https://blog.inliniac.net/2011/11/29/file-extraction-in-suricata/>
 - <https://blog.inliniac.net/2014/11/11/smtp-file-extraction-in-suricata/>

8.51.20 Lua Scripting

- Suricata has the `lua` keyword which allows for a rule to reference a Lua script that can access the packet, payload, HTTP buffers, etc.
- Provides powerful flexibility and capabilities that Snort does not have.
- More details in: *Lua Scripting for Detection*

8.51.21 Fast Pattern

- Snort's fast pattern matcher is always case insensitive; Suricata's is case sensitive unless 'nocase' is set on the content match used by the fast pattern matcher.
- Snort will truncate fast pattern matches based on the `max-pattern-len` config (default no limit) unless `fast_pattern:only` is used in the rule. Suricata does not do any automatic fast pattern truncation cannot be configured to do so.
- Just like in Snort, in Suricata you can specify a substring of the content string to be use as the fast pattern match. e.g. `fast_pattern:5,20;`
- In Snort, leading NULL bytes (0x00) will be removed from content matches when determining/using the longest content match unless `fast_pattern` is explicitly set. Suricata does not truncate anything, including NULL bytes.
- Snort does not allow for all `http_*` buffers to be used for the fast pattern match (e.g. `http_raw_*`, `http_method`, `http_cookie`, etc.). Suricata lets you use any 'http_*' buffer you want for the fast pattern match, including `http_raw_*` and `http_cookie` buffers.
- Suricata supports the `fast_pattern:only` syntax but technically it is not really implemented; the `only` is silently ignored when encountered in a rule. It is still recommended that you use `fast_pattern:only` where appropriate in case this gets implemented in the future and/or if the rule will be used by Snort as well.
- With Snort, unless `fast_pattern` is explicitly set, content matches in normalized HTTP Inspect buffers (e.g. `http_content_modifiers` such `http_uri`, `http_header`, etc.) take precedence over non-HTTP Inspect content matches, even if they are shorter. Suricata does the same thing and gives a higher 'priority' (precedence) to `http_*` buffers (except for `http_method`, `http_stat_code`, and `http_stat_msg`).
- See *Suricata Fast Pattern Determination Explained* for full details on how Suricata automatically determines which content to use as the fast pattern match.

- When in doubt about what is going to be use as the fast pattern match by Suricata, set `fast_pattern` explicitly in the rule and/or run Suricata with the `--engine-analysis` switch and view the generated file (`rules_fast_pattern.txt`).
- Like Snort, the fast pattern match is checked before `flowbits` in Suricata.
- Using Hyperscan as the MPM matcher (`mpm-algo` setting) for Suricata can greatly improve performance, especially when it comes to fast pattern matching. Hyperscan will also take into account depth and offset when doing fast pattern matching, something the other algorithms and Snort do not do.
- *fast_pattern*

8.51.22 Don't Cross The Streams

Suricata will examine network traffic as individual packets and, in the case of TCP, as part of a (reassembled) stream. However, there are certain rule keywords that only apply to packets only (`dsize`, `flags`, `ttl`) and certain ones that only apply to streams only (`http_*`) and you can't mix packet and stream keywords. Rules that use packet keywords will inspect individual packets only and rules that use stream keywords will inspect streams only. Snort is a little more forgiving when you mix these – for example, in Snort you can use `dsize` (a packet keyword) with `http_*` (stream keywords) and Snort will allow it although, because of `dsize`, it will only apply detection to individual packets (unless PAF is enabled then it will apply it to the PDU).

If `dsize` is in a rule that also looks for a stream-based application layer protocol (e.g. `http`), Suricata will not match on the *first application layer packet* since `dsize` make Suricata evaluate the packet and protocol detection doesn't happen until after the protocol is checked for that packet; *subsequent* packets in that flow should have the application protocol set appropriately and will match rules using `dsize` and a stream-based application layer protocol.

If you need to check sizes on a stream in a rule that uses a stream keyword, or in a rule looking for a stream-based application layer protocol, consider using the `stream_size` keyword and/or `isdataat`.

Suricata also supports these protocol values being used in rules and Snort does not:

- `tcp-pkt` – example:
 - `alert tcp-pkt ...`
 - This tells Suricata to only apply the rule to TCP packets and not the (reassembled) stream.
- `tcp-stream` – example:
 - `alert tcp-stream ...`
 - This tells Suricata to inspect the (reassembled) TCP stream only.

8.51.23 Alerts

- In Snort, the number of alerts generated for a packet/stream can be limited by the `event_queue` configuration.
- Suricata has an internal hard-coded limit of 15 alerts per packet/stream (and this cannot be configured); all rules that match on the traffic being analyzed will fire up to that limit.
- Sometimes Suricata will generate what appears to be two alerts for the same TCP packet. This happens when Suricata evaluates the packet by itself and as part of a (reassembled) stream.

8.51.24 Buffer Reference Chart

Buffer	Snort 2.9.x Support?	Suricata Support?	PCRE flag	Can be used as Fast Pattern?	Suricata Fast Pattern Priority (lower number is higher priority)
content (no modifier)	YES	YES	<none>	YES	3
http_method	YES	YES	M	Suricata only	3
http_stat_code	YES	YES	S	Suricata only	3
http_stat_reason	YES	YES	Y	Suricata only	3
uricontent	YES but deprecated, use http_uri instead	YES but deprecated, use http_uri instead	U	YES	2
http_uri	YES	YES	U	YES	2
http_raw_uri	YES	YES	I	Suricata only	2
http_header	YES	YES	H	YES	2
http_raw_header	YES	YES	D	Suricata only	2
http_cookie	YES	YES	C	Suricata only	2
http_raw_content	YES	NO (use http_raw_header instead)	K	NO	n/a
http_host	NO	YES	W	Suricata only	2
http_raw_host	NO	YES	Z	Suricata only	2
http_client_ip	YES	YES	P	YES	2
http_server_ip	NO	YES	Q	Suricata only	2
http_user_agent	NO	YES	V	Suricata only	2
dns_query	NO	YES	n/a*	Suricata only	2
tls_sni	NO	YES	n/a*	Suricata only	2
tls_cert_issuer	NO	YES	n/a*	Suricata only	2
tls_cert_subject	NO	YES	n/a*	Suricata only	2
file_data	YES	YES	n/a*	YES	2

* Sticky buffer

8.52 Multiple Buffer Matching

Suricata 7 and newer now supports matching contents in multiple buffers within the same transaction.

For example a single DNS transaction that has two queries in it:

query 1: example.net query 2: something.com

Example rule:

```
alert dns $HOME_NET any -> $EXTERNAL_NET any (msg:"DNS Multiple Question Example Rule"; dns.query; content:"example"; dns.query; content:".com"; classtype:misc-activity; sid:1; rev:1;)
```

Within the single DNS query transaction, there are two queries and Suricata will set up two instances of a dns.query buffer.

The first dns.query buffer will look for content:"example";

The second dns.query buffer will look for content:".com";

The example rule will alert on the example query since all the content matches are satisfied for the rule.

For matching multiple headers in HTTP2 traffic a rule using the new functionality would look like:

```
alert http2 any any -> any any (msg:"HTTP2 Multiple Header Buffer Example"; flow:established,to_server;
http.request_header; content:"method|3a 20|GET"; http.request_header; content:"authority|3a 20|example.com";
classtype:misc-activity; sid:1; rev:1;)
```

With HTTP2 there are multiple headers seen in the same flow record. We now have a way to write a rule in a more efficient way using the multiple buffer capability.

Note Existing behavior when using sticky buffers still applies:

Example rule:

```
alert dns $HOME_NET any -> $EXTERNAL_NET any (msg:"DNS Query Sticky Buffer Classic Example Rule";
dns.query; content:"example"; content:".net"; classtype:misc-activity; sid:1; rev:1;)
```

The above rule will alert on a single dns query containing "example.net" or "example.domain.net" since the rule content matches are within a single `dns.query` buffer and all content match requirements of the rule are met.

Note: This is new behavior. In versions of Suricata prior to version 7 multiple statements of the same sticky buffer did not make a second instance of the buffer. For example:

```
dns.query; content:"example"; dns.query; content:".com";
```

would be equivalent to:

```
dns.query; content:"example"; content:".com";
```

Using our example from above, the first query is for example.net which matches `content:"example"`; but does not match `content:".com"`;

The second query is for something.com which would match on the `content:".com"`; but not the `content:"example"`;

So with the Suricata behavior prior to Suricata 7, the signature would not fire in this case since both content conditions will not be met.

Multiple buffer matching is currently enabled for use with the following keywords:

- `dns.additional.rrname`
- `dns.answer.name`
- `dns.answers.rrname`
- `dns.authorities.rrname`
- `dns.queries.rrname`
- `dns.query`
- `dns.query.name`
- `email.received`
- `email.url`
- `file.data`
- `file.magic`
- `file.name`
- `ftp.completion_code`
- `ftp.reply`
- `http.request_header`
- `http.response_header`

- `http2.header_name`
- `ike.vendor`
- `krb5.cname`
- `krb5.sname`
- `ldap.request.attribute_type`
- `ldap.responses.attribute_type`
- `ldap.responses.dn`
- `ldap.responses.message`
- `mdns.additional_rrname`
- `mdns.answers_rrname`
- `mdns.authorities_rrname`
- `mdns.queries_rrname`
- `mqtt.subscribe.topic`
- `mqtt.unsubscribe.topic`
- `quic.cyu.hash`
- `quic.cyu.string`
- `sdp.attribute`
- `sdp.bandwidth`
- `sdp.media.connection_data`
- `sdp.media.encryption_key`
- `sdp.media.media`
- `sdp.media.media_info`
- `sdp.repeat_time`
- `sdp.time`
- `sip.content_length`
- `sip.content_type`
- `sip.from`
- `sip.to`
- `sip.user_agent`
- `sip.via`
- `smtp.rcpt_to`
- `tls.alpn`
- `tls.certs`
- `tls.subjectaltname`

8.53 Tag

The *tag* keyword allows tagging of the current and future packets.

Tagged packets can be logged in *EVE* and conditional PCAP logging.

Tagging is limited to a scope: *host* or *session* (flow). When using *host* a direction can be specified: *src* or *dst*. Tagging will then occur based on the *src* or *dst* IP address of the packet generating the alert.

Tagging is further controlled by count: *packets*, *bytes* or *seconds*. If the count is omitted built-in defaults will be used:

- for *session*: 256 packets
- for *host*: 256 packets for the destination IP of the packet triggering the alert

The *tag* keyword can appear multiple times in a rule.

8.53.1 Syntax

```
tag:<scope>[,<count>, <metric>[,<direction>]];
```

Values for *scope*: *session* and *host* Values for *metric*: *packets*, *bytes*, *seconds* Values for *direction*: *src* and *dst*

Note

"direction" can only be specified if scope is "host" and both "count" and "metric" are also specified.

8.53.2 Examples

Keyword:

```
tag:session;           # tags next 256 packets in the flow
tag:host;              # tags next 256 packets for the dst ip of the alert
tag:host,100,packets,src; # tags next 100 packets for src ip of the alert
tag:host,3600,seconds,dst; # tags packets for dst host for the next hour
```

Full rule examples:

```
alert dns any any -> any any (dns.query; content:"evil"; tag:host,60,seconds,src; sid:1;)
```

```
alert http any any -> any any (http.method; content:"POST"; tag:session; sid:1;)
```

8.53.3 How to Use Tags

EVE

Tags can be set to generate *EVE tag* records:

```
outputs:
- eve-log:
  enabled: yes
  filename: eve.json
  types:
  - alert:
    tagged-packets: true
```

The tagged packets will then be logged with *event_type*: *packet*:

```
{
  "timestamp": "2020-06-03T10:29:17.850417+0000",
  "flow_id": 1576832511820424,
  "event_type": "packet",
  "src_ip": "192.168.0.27",
  "src_port": 54634,
  "dest_ip": "192.168.0.103",
  "dest_port": 22,
  "proto": "TCP",
  "pkt_src": "wire/pcap",
  "packet":
  → "CAAn6mWJAPSNvfrHCABFAAAogkVAAIAG9rfAqAAbwKgAZ9VqABZvnJXH5Zf6aFAQEAljEwAAAAAAAAAAAA",
  "packet_info": {
    "linktype": 1
  }
}
```

EVE: *Eve JSON Output*

Conditional PCAP Logging

Using the conditional PCAP logging option the tag keyword can control which packets are logged by the PCAP logging.

```
outputs:
- pcap-log:
  enabled: yes
  filename: log.pcap
  limit: 1000mb
  max-files: 2000
  compression: none
  mode: normal
  use-stream-depth: no #If set to "yes" packets seen after reaching stream
  → inspection depth are ignored. "no" logs all packets
  honor-pass-rules: no # If set to "yes", flows in which a pass rule matched will
  → stop being logged.
  # Use "all" to log all packets or use "alerts" to log only alerted packets and
  → flows or "tag"
  # to log only flow tagged via the "tag" keyword
  conditional: tag
```

PCAP Logging: *PCAP log*

8.53.4 Tracking by Host/Flow

When the tags are using the *session* scope, the tag is added to the *Flow* structure. If a packet has no flow, no tagging will happen. No errors/warnings are generated for this.

See *Flow Settings* for managing flow limits and resources.

When tags are using the *host* scope, the tag is stored with a *Host* object in the host table. The Host table size will affect effectiveness of per host tags.

See *Host Settings* for managing host table size.

8.54 VLAN Keywords

8.54.1 vlan.id

Suricata has a `vlan.id` keyword that can be used in signatures to identify and filter network packets based on Virtual Local Area Network IDs. By default, it matches all layers if a packet contains multiple VLAN layers. However, if a specific layer is defined, it will only match that layer.

Syntax:

```
vlan.id: [op]id[,layer];
```

The id can be matched exactly, or compared using the op setting:

```
vlan.id:300      # exactly 300
vlan.id:<300,0    # smaller than 300 at layer 0
vlan.id:>=200,1   # greater or equal than 200 at layer 1
```

`vlan.id` uses *unsigned 16-bit integer*.

The valid range for VLAN id values is 0 - 4095.

This keyword also supports `all` and `any` as arguments for `layer`. `all` matches only if all VLAN layers match and `any` matches with any layer.

Table 2: Layer values for `vlan.id` keyword

Value	Description
[default]	Match with any layer
0 - 2	Match specific layer
-3 - -1	Match specific layer with back to front indexing
all	Match only if all layers match
any	Match with any layer

This small illustration shows how indexing works for `vlan.id`:

```
[ethernet]
[vlan 666 (index 0 and -2)]
[vlan 123 (index 1 and -1)]
[ipv4]
[udp]
```

Examples

Example of a signature that would alert if any of the VLAN IDs is equal to 300:

```
alert ip any any -> any any (msg:"Vlan ID is equal to 300"; vlan.id:300; sid:1;)
```

Example of a signature that would alert if the VLAN ID at layer 1 is equal to 300:

```
alert ip any any -> any any (msg:"Vlan ID is equal to 300 at layer 1"; vlan.id:300,1; sid:1;)
```

Example of a signature that would alert if the VLAN ID at the last layer is equal to 400:

```
alert ip any any -> any any (msg:"Vlan ID is equal to 400 at the last layer"; vlan.id:400,-1; sid:1;)
```

Example of a signature that would alert only if all the VLAN IDs are greater than 100:

```
alert ip any any -> any any (msg:"All Vlan IDs are greater than 100"; vlan.id:>100,all; sid:1;)
```

It is also possible to use the `vlan.id` content as a `fast_pattern` by using the `prefilter` keyword, as shown in the following example.

```
alert ip any any -> any any (msg:"Vlan ID is equal to 200 at layer 1"; vlan.id:200,1; prefilter; sid:1;)
```

8.54.2 `vlan.layers`

Matches based on the number of layers.

Syntax:

```
vlan.layers: [op]number;
```

It can be matched exactly, or compared using the `op` setting:

```
vlan.layers:3      # exactly 3 vlan layers  
vlan.layers:<3     # less than 3 vlan layers  
vlan.layers:>=2    # more or equal to 2 vlan layers
```

`vlan.layers` uses *unsigned 8-bit integer*.

The minimum and maximum values that `vlan.layers` can be are 0 and 3.

Examples

Example of a signature that would alert if a packet has 0 VLAN layers:

```
alert ip any any -> any any (msg:"Packet has 0 vlan layers"; vlan.layers:0; sid:1;)
```

Example of a signature that would alert if a packet has more than 1 VLAN layers:

```
alert ip any any -> any any (msg:"Packet has more than 1 vlan layer"; vlan.layers:>1; sid:1;)
```

It is also possible to use the `vlan.layers` content as a `fast_pattern` by using the `prefilter` keyword, as shown in the following example.

```
alert ip any any -> any any (msg:"Packet has 2 vlan layers"; vlan.layers:2; prefilter; sid:1;)
```

8.55 LDAP Keywords

8.55.1 LDAP Request and Response operations

Table 3: Operation values for `ldap.request.operation` and `ldap.responses.operation` keywords

Code	Operation
0	bind_request
1	bind_response
2	unbind_request
3	search_request
4	search_result_entry
5	search_result_done
6	modify_request
7	modify_response
8	add_request
9	add_response
10	del_request
11	del_response
12	mod_dn_request
13	mod_dn_response
14	compare_request
15	compare_response
16	abandon_request
19	search_result_reference
23	extended_request
24	extended_response
25	intermediate_response

The keywords `ldap.request.operation` and `ldap.responses.operation` accept both the operation code and the operation name as arguments.

8.55.2 `ldap.request.operation`

Suricata has a `ldap.request.operation` keyword that can be used in signatures to identify and filter network packets based on Lightweight Directory Access Protocol request operations.

Syntax:

```
ldap.request.operation: operation;
```

`ldap.request.operation` uses *unsigned 8-bit integer*.

This keyword maps to the EVE field `ldap.request.operation`

Examples

Example of a signatures that would alert if the packet has an LDAP bind request operation:

```
alert ldap any any -> any any (msg:"Test LDAP bind request"; ldap.request.operation:0; sid:1;)
```

```
alert ldap any any -> any any (msg:"Test LDAP bind request"; ldap.request.operation:bind_request; sid:1;)
```

8.55.3 ldap.responses.operation

Suricata has a `ldap.responses.operation` keyword that can be used in signatures to identify and filter network packets based on Lightweight Directory Access Protocol response operations.

Syntax:

```
ldap.responses.operation: operation[,index];
```

`ldap.responses.operation` uses *unsigned 8-bit integer*.

This keyword maps to the EVE field `ldap.responses[].operation`

An LDAP request operation can receive multiple responses. By default, the `ldap.responses.operation` keyword matches all indices, but it is possible to specify a particular index for matching and also use flags such as `all` and `any`.

Table 4: Index values for `ldap.responses.operation` keyword

Value	Description
[default]	Match with any index
all	Match only if all indexes match
any	Match with any index
0>=	Match specific index
0<	Match specific index with back to front indexing

Examples

Example of a signatures that would alert if the packet has an LDAP bind response operation:

```
alert ldap any any -> any any (msg:"Test LDAP bind response"; ldap.responses.operation:1; sid:1;)
```

```
alert ldap any any -> any any (msg:"Test LDAP bind response"; ldap.responses.operation:bind_response; sid:1;)
```

Example of a signature that would alert if the packet has an LDAP search_result_done response operation at index 1:

```
alert ldap any any -> any any (msg:"Test LDAP search response"; ldap.responses.operation:search_result_done,1; sid:1;)
```

Example of a signature that would alert if all the responses are of type `search_result_entry`:

```
alert ldap any any -> any any (msg:"Test LDAP search response"; ldap.responses.operation:search_result_entry,all; sid:1;)
```

The keyword `ldap.responses.operation` supports back to front indexing with negative numbers, this means that `-1` will represent the last index, `-2` the second to last index, and so on. This is an example of a signature that would alert if a `search_result_entry` response is found at the last index:

```
alert ldap any any -> any any (msg:"Test LDAP search response"; ldap.responses.operation:search_result_entry,-1; sid:1;)
```

8.55.4 ldap.responses.count

Matches based on the number of responses.

Syntax:

```
ldap.responses.count: [op]number;
```

It can be matched exactly, or compared using the `op` setting:

```

ldap.responses.count:3      # exactly 3 responses
ldap.responses.count:<3     # less than 3 responses
ldap.responses.count:>=2    # more or equal to 2 responses

```

ldap.responses.count uses *unsigned 32-bit integer*.

This keyword maps to the EVE field `len(ldap.responses[])`

Examples

Example of a signature that would alert if a packet has 0 LDAP responses:

```
alert ldap any any -> any any (msg:"Packet has 0 LDAP responses"; ldap.responses.count:0; sid:1;)
```

Example of a signature that would alert if a packet has more than 2 LDAP responses:

```
alert ldap any any -> any any (msg:"Packet has more than 2 LDAP responses"; ldap.responses.count:>2; sid:1;)
```

8.55.5 ldap.request.dn

Matches on LDAP distinguished names from request operations.

Comparison is case-sensitive.

Syntax:

```
ldap.request.dn; content:"<content to match against>";
```

ldap.request.dn is a 'sticky buffer' and can be used as a *fast_pattern*.

This keyword maps to the EVE fields:

- ldap.request.bind_request.name
- ldap.request.add_request.entry
- ldap.request.search_request.base_object
- ldap.request.modify_request.object
- ldap.request.del_request.dn
- ldap.request.mod_dn_request.entry
- ldap.request.compare_request.entry

Example

Example of a signature that would alert if a packet has the LDAP distinguished name uid=jdoe,ou=People,dc=example,dc=com:

```
alert ldap any any -> any any (msg:"Test LDAPDN"; ldap.request.dn; content:"uid=jdoe,ou=People,dc=example,dc=com"; sid:1;)
```

It is possible to use the keyword `ldap.request.operation` in the same rule to specify the operation to match.

Here is an example of a signature that would alert if a packet has an LDAP search request operation and contains the LDAP distinguished name dc=example,dc=com.

```
alert ldap any any -> any any (msg:"Test LDAPDN and operation"; ldap.request.operation:search_request; ldap.request.dn; content:"dc=example,dc=com"; sid:1;)
```

8.55.6 ldap.responses.dn

Matches on LDAP distinguished names from response operations.

Comparison is case-sensitive.

Syntax:

```
ldap.responses.dn; content:"<content to match against>";
```

`ldap.responses.dn` is a 'sticky buffer' and can be used as a `fast_pattern`.

`ldap.responses.dn` supports multiple buffer matching, see [Multiple Buffer Matching](#).

This keyword maps to the EVE fields:

- `ldap.responses[].search_result_entry.base_object`
- `ldap.responses[].bind_response.matched_dn`
- `ldap.responses[].search_result_done.matched_dn`
- `ldap.responses[].modify_response.matched_dn`
- `ldap.responses[].add_response.matched_dn`
- `ldap.responses[].del_response.matched_dn`
- `ldap.responses[].mod_dn_response.matched_dn`
- `ldap.responses[].compare_response.matched_dn`
- `ldap.responses[].extended_response.matched_dn`

Note

If a response within the array does not contain the distinguished name field, this field will be interpreted as an empty buffer.

Example

Example of a signature that would alert if a packet has the LDAP distinguished name `dc=example,dc=com`:

```
alert ldap any any -> any any (msg:"Test LDAPDN"; ldap.responses.dn; content:"dc=example,dc=com"; sid:1;)
```

It is possible to use the keyword `ldap.responses.operation` in the same rule to specify the operation to match.

Here is an example of a signature that would alert if a packet has an LDAP search result entry operation at index 1 on the responses array, and contains the LDAP distinguished name `dc=example,dc=com`.

```
alert ldap any any -> any any (msg:"Test LDAPDN and operation"; ldap.responses.operation:search_result_entry,1; ldap.responses.dn; content:"dc=example,dc=com"; sid:1;)
```

8.55.7 ldap.responses.result_code

Suricata has a `ldap.responses.result_code` keyword that can be used in signatures to identify and filter network packets based on their LDAP result code.

Syntax:

```
ldap.responses.result_code: code[,index];
```

`ldap.responses.result_code` uses *unsigned 32-bit integer*.

This keyword maps to the following eve fields:

- `ldap.responses[].bind_response.result_code`
- `ldap.responses[].search_result_done.result_code`
- `ldap.responses[].modify_response.result_code`
- `ldap.responses[].add_response.result_code`
- `ldap.responses[].del_response.result_code`
- `ldap.responses[].mod_dn_response.result_code`
- `ldap.responses[].compare_response.result_code`
- `ldap.responses[].extended_response.result_code`

Table 5: Result code values for `ldap.responses.result_code`

Code	Name
0	success
1	operations_error
2	protocol_error
3	time_limit_exceeded
4	size_limit_exceeded
5	compare_false
6	compare_true
7	auth_method_not_supported
8	stronger_auth_required
10	referral
11	admin_limit_exceeded
12	unavailable_critical_extension
13	confidentiality_required
14	sasl_bind_in_progress
16	no_such_attribute
17	undefined_attribute_type
18	inappropriate_matching
19	constraint_violation
20	attribute_or_value_exists
21	invalid_attribute_syntax
32	no_such_object
33	alias_problem
34	invalid_dns_syntax
35	is_leaf
36	alias_dereferencing_problem
48	inappropriate_authentication
49	invalid_credentials
50	insufficient_access_rights
51	busy
52	unavailable
53	unwilling_to_perform
54	loop_detect
60	sort_control_missing
61	offset_range_error

continues on next page

Table 5 – continued from previous page

Code	Name
64	naming_violation
65	object_class_violation
66	not_allowed_on_non_leaf
67	not_allowed_on_rdn
68	entry_already_exists
69	object_class_mods_prohibited
70	results_too_large
71	affects_multiple_dsas
76	control_error
80	other
81	server_down
82	local_error
83	encoding_error
84	decoding_error
85	timeout
86	auth_unknown
87	filter_error
88	user_canceled
89	param_error
90	no_memory
91	connect_error
92	not_supported
93	control_not_found
94	no_results_returned
95	more_results_to_return
96	client_loop
97	referral_limit_exceeded
100	invalid_response
101	ambiguous_response
112	tls_not_supported
113	intermediate_response
114	unknown_type
118	canceled
119	no_such_operation
120	too_late
121	cannot_cancel
122	assertion_failed
123	authorization_denied
4096	e_sync_refresh_required
16654	no_operation

More information about LDAP result code values can be found here: <https://ldap.com/ldap-result-code-reference/>

An LDAP request operation can receive multiple responses. By default, the `ldap.responses.result_code` keyword matches with any indices, but it is possible to specify a particular index for matching and also use flags such as `all` and `any`.

Table 6: Index values for `ldap.responses.result_code` keyword

Value	Description
[default]	Match with any index
all	Match only if all indexes match
any	Match with any index
0>=	Match specific index
0<	Match specific index with back to front indexing

Examples

Example of signatures that would alert if the packet has a **success** LDAP result code at any index:

```
alert ldap any any -> any any (msg:"Test LDAP result code"; ldap.responses.result_code:0; sid:1;)
```

```
alert ldap any any -> any any (msg:"Test LDAP result code"; ldap.responses.result_code:success,any; sid:1;)
```

Example of a signature that would alert if the packet has an **unavailable** LDAP result code at index 1:

```
alert ldap any any -> any any (msg:"Test LDAP result code at index 1"; ldap.responses.result_code:unavailable,1; sid:1;)
```

Example of a signature that would alert if all the responses have a **success** LDAP result code:

```
alert ldap any any -> any any (msg:"Test all LDAP responses have success result code";
ldap.responses.result_code:success,all; sid:1;)
```

The keyword `ldap.responses.result_code` supports back to front indexing with negative numbers, this means that `-1` will represent the last index, `-2` the second to last index, and so on. This is an example of a signature that would alert if a success result code is found at the last index:

```
alert ldap any any -> any any (msg:"Test LDAP success at last index"; ldap.responses.result_code:success,-1; sid:1;)
```

8.55.8 ldap.responses.message

Matches on LDAP error messages from response operations.

Comparison is case-sensitive.

Syntax:

```
ldap.responses.message; content:"<content to match against>";
```

`ldap.responses.message` is a 'sticky buffer' and can be used as a `fast_pattern`.

`ldap.responses.message` supports multiple buffer matching, see [Multiple Buffer Matching](#).

This keyword maps to the EVE fields:

- `ldap.responses[].bind_response.message`
- `ldap.responses[].search_result_done.message`
- `ldap.responses[].modify_response.message`
- `ldap.responses[].add_response.message`
- `ldap.responses[].del_response.message`
- `ldap.responses[].mod_dn_response.message`
- `ldap.responses[].compare_response.message`
- `ldap.responses[].extended_response.message`

Note

If a response within the array does not contain the error message field, this field will be interpreted as an empty buffer.

Example

Example of a signature that would alert if a packet has the LDAP error message `Size limit exceeded`:

```
alert ldap any any -> any any (msg:"Test LDAP error message"; ldap.responses.message; content:"Size limit exceeded"; sid:1;)
```

8.55.9 ldap.request.attribute_type

Matches on LDAP attribute type from request operations.

Comparison is case-sensitive.

Syntax:

```
ldap.request.attribute_type; content:"<content to match against>";
```

`ldap.request.attribute_type` is a 'sticky buffer' and can be used as a `fast_pattern`.

`ldap.request.attribute_type` supports multiple buffer matching, see *Multiple Buffer Matching*.

This keyword maps to the EVE fields:

- `ldap.request.search_request.attributes[]`
- `ldap.request.modify_request.changes[].modification.attribute_type`
- `ldap.request.add_request.attributes[].name`
- `ldap.request.compare_request.attribute_value_assertion.description`

Example

Example of a signature that would alert if a packet has the LDAP attribute type `objectClass`:

```
alert ldap any any -> any any (msg:"Test attribute type"; ldap.request.attribute_type; content:"objectClass"; sid:1;)
```

It is possible to use the keyword `ldap.request.operation` in the same rule to specify the operation to match.

Here is an example of a signature that would alert if a packet has an LDAP add request operation and contains the LDAP attribute type `objectClass`.

```
alert ldap any any -> any any (msg:"Test attribute type and operation"; ldap.request.operation:add_request; ldap.request.attribute_type; content:"objectClass"; sid:1;)
```

8.55.10 ldap.responses.attribute_type

Matches on LDAP attribute type from response operations.

Comparison is case-sensitive.

Syntax:

```
ldap.responses.attribute_type; content:"<content to match against>";
```

`ldap.responses.attribute_type` is a 'sticky buffer' and can be used as a `fast_pattern`.

`ldap.responses.attribute_type` supports multiple buffer matching, see [Multiple Buffer Matching](#).

This keyword maps to the EVE field `ldap.responses[].search_result_entry.attributes[].type`

Example

Example of a signature that would alert if a packet has the LDAP attribute type `dc`:

```
alert ldap any any -> any any (msg:"Test responses attribute type"; ldap.responses.attribute_type; content:"dc"; sid:1;)
```

It is possible to use the keyword `ldap.responses.operation` in the same rule to specify the operation to match.

Here is an example of a signature that would alert if a packet has an LDAP search result entry operation at index 1 on the responses array, and contains the LDAP attribute type `dc`.

```
alert ldap any any -> any any (msg:"Test attribute type and operation"; ldap.responses.operation:search_result_entry,1; ldap.responses.attribute_type; content:"dc"; sid:1;)
```

8.56 PGSQL Keywords

8.56.1 pgsql.query

This keyword is a sticky buffer that allows matching on the contents of PostgreSQL's *query* request messages parsed by the engine. Note that this buffer inspects only the *string* portion of the PostgreSQL message, skipping other fields such as identifier and length, and focusing on the query itself.

Currently, it exposes the contents of the `pgsql.request.simple_query` field from EVE output.

`pgsql.query` can be used as a `fast_pattern` (see [fast_pattern](#)).

Use `nocase` with this keyword to avoid case sensitivity for the matches.

Examples

```
alert pgsql any any -> any any (msg:"Simple SELECT rule"; pgsql.query; content:"SELECT *"; sid:1;)
```

```
alert pgsql any any -> any any (msg:"Simple delete rule"; pgsql.query; content:"delete"; nocase sid:2;)
```

8.57 Email Keywords

8.57.1 email.from

Matches the MIME From field of an email.

Comparison is case-sensitive.

Syntax:

```
email.from; content:"<content to match against>";
```

`email.from` is a 'sticky buffer' and can be used as a `fast_pattern`.

This keyword maps to the EVE field `email.from`

Example

Example of a signature that would alert if a packet contains the MIME field `from` with the value `toto <toto@gmail.com>`

```
alert smtp any any -> any any (msg:"Test mime email from"; email.from; content:"toto <toto@gmail.com>"; sid:1;)
```

8.57.2 email.subject

Matches the MIME Subject field of an email.

Comparison is case-sensitive.

Syntax:

```
email.subject; content:"<content to match against>";
```

`email.subject` is a 'sticky buffer' and can be used as a `fast_pattern`.

This keyword maps to the EVE field `email.subject`

Example

Example of a signature that would alert if a packet contains the MIME field `subject` with the value `This is a test email`

```
alert smtp any any -> any any (msg:"Test mime email subject"; email.subject; content:"This is a test email"; sid:1;)
```

8.57.3 email.to

Matches the MIME To field of an email.

Comparison is case-sensitive.

Syntax:

```
email.to; content:"<content to match against>";
```

`email.to` is a 'sticky buffer' and can be used as a `fast_pattern`.

This keyword maps to the EVE field `email.to`

Example

Example of a signature that would alert if a packet contains the MIME field `to` with the value `172.16.92.2@linuxbox`

```
alert smtp any any -> any any (msg:"Test mime email to"; email.to; content:"172.16.92.2@linuxbox"; sid:1;)
```

8.57.4 email.cc

Matches the MIME Cc field of an email.

Comparison is case-sensitive.

Syntax:

```
email.cc; content:"<content to match against>";
```

`email.cc` is a 'sticky buffer' and can be used as a `fast_pattern`.

This keyword maps to the EVE field `email.cc[]`

Example

Example of a signature that would alert if a packet contains the MIME field `cc` with the value Emily <emily.roberts@example.com>, Ava <ava.johnson@example.com>, Sophia Wilson <sophia.wilson@example.com>

```
alert smtp any any -> any any (msg:"Test mime email cc"; email.cc; content:"Emily <emily.roberts@example.com>,
Ava <ava.johnson@example.com>, Sophia Wilson <sophia.wilson@example.com>"; sid:1;)
```

8.57.5 email.date

Matches the MIME Date field of an email.

Comparison is case-sensitive.

Syntax:

```
email.date; content:"<content to match against>";
```

`email.date` is a 'sticky buffer' and can be used as a `fast_pattern`.

This keyword maps to the EVE field `email.date`

Example

Example of a signature that would alert if a packet contains the MIME field `date` with the value Fri, 21 Apr 2023 05:10:36 +0000

```
alert smtp any any -> any any (msg:"Test mime email date"; email.date; content:"Fri, 21 Apr 2023 05:10:36 +0000";
sid:1;)
```

8.57.6 email.message_id

Matches the MIME Message-Id field of an email.

Comparison is case-sensitive.

Syntax:

```
email.message_id; content:"<content to match against>";
```

`email.message_id` is a 'sticky buffer' and can be used as a `fast_pattern`.

This keyword maps to the EVE field `email.message_id`

Example

Example of a signature that would alert if a packet contains the MIME field `message id` with the value <alpine.DEB.2.00.1311261630120.9535@sd-26634.dedibox.fr>

```
alert smtp any any -> any any (msg:"Test mime email message id"; email.message_id;
content:"<alpine.DEB.2.00.1311261630120.9535@sd-26634.dedibox.fr>"; sid:1;)
```

8.57.7 email.x_mailer

Matches the MIME X-Mailer field of an email.

Comparison is case-sensitive.

Syntax:

```
email.x_mailer; content:"<content to match against>;"
```

`email.x_mailer` is a 'sticky buffer' and can be used as a `fast_pattern`.

This keyword maps to the EVE field `email.x_mailer`

Example

Example of a signature that would alert if a packet contains the MIME field `x-mailer` with the value `Microsoft Office Outlook, Build 11.0.5510`

```
alert smtp any any -> any any (msg:"Test mime email x-mailer"; email.x_mailer; content:"Microsoft Office Outlook, Build 11.0.5510"; sid:1;)
```

8.57.8 email.url

Matches URL extracted of an email.

Comparison is case-sensitive.

Syntax:

```
email.url; content:"<content to match against>;"
```

`email.url` is a 'sticky buffer' and can be used as a `fast_pattern`.

`email.url` supports multiple buffer matching, see *Multiple Buffer Matching*.

This keyword maps to the EVE field `email.url[]`

Example

Example of a signature that would alert if an email contains the url `test-site.org/blah/123/`.

```
alert smtp any any -> any any (msg:"Test mime email url"; email.url; content:"test-site.org/blah/123/"; sid:1;)
```

8.57.9 email.received

Matches Received field of an email.

Comparison is case-sensitive.

Syntax:

```
email.received; content:"<content to match against>;"
```

`email.received` is a 'sticky buffer' and can be used as a `fast_pattern`.

`email.received` supports multiple buffer matching, see *Multiple Buffer Matching*.

This keyword maps to the EVE field `email.received[]`

Example

Example of a signature that would alert if a packet contains the MIME field `received` with the value from [65.201.218.30] (helo=COZOXORY.club)by 173-66-46-112.wash.fios.verizon.net with esmtpa (Exim 4.86)(envelope-from)id 71cF63a9for mirjam@abracadabra.ch; Mon, 29 Jul 2019 17:01:45 +0000

alert smtp any any -> any any (msg:"Test mime email received"; email.received; content:"from [65.201.218.30] (helo=COZOXORY.club)by 173-66-46-112.wash.fios.verizon.net with esmtpa (Exim 4.86)(envelope-from)id 71cF63a9for mirjam@abrakadabra.ch; Mon, 29 Jul 2019 17:01:45 +0000"; sid:1;)

8.58 Rule Types and Categorization

Once parsed, Suricata rules are categorized for performance and further processing (as different rule types will be handled by specific engine modules). The signature types are defined in `src/detect.h`:

Listing 1: `src/detect.h`

```
enum SignatureType {
    SIG_TYPE_NOT_SET = 0,
    SIG_TYPE_IPONLY,      // rule is handled by IPONLY engine
    SIG_TYPE_LIKE_IPONLY, // rule is handled by pkt engine, has action effect like ip-
    → only
    /** Proto detect only signature.
     *  Inspected once per direction when protocol detection is done. */
    SIG_TYPE_PDONLY, // rule is handled by PDONLY engine
    SIG_TYPE_DEONLY,
    SIG_TYPE_PKT,
    SIG_TYPE_PKT_STREAM,
    SIG_TYPE_STREAM,

    SIG_TYPE_APPLAYER, // app-layer but not tx, e.g. appproto
    SIG_TYPE_APP_TX,   // rule is handled by TX engine

    SIG_TYPE_MAX,
};
```

In more human readable terms:

Table 7: Suricata Rule Types, and their Engine Analysis Term

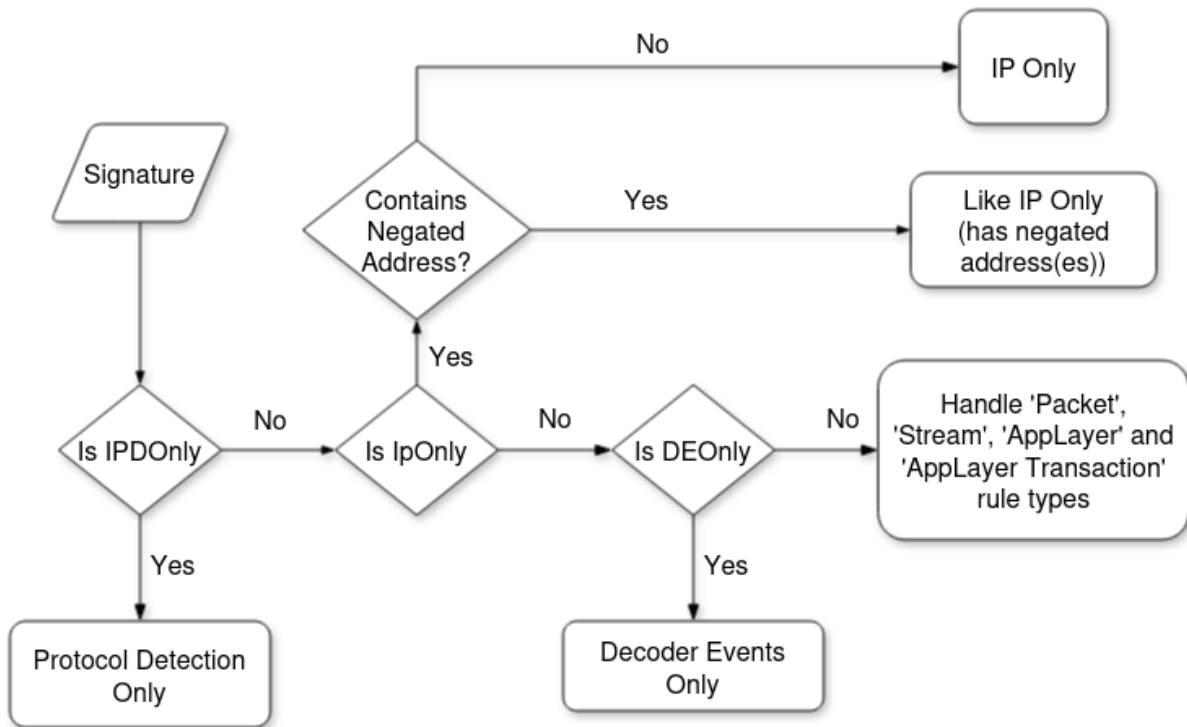
Rule Type	Code Symbol	Engine-Analysis Representation
Decoder Events Only	SIG_TYPE_DEONLY	de_only
Packet	SIG_TYPE_PKT	pkt
IP Only	SIG_TYPE_IPONLY	ip_only
IP Only (contains negated address(es))	SIG_TYPE_LIKE_IPONLY	like_ip_only
Protocol Detection Only	SIG_TYPE_PDONLY	pd_only
Packet-Stream	SIG_TYPE_PKT_STREAM	pkt_stream
Stream	SIG_TYPE_STREAM	stream
Application Layer Protocol	SIG_TYPE_APPLAYER	app_layer
Application Layer Protocol Transactions	SIG_TYPE_APP_TX	app_tx

The rule type will impact:

- To what does the signature action apply, in case of a match (*Action Scope*)
- When is the rule matched against traffic (*Inspection Hook*)
- Against what the rule matches (*Data Exposed*)

This categorization is done taking into consideration the presence or absence of certain rule elements, as well as the type of keywords used. The categorization currently takes place in `src/detect-engine-build.c:void SignatureSetType()`.

The SignatureSetType() overall flow is described below:



Flowcharts expanding uncovered functions or portions of the overall algorithm above are shown in the [Detailed Flowcharts](#) section.

The following table lists all Suricata signature types, and how they impact the aspects aforementioned.

Table 8: Suricata Rule Types

Type	Action Scope	Inspection Hook	Data Exposed	Keyword Examples (non-exhaustive)
<i>De-coder Events Only</i> (de_only)	Packet	Per-broken/ invalid packet	Decoding events	decode-event
<i>Packet (pkt)</i>	Packet	Per-packet basis	Packet-level info (e.g.: header info)	tcp-pkt, itype, tcp.hdr, tcp.seq, ttl etc.
<i>IP Only (ip_only)</i>	Flow (if existing). Packets (if not part of a flow)	Once per direction	IP addresses on the flow	Source/ Destination field of a rule
<i>IP Only (contains negated address)²</i> (like_ip)	Flow	All packets	IP addresses on the flow	Source/ Destination field of a rule containing negated address
<i>Protocol Detection Only</i> (pd_only)	Flow	Once per direction, when protocol detection is done	Protocol detected for the flow	app-layer-protocol
<i>Packet-Stream (pkt_str)</i>	Flow, if stateful ¹	Per stream chunk, if stateful, per-packet if not (stream payload AND packet payload)	The reassembled stream and/or payload data	content with startswith or depth
<i>Stream (stream)</i>	Flow, if stateful ¹	Stream chunks, if stateful, just packets if not	Stream reassembled payload or packet payload data	tcp-stream in protocol field; simple content; byte_extract
<i>Application Layer Protocol</i> (app_lay)	Flow	Per-packet basis	'protocol' field in a rule	Protocol field of a rule
<i>Application Layer Protocol Transactions</i> (app_tx)	Flow	Per <i>transaction</i> update	Buffer keywords	Application layer protocol-related, e.g. http.host, rfb.secrestult, dcerpc.stub_data, frame keywords

Note

Action Scope: *Flow, if stateful*

(1) Apply to the flow. If a segment isn't accepted into a stream for any reason (such as packet anomalies, errors, memcap reached etc), the rule will be applied on a packet level.

Warning

Although both are related to matching on application layer protocols, as the table suggests, since Suricata 7 a Protocol Detection rule (that uses the `app-layer-protocol` keyword) is not internally classified the same as a rule simply matching on the application layer protocol on the `protocol` field.

8.58.1 Signature Properties

The *Action Scope* mentioned above relates to the Signature Properties, as seen in `src/detect-engine.c`:

Listing 2: `src/detect-engine.c`

```
const struct SignatureProperties signature_properties[SIG_TYPE_MAX] = {
    /* SIG_TYPE_NOT_SET */      { SIG_PROP_FLOW_ACTION_PACKET, },
    /* SIG_TYPE_IPONLY */      { SIG_PROP_FLOW_ACTION_FLOW, },
    /* SIG_TYPE_LIKE_IPONLY */ { SIG_PROP_FLOW_ACTION_FLOW, },
    /* SIG_TYPE_PDONLY */      { SIG_PROP_FLOW_ACTION_FLOW, },
    /* SIG_TYPE_DEONLY */      { SIG_PROP_FLOW_ACTION_PACKET, },
    /* SIG_TYPE_PKT */          { SIG_PROP_FLOW_ACTION_PACKET, },
    /* SIG_TYPE_PKT_STREAM */   { SIG_PROP_FLOW_ACTION_FLOW_IF_STATEFUL, },
    /* SIG_TYPE_STREAM */       { SIG_PROP_FLOW_ACTION_FLOW_IF_STATEFUL, },
    /* SIG_TYPE_APPPLAYER */     { SIG_PROP_FLOW_ACTION_FLOW, },
    /* SIG_TYPE_APP_TX */       { SIG_PROP_FLOW_ACTION_FLOW, },
};
```

Signature: Require Real Packet

Aside from the scope of action of a signature, certain rule conditions will require that it matches against a *real packet* (as opposed to a *pseudo packet*). These rules are flagged with `SIG_MASK_REQUIRE_REAL_PKT` by the engine, and will have `real_pkt` listed as one of the rule's requirements. (See `engine-analysis` example output for the *Packet* rule type.)

A *pseudo packet* is an internal resource used by the engine when a flow is over but there is still data to be processed, such as when there is a flow timeout. A fake packet is then injected in the flow to finish up processing before ending it.

Those two types will be more documented soon (tracking #7424).

8.58.2 Signature Types and Variable-like Keywords

Keywords such as flow variables (`flowint`, `flowbits`), datasets, and similar ones can alter the rule type, if present in a signature.

That happens because the variable condition can change per packet. Thus, the Signature is categorized as a *packet* rule.

This affects rule types:

- Application Layer (`app_layer`)

- Protocol Detection Only (`pd_only`)
- Decoder Events Only (`de_only`)
- IP Only (`ip_only`)³
- Like IP Only (`like_ip_only`)³

The rule examples provided further cover some such cases, but the table below lists those keywords with more details:

Table 9: Variable-like Keywords

Keyword	Keyword Option	Rule Type change?
<code>flow</code>	<code>to_server</code> , <code>to_client</code>	no type changes ³
<code>flow</code>	<code>established</code> , <code>not_established</code>	to <i>packet</i>
<code>flowbits</code> , <code>xbits</code> , <code>hostbits</code>	<code>isset</code> , <code>isnotset</code>	to <i>packet</i>
<code>flowbits</code> , <code>xbits</code> , <code>hostbits</code>	<code>set</code> , <code>unset</code> , <code>toggle</code>	no type change
<code>flowint</code>	<code>isset</code> , <code>notset</code> , all operators	to <i>packet</i>
<code>flowint</code>	defining the variable; unsetting;	no type change
<code>iprep</code>	<code>isset</code> , <code>notset</code> , all operators	to <i>packet</i>

Note

IP Only and Like IP Only

(3) Unlike the other affected types, signatures that would otherwise be classified as `ip_only` or `like_ip_only` become Packet rules if the `flow` keyword is used, regardless of option.

Note

`dataset`, while may look similar to the keywords above, doesn't pertain to this list as it can only be used with sticky buffer keywords, thus being only available to Application Layer Transaction rules (`app_tx`), which are not affected by this.

Flowbits: `isset`

If a non-stateful rule (e.g. a `pkt` rule) checks if a `flowbit` is set (like in `flowbits:fb6,isset`) and the rule that sets that variable is a stateful one, such as an `app_tx` rule, the engine will set a flag to indicate that that rule is also stateful - without altering its signature type. This flag is currently `SIG_FLAG_INIT_STATE_MATCH` (cf. ticket #7483).

There is a work-in-progress to add information about this to the `engine-analysis` report (ticket #7456).

8.58.3 Signatures per Type

This section offers brief descriptions for each rule type, and illustrates what signatures of each type may look like. It is possible to learn the type of a signature, as well as other important information, by running Suricata in *engine analysis* mode.

For each rule type, there is also a sample of the Engine Analysis report for one or more of rule(s) shown.

Decoder Events Only

Signatures that inspect broken or invalid packets. They expose Suricata decoding events.

For more examples check <https://github.com/OISF/suricata/blob/main/rules/decoder-events.rules>.

Example

```
alert pkthdr any any -> any any (msg:"SURICATA IPv6 duplicated Hop-By-Hop Options extension header"; decode-event:ipv6.exthdr_dupl_hh; classtype:protocol-command-decode; sid:1101;)
```

```
drop pkthdr any any -> any any (msg:"SURICATA IPv4 invalid option length"; :example-rule-emphasis: `decode-event:ipv4.opt_invalid_len; classtype:protocol-command-decode; sid:2200005; rev:2;)
```

Engine-Analysis Report

```
{
  "raw": "alert pkthdr any any -> any any (msg:\"SURICATA IPv6 duplicated Hop-By-Hop Options extension header\"; decode-event:ipv6.exthdr_dupl_hh; classtype:protocol-command-decode; sid:1101;)",
  "id": 1101,
  "gid": 1,
  "rev": 0,
  "msg": "SURICATA IPv6 duplicated Hop-By-Hop Options extension header",
  "app_proto": "unknown",
  "requirements": [
    "engine_event"
  ],
  "type": "de_only",
  "flags": [
    "src_any",
    "dst_any",
    "sp_any",
    "dp_any",
    "toserver",
    "toclient"
  ],
  "pkt_engines": [
    {
      "name": "packet",
      "is_mpm": false
    }
  ],
  "frame_engines": [],
  "lists": {
    "packet": {
      "matches": [
        {
          "name": "decode-event"
        }
      ]
    }
  }
}
```

Packet

Rules that expose/ inspect information on a packet-level (for instance, the header). Certain flow keywords may also turn a rule into a pkt rule, if they require per-packet inspection (cf. *Signature Types and Variable-like Keywords*).

Examples

```
alert tcp-pkt any any -> any any (msg:"tcp-pkt, anchored content"; content:"abc"; startswith; sid:203;)
```

```
alert tcp any any -> any any (msg:"ttl"; ttl:123; sid:701;)
```

```
alert udp any any -> any any (msg:"UDP with flow direction"; flow:to_server; sid:1001;)
```

```
alert tcp any any -> any 443 (flow: to_server; flowbits:set,tls_error; sid:1604; msg:"Allow TLS error handling (outgoing packet) - non-stateful rule");
```

```
alert tcp-pkt any any -> any any (msg:"Flowbit isset"; flowbits:isset,fb6; flowbits:isset,fb7; sid:1919;)
```

Engine-Analysis Report

```
{
  "raw": "alert tcp-pkt any any -> any any (msg:\"tcp-pkt, anchored content\"; content:\n
  ↪"abc\"; startswith; sid:203;)",
  "id": 203,
  "gid": 1,
  "rev": 0,
  "msg": "tcp-pkt, anchored content",
  "app_proto": "unknown",
  "requirements": [
    "payload",
    "real_pkt"
  ],
  "type": "pkt",
  "flags": [
    "src_any",
    "dst_any",
    "sp_any",
    "dp_any",
    "need_packet",
    "toserver",
    "toclient",
    "prefilter"
  ],
  "pkt_engines": [
    {
      "name": "payload",
      "is_mpm": true
    }
  ],
  "frame_engines": [],
  "lists": {
    "payload": {
      "matches": [
        {
          "name": "content",

```

(continues on next page)

(continued from previous page)

```

        "content": {
          "pattern": "abc",
          "length": 3,
          "nocase": false,
          "negated": false,
          "starts_with": true,
          "ends_with": false,
          "is_mpm": true,
          "no_double_inspect": false,
          "depth": 3,
          "fast_pattern": false,
          "relative_next": false
        }
      }
    ]
  },
  "mpm": {
    "buffer": "payload",
    "pattern": "abc",
    "length": 3,
    "nocase": false,
    "negated": false,
    "starts_with": true,
    "ends_with": false,
    "is_mpm": true,
    "no_double_inspect": false,
    "depth": 3,
    "fast_pattern": false,
    "relative_next": false
  }
}

```

IP Only

The IP ONLY rule type is used when rules match only on source and destination IP addresses, and not on any other flow or content modifier.

Examples

```
alert tcp-stream any any -> any any (msg:"tcp-stream, no content"; sid:101;)
```

```
alert tcp-pkt [192.168.0.0/16,10.0.0.0/8,172.16.0.0/12] any -> any any (msg:"tcp-pkt, no content"; sid:201;)
```

```
alert ip any any -> any any (hostbits:set,myflow2; sid:1505;)
```

```
alert udp any any -> any any (msg:"UDP with flow direction"; sid:1601;)
```

Engine-Analysis Report

```

{
  "raw": "alert ip any any -> any any (hostbits:set,myflow2; sid:1505;)",
  "id": 1505,

```

(continues on next page)

(continued from previous page)

```

"gid": 1,
"rev": 0,
"app_proto": "unknown",
"requirements": [],
"type": "ip_only",
"flags": [
  "src_any",
  "dst_any",
  "sp_any",
  "dp_any",
  "toserver",
  "toclient"
],
"pkt_engines": [],
"frame_engines": [],
"lists": {
  "postmatch": {
    "matches": [
      {
        "name": "hostbits"
      }
    ]
  }
}
}

```

IP Only (contains negated address)

A rule that inspects IP only properties, but contains negated IP addresses.

IP Only signatures with negated addresses are *like* IP-only signatures, but currently handled differently due to limitations of the algorithm processing IP Only rules. Impactful differences from a user-perspective are listed on the Signature Types table.

Examples

```
alert tcp 192.168.0.0/16,10.0.0.0/8,172.16.0.0/12 any -> ![192.168.0.0/16,10.0.0.0/8,172.16.0.0/12] any (msg:"tcp, has negated IP address"; sid:304;)
```

```
alert tcp [10.0.0.0/8,!10.10.10.10] any -> [10.0.0.0/8,!10.10.10.10] any (msg:"tcp, has negated IP address"; sid:305;)
```

Engine-Analysis Report

```

{
  "raw": "alert tcp [10.0.0.0/8,!10.10.10.10] any -> [10.0.0.0/8,!10.10.10.10] any (msg:\
↪"tcp, has negated IP address\"; sid:305;)",
  "id": 305,
  "gid": 1,
  "rev": 0,
  "msg": "tcp, has negated IP address",
  "app_proto": "unknown",
  "requirements": [],
  "type": "like_ip_only",

```

(continues on next page)

(continued from previous page)

```

"flags": [
  "sp_any",
  "dp_any",
  "toserver",
  "toclient"
],
"pkt_engines": [],
"frame_engines": [],
"lists": {}
}

```

Protocol Detection Only

When a signature checks for the application layer protocol but there is no need for a per-packet inspection, protocol detection can be done with the `app-layer-protocol` keyword. Check the [keyword documentation](#) full for usage.

See *Protocol Detection Only* for a flowchart representing how the type is defined.

See *Application Layer Protocol* for a packet-based inspection.

Warning

Since Suricata 7, a Protocol Detection rule (that uses the `app-layer-protocol` keyword) is not internally classified the same as a rule simply matching on the application layer protocol on the `protocol` field.

Examples

```
alert tcp any any -> any any (msg:"tcp, pd negated"; app-layer-protocol:!http; sid:401;)
```

```
alert tcp any any -> any any (msg:"tcp, pd positive"; app-layer-protocol:http; sid:402;)
```

```
alert tcp any any -> any any (msg:"tcp, pd positive dns"; app-layer-protocol:dns; sid:403;)
```

```
alert tcp any any -> any any (msg:"tcp, pd positive, dns, flow:to_server"; app-layer-protocol:dns; flow:to_server; sid:405;)
```

Engine-Analysis Report

```

{
  "raw": "alert tcp any any -> any any (msg:\"tcp, pd positive dns\"; app-layer-
  ↳protocol:dns; sid:403;)",
  "id": 403,
  "gid": 1,
  "rev": 0,
  "msg": "tcp, pd positive dns",
  "app_proto": "unknown",
  "requirements": [],
  "type": "pd_only",
  "flags": [
    "src_any",
    "dst_any",
    "sp_any",
    "dp_any",

```

(continues on next page)

(continued from previous page)

```

    "toserver",
    "toclient"
  ],
  "pkt_engines": [
    {
      "name": "packet",
      "is_mpm": false
    }
  ],
  "frame_engines": [],
  "lists": {
    "packet": {
      "matches": [
        {
          "name": "app-layer-protocol"
        }
      ]
    }
  }
}

```

Packet-Stream

A rule is categorized as such when it inspects on traffic in specific portions of the packet payload, using `content` buffer with the `startswith` or `depth` keywords.

Examples

```
alert tcp any any -> any any (msg:"tcp, anchored content"; content:"abc"; startswith; sid:303;)
```

```
alert http any any -> any any (msg:"http, anchored content"; content:"abc"; depth:30; sid:603;)
```

Engine-Analysis Report

```

{
  "raw": "alert http any any -> any any (msg:\"http, anchored content\"; content:\"abc\";
  ↳ depth:30; sid:603;)",
  "id": 603,
  "gid": 1,
  "rev": 0,
  "msg": "http, anchored content",
  "app_proto": "http_any",
  "requirements": [
    "payload",
    "flow"
  ],
  "type": "pkt_stream",
  "flags": [
    "src_any",
    "dst_any",
    "sp_any",
    "dp_any",

```

(continues on next page)

(continued from previous page)

```
"applayer",
"need_packet",
"need_stream",
"toserver",
"toclient",
"prefilter"
],
"pkt_engines": [
{
  "name": "payload",
  "is_mpm": true
}
],
"frame_engines": [],
"lists": {
  "payload": {
    "matches": [
      {
        "name": "content",
        "content": {
          "pattern": "abc",
          "length": 3,
          "nocase": false,
          "negated": false,
          "starts_with": false,
          "ends_with": false,
          "is_mpm": true,
          "no_double_inspect": false,
          "depth": 30,
          "fast_pattern": false,
          "relative_next": false
        }
      }
    ]
  }
},
"mpm": {
  "buffer": "payload",
  "pattern": "abc",
  "length": 3,
  "nocase": false,
  "negated": false,
  "starts_with": false,
  "ends_with": false,
  "is_mpm": true,
  "no_double_inspect": false,
  "depth": 30,
  "fast_pattern": false,
  "relative_next": false
}
```

Stream

A rule that matches payload traffic without regards to its position, that is, on an unanchored content buffer, uses byte extraction or matches on tcp-stream is classified a stream rule.

Examples

```
alert tcp-stream any any -> any any (msg:"tcp-stream, simple content"; content:"abc"; sid:102;)
```

```
alert http any any -> any any (msg:"http, simple content"; content:"abc"; sid:602;)
```

```
alert tcp any any -> any 443 (flow: to_server; content:"abc"; flowbits:set,tls_error; sid:1605; msg:"Allow TLS error handling (outgoing packet) with simple content - Stream rule");
```

```
alert tcp any any -> any 443 (flow: to_server; content:"abc"; sid:160401; msg:"Allow TLS error handling (outgoing packet) - stream rule");
```

```
alert tcp any any -> any 443 (content:"abc"; sid:160402; msg:"Allow TLS error handling (outgoing packet) - stream rule");
```

```
alert tcp any any -> any any (msg:"byte_extract with dce"; byte_extract:4,0,var,dce; byte_test:4,>,var,4,little; sid:901;)
```

Engine-Analysis Report

```
{
  "raw": "alert tcp any any -> any any (msg:\"byte_extract with dce\"; byte_extract:4,0,
  ↪var,dce; byte_test:4,>,var,4,little; sid:901;)",
  "id": 901,
  "gid": 1,
  "rev": 0,
  "msg": "byte_extract with dce",
  "app_proto": "dcerpc",
  "requirements": [
    "payload",
    "flow"
  ],
  "type": "stream",
  "flags": [
    "src_any",
    "dst_any",
    "sp_any",
    "dp_any",
    "applayer",
    "need_stream",
    "toserver",
    "toclient"
  ],
  "pkt_engines": [
    {
      "name": "payload",
      "is_mpm": false
    }
  ],
  "frame_engines": [],
  "lists": {
    "payload": {
```

(continues on next page)

(continued from previous page)

```

    "matches": [
      {
        "name": "byte_extract"
      },
      {
        "name": "byte_test",
        "byte_test": {
          "nbytes": 4,
          "offset": 4,
          "base": "unset",
          "flags": [
            "little_endian"
          ]
        }
      }
    ]
  }
}

```

Application Layer Protocol

For a packet-based inspection of the application layer protocol, a rule should use the `protocol` field for the matches.

Warning

Since Suricata 7, a simple rule matching traffic on the `protocol` field is not internally classified the same as a rule using the `app-layer-protocol` keyword).

Warning

As per Suricata 7, if `flow:established` or `flow:not_established` is added to a base Application Layer Protocol rule, that signature will become a *Packet* rule.

Examples

```
alert dns any any -> any any (msg:"app-layer, dns"; sid:404;)
```

```
alert http any any -> any any (msg:"http, no content"; sid:601;)
```

```
alert tls any any -> any any (msg:"tls, pkt or app-layer?"; flowint:tls_error_int,=,0; sid:613;)
```

Engine-Analysis Report

```

{
  "raw": "alert dns any any -> any any (msg:\"app-layer, dns\"; sid:404;)",
  "id": 404,
  "gid": 1,
  "rev": 0,
  "msg": "app-layer, dns",

```

(continues on next page)

(continued from previous page)

```

"app_proto": "dns",
"requirements": [
  "flow"
],
"type": "app_layer",
"flags": [
  "src_any",
  "dst_any",
  "sp_any",
  "dp_any",
  "applayer",
  "toserver",
  "toclient"
],
"pkt_engines": [],
"frame_engines": [],
"lists": {}
}

```

Application Layer Protocol Transactions

Rules inspecting traffic using keywords related to application layer protocols are classified with this signature type. This also includes *frame* keywords.

Examples

```
alert tcp any any -> any any (msg:"http, pos event"; app-layer-event:http.file_name_too_long; sid:501;)
```

```
alert http any any -> any any (msg:"Test"; flow:established,to_server; http.method; content:"GET"; http.uri; content:".exe"; endswith; http.host; content:"!"; google.com"; endswith; sid:1102;)
```

```
alert udp any any -> any any (msg:"DNS UDP Frame"; flow:to_server; frame:dns.pdu; content:"|01 20 00 01|"; offset:2; content:"suricata"; offset:13; sid:1402; rev:1;)
```

```
alert tcp any any -> any any (msg:"byte_extract with dce"; dcerpc.stub_data; content:"abc"; byte_extract:4,0,var,relative; byte_test:4,>,var,4,little; sid:902;)
```

Engine-Analysis Report

```

{
  "raw": "alert tcp any any -> any any (msg:\"byte_extract with dce\"; dcerpc.stub_data; \u2192content:\"abc\"; byte_extract:4,0,var,relative; byte_test:4,>,var,4,little; sid:902;)",
  "id": 902,
  "gid": 1,
  "rev": 0,
  "msg": "byte_extract with dce",
  "app_proto": "dcerpc",
  "requirements": [
    "flow"
  ],
  "type": "app_tx",
  "flags": [
    "src_any",

```

(continues on next page)

(continued from previous page)

```

    "dst_any",
    "sp_any",
    "dp_any",
    "applayer",
    "toserver",
    "toclient",
    "prefilter"
  ],
  "pkt_engines": [],
  "frame_engines": [],
  "engines": [
    {
      "name": "dce_stub_data",
      "direction": "toclient",
      "is_mpm": true,
      "app_proto": "dcerpc",
      "progress": 0,
      "matches": [
        {
          "name": "content",
          "content": {
            "pattern": "abc",
            "length": 3,
            "nocase": false,
            "negated": false,
            "starts_with": false,
            "ends_with": false,
            "is_mpm": true,
            "no_double_inspect": false,
            "fast_pattern": false,
            "relative_next": true
          }
        },
        {
          "name": "byte_extract"
        },
        {
          "name": "byte_test",
          "byte_test": {
            "nbytes": 4,
            "offset": 4,
            "base": "unset",
            "flags": [
              "little_endian"
            ]
          }
        }
      ]
    },
    {
      "name": "dce_stub_data",
      "direction": "toserver",

```

(continues on next page)

(continued from previous page)

```

"is_mpm": true,
"app_proto": "dcerpc",
"progress": 0,
"matches": [
  {
    "name": "content",
    "content": {
      "pattern": "abc",
      "length": 3,
      "nocase": false,
      "negated": false,
      "starts_with": false,
      "ends_with": false,
      "is_mpm": true,
      "no_double_inspect": false,
      "fast_pattern": false,
      "relative_next": true
    }
  },
  {
    "name": "byte_extract"
  },
  {
    "name": "byte_test",
    "byte_test": {
      "nbytes": 4,
      "offset": 4,
      "base": "unset",
      "flags": [
        "little_endian"
      ]
    }
  }
]
},
{
  "name": "dce_stub_data",
  "direction": "toclient",
  "is_mpm": true,
  "app_proto": "smb",
  "progress": 0,
  "matches": [
    {
      "name": "content",
      "content": {
        "pattern": "abc",
        "length": 3,
        "nocase": false,
        "negated": false,
        "starts_with": false,
        "ends_with": false,
        "is_mpm": true,

```

(continues on next page)

(continued from previous page)

```

        "no_double_inspect": false,
        "fast_pattern": false,
        "relative_next": true
    }
},
{
    "name": "byte_extract"
},
{
    "name": "byte_test",
    "byte_test": {
        "nbytes": 4,
        "offset": 4,
        "base": "unset",
        "flags": [
            "little_endian"
        ]
    }
}
]
},
{
    "name": "dce_stub_data",
    "direction": "toserver",
    "is_mpm": true,
    "app_proto": "smb",
    "progress": 0,
    "matches": [
        {
            "name": "content",
            "content": {
                "pattern": "abc",
                "length": 3,
                "nocase": false,
                "negated": false,
                "starts_with": false,
                "ends_with": false,
                "is_mpm": true,
                "no_double_inspect": false,
                "fast_pattern": false,
                "relative_next": true
            }
        }
    ],
    {
        "name": "byte_extract"
    },
    {
        "name": "byte_test",
        "byte_test": {
            "nbytes": 4,
            "offset": 4,
            "base": "unset",

```

(continues on next page)

(continued from previous page)

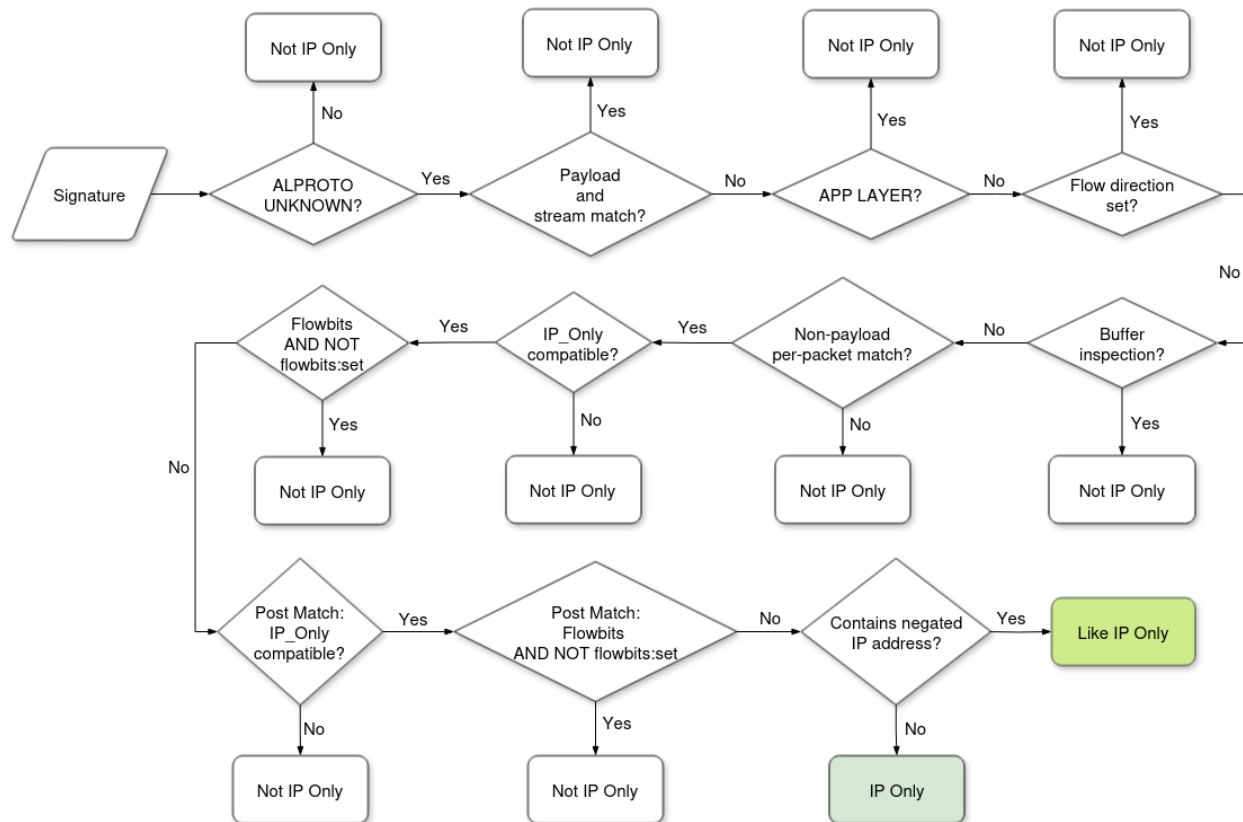
```
        "flags": [
          "little_endian"
        ]
      }
    ]
  },
  "lists": {},
  "mpm": {
    "buffer": "dce_stub_data",
    "pattern": "abc",
    "length": 3,
    "nocase": false,
    "negated": false,
    "starts_with": false,
    "ends_with": false,
    "is_mpm": true,
    "no_double_inspect": false,
    "fast_pattern": false,
    "relative_next": true
  }
}
```

8.58.4 Detailed Flowcharts

A look into the illustrated overall representation of functions or paths that determine signature types.

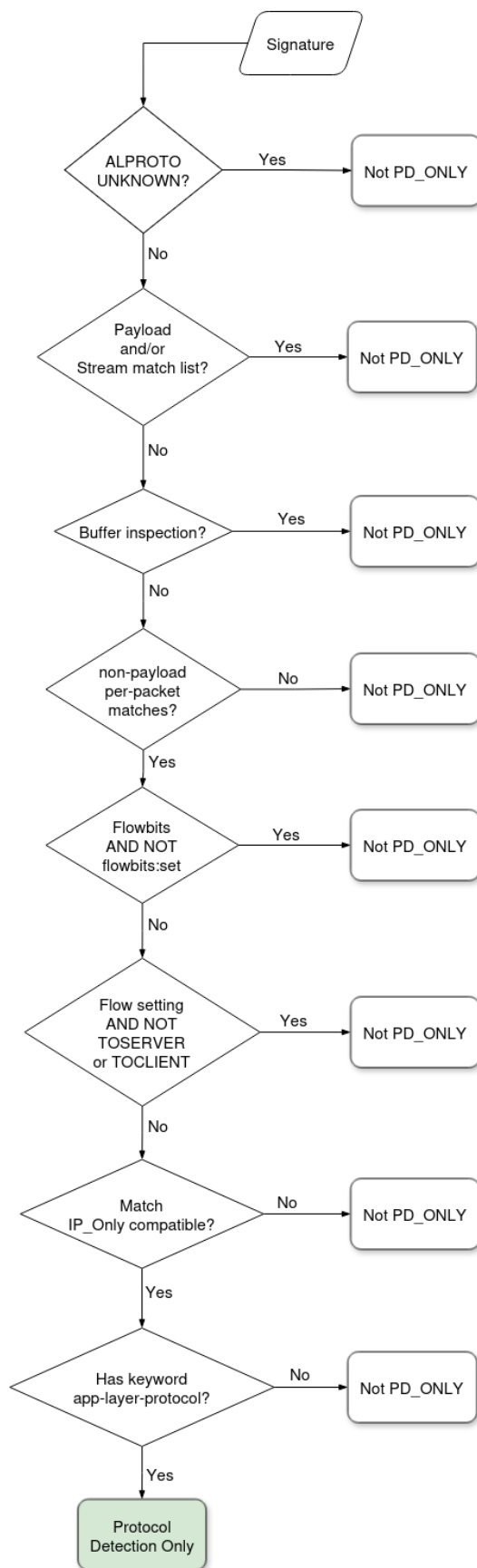
IP Only and IP Only with negated addresses

ip_only and like_ip_only flows.



Protocol Detection Only

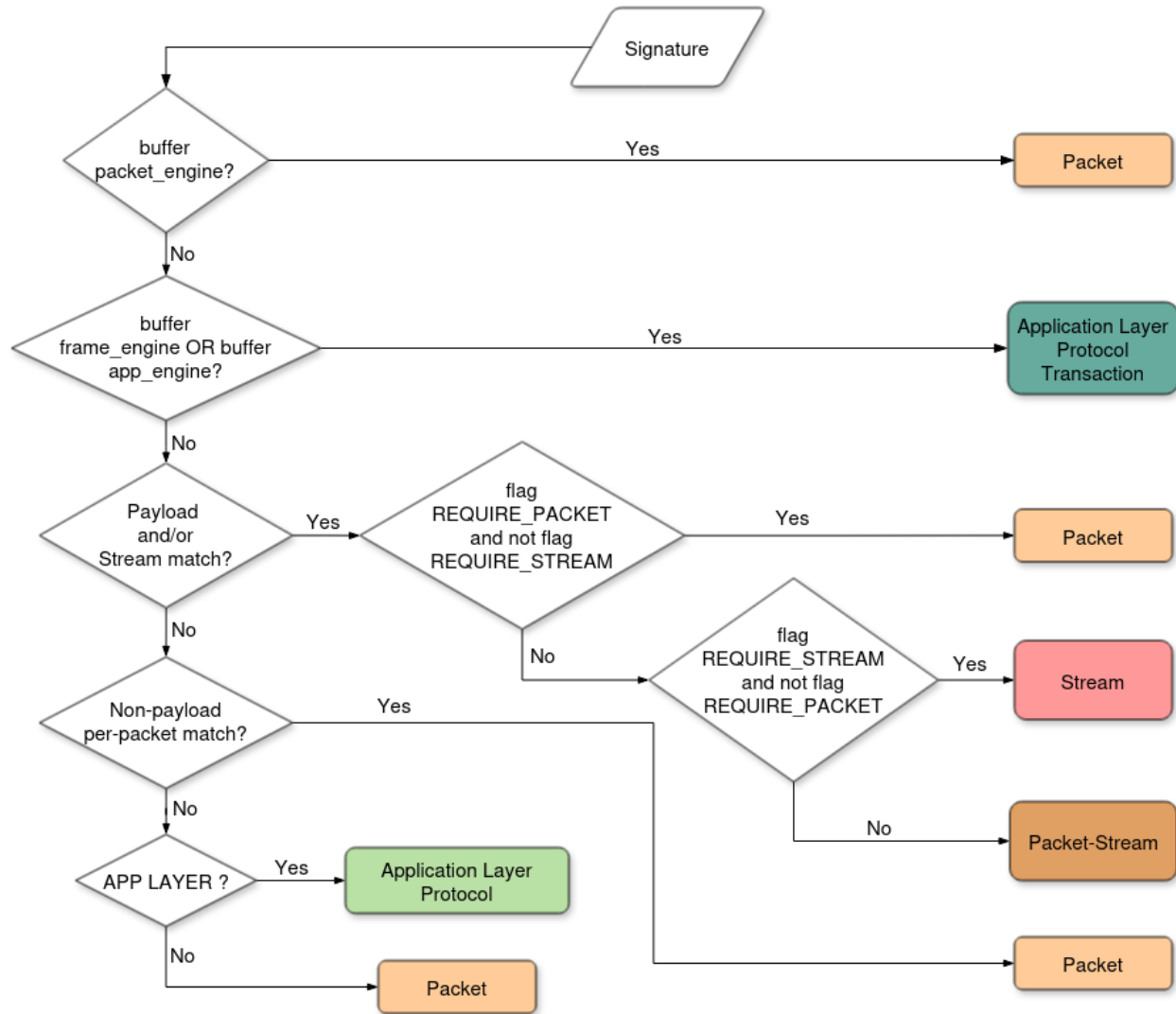
pd_only flow.



Application Layer Protocol, Transaction, Packet, Stream and Stream-Packet rules

app_layer, app_tx, pkt, stream and stream-pkt flows.

REQUIRE_PACKET and REQUIRE_STREAM can be seen as flags need_packet and need_stream in the engine-analysis output.



8.59 Rule Processing

Suricata rules have many elements that influence how they are processed by Suricata and matched against network traffic.

This section explains some key aspects of how Suricata handles rules internally, so it can be easier to understand/predict how different rules may interact in specific scenarios.

This material is intended for: rule writers; developers.

Possible questions one should be better equipped to answer after reading this document:

- What happens if two rules have the same priority value (the keyword)?
- What type of rules will be evaluated first, given a set of rules?

- How does Suricata decide what rule is "more important" when matching traffic?

Important

Rules processing is also heavily affected by rule types, as mentioned in this chapter. You may want to read more on [Rule Types and Categorization](#).

Note

Throughout this documentation and for Suricata, the terms "rule" and "signature" are mainly interchangeable, unless context indicates otherwise.

8.59.1 Overview

Rules are provided to Suricata via rules files. Starting from those, the Detection Engine loader will:

1. Load all the signatures;
2. Check for validity (non-existing keywords; duplicated sid etc);
3. Report stats on loaded, good and bad signatures;
4. Sort all valid signatures and store them in a list in the Detect Engine Context (DetectEngineCtx) structure, taking into consideration several rule aspects according to their order of relevance to the Detection Engine;
5. Attribute internal rule IDs that reflect this [rule prioritization](#);
6. During inspection, match rules against the inspected traffic, according to rule and traffic type.

8.59.2 Rule Prioritization

Suricata registers several different ordering functions (with `SCSigRegisterSignatureOrderingFuncs()`), which are then used to compare the rules, sort them, and define their priority. The elements taken into consideration for such are the signature's:

1. *Action*
2. Usage of *flowbits*
3. Usage of *flowint*
4. Usage of *flowvar*
5. Usage of *pktvar*
6. Usage of *hostbits*
7. Usage of *ippair*
8. *"Priority" keyword*

In this order. Once signatures are ordered, they are attributed a unique internal ID (`Signature::iid`) which symbolizes their priority (the lower the `iid`, the higher the priority). This could mean that a rule with a keyword-defined priority of 1 could have lower priority than another rule that had flowbits set and a rule action with higher priority, for instance.

Note

this list isn't fully comprehensive, in the sense that each item has extra logic for prioritization. For example, considering flowbits, the priority is write (highest) > write + read > read (lowest) > no flowbits.

Another important element when considering rule parsing, processing and matching is that the ruleset is optimized into signature group heads based on the signature elements (thus, for instance, a TCP rule and an UDP rule would be loaded into different groups, and their internal ids will not interfere between one another, as they're matched against different traffic). For more on this, see [Detection engine](#).

8.59.3 Inspection Process

Once it is time to inspect network traffic against the loaded rules, the Detect Engine will match against - if applicable:

1. IP Only rules;
2. Packet/payload-related rules;
3. Frame keywords;
4. Application layer protocol transaction rules.

During packet inspection, if the signature uses the last two in this list, inspection is left to those steps.

Tip

With the introduction of [Firewall mode](#), it is possible to explicitly control to which step of the detection engine flow a rule will be hooked. This is done with [Explicit rule hooks](#).

For each rule that is matched, a `PacketAlert` is created. After all matches for a packet have been processed, and the [alert queue limit](#) is taken into account, the remaining `PacketAlerts` become the alerts in Suricata logs.

8.59.4 Considerations on Inspection Steps

IP Only rules

Without optimization, IP Only signatures would match on every packet on a flow. To improve performance, what Suricata does is to evaluate rules that are `ip-only` only once per flow direction, for the first packet in each direction.

Application layer protocol transactions

Each parser has its own state machine, and uses a per-direction parsing state "progress". Keywords can be registered for each progress value. So, for instance, `http` has a value "request line available" for which there are keywords like [http.uri](#), [http.method](#) etc. registered. While parsing the traffic, if the engine reaches this state, the signatures with those keywords may be already evaluated, even if they have a lower priority than an `http` body inspecting signature.

Relatedly, a rule with two keywords matching at two different progress stages may be evaluated against two different packets.

8.59.5 Implications

Action precedence and interaction with `ip-only` rules

To illustrate what may be counter-intuitive implications of how inspection steps, action prioritization and rule keywords interact and affect the engine behavior, we will use a real case example. Consider these three rules:

```
pass tcp 0.0.0.0/0 any <> 0.0.0.0/0 443 (msg:"Allow TCP in port 443"; flow: not_established; sid:1; rev:1;)
```

```
pass tcp 0.0.0.0/0 any <> 0.0.0.0/0 80 (msg:"Allow TCP in port 80"; flow: not_established; sid:2; rev:1;)
```

```
drop ip 0.0.0.0/0 any -> 0.0.0.0/0 any (msg:"No outbound internet access from host"; sid:3; rev:1;)
```

The first two are signatures that analyze individual packets and match only if the flow has not been established (`flow: not_established`): the rules grant PASS to the matched packet - but not to its flow.

The third signature is considered `ip-only`. This means it will be evaluated for the *first* packet in both directions of a flow, in addition to rules 1 and 2. By extension, **the other packets in the same flow will not be evaluated against this rule.**

With an action order configuration that prioritizes PASS over DROP, this means that rules 1 and 2 will have a higher internal priority over rule 3, therefore nullifying the DROP outcome. The result: a flow for outbound internet traffic from the host, expected to be dropped, wouldn't be.

If the expected behavior with those three signatures was to allow traffic on ports 80 and 443 only, while dropping everything else, the simplest way to achieve this would be to remove the `flow: not_established` portion from rules `sid:1` and `sid:2`. This ensures that the PASS action would be applied to the whole flow following the match on the first packet and that all other traffic would be dropped.

Following that, all three rules will be evaluated on the same step, and if a flow isn't flagged with `pass`, it will be dropped with the third rule.

Tip

A more straightforward way to achieve that in Suricata 8 is using the firewall more. See *Firewall Mode Design*.

Alerts not seen

Another aspect of rule prioritization combined with the alerts queue size is that, in corner case scenarios, if a packet matches against too many rules, signatures with lower priority could be discarded from the `PacketAlert` queue (see the section on *alert queue overflow impact* for more).

The stats counter `detect.alert_queue_overflow` will be higher than zero if an alert was discarded due to `Alert Queue` overflow (cf. *Discarded and Suppressed Alerts Stats*).

RULE MANAGEMENT

9.1 Rule Management with Suricata-Update

While it is possible to download and install rules manually, it is recommended to use a management tool for this. `suricata-update` is the official way to update and manage rules for Suricata.

`suricata-update` is bundled with Suricata and is normally installed with it. For instructions on installing manually, see <http://suricata-update.readthedocs.io/en/latest/quickstart.html#install-suricata-update>

Note

`suricata-update` is bundled with Suricata version 4.1 and later. It can be used with older versions as well. It will have to be installed separately in that case.

To download the Emerging Threats Open ruleset, it is enough to simply run:

```
sudo suricata-update
```

This will download the ruleset into `/var/lib/suricata/rules/`

Suricata's configuration will have to be updated to have a rules config like this:

```
default-rule-path: /var/lib/suricata/rules
rule-files:
- suricata.rules
```

Now (re)start Suricata.

9.1.1 Updating your rules

To update the rules, simply run

```
sudo suricata-update
```

It is recommended to update your rules frequently.

9.1.2 Using other rulesets

Suricata-Update is capable of making other rulesets accessible as well.

To see what is available, fetch the master index from the OISF hosts:

```
sudo suricata-update update-sources
```

Then have a look at what is available:

```
sudo suricata-update list-sources
```

This will give a result similar to

```
Name: oisf/trafficid
  Vendor: OISF
  Summary: Suricata Traffic ID ruleset
  License: MIT
Name: ptresearch/attackdetection
  Vendor: Positive Technologies
  Summary: Positive Technologies Attack Detection Team ruleset
  License: Custom
Name: sslbl/ssl-fp-blacklist
  Vendor: Abuse.ch
  Summary: Abuse.ch SSL Blacklist
  License: Non-Commercial
Name: et/open
  Vendor: Proofpoint
  Summary: Emerging Threats Open Ruleset
  License: MIT
Name: scwx/security
  Vendor: Secureworks
  Summary: Secureworks suricata-security ruleset.
  License: Commercial
  Parameters: secret-code
  Subscription: https://www.secureworks.com/contact/ (Please reference CTU Countermeasures)
Name: scwx/malware
  Vendor: Secureworks
  Summary: Secureworks suricata-malware ruleset.
  License: Commercial
  Parameters: secret-code
  Subscription: https://www.secureworks.com/contact/ (Please reference CTU Countermeasures)
Name: et/pro
  Vendor: Proofpoint
  Summary: Emerging Threats Pro Ruleset
  License: Commercial
  Replaces: et/open
  Parameters: secret-code
  Subscription: https://www.proofpoint.com/us/threat-insight/et-pro-ruleset
```

Each of the rulesets has a name that has a 'vendor' prefix, followed by a set name. For example, OISF's traffic id ruleset is called 'oisf/trafficid'.

To enable 'oisf/trafficid', enter:

```
sudo suricata-update enable-source oisf/trafficid
sudo suricata-update
```

Now restart Suricata again and the rules from the OISF TrafficID ruleset are loaded.

To see which rulesets are currently active, use "list-enabled-sources".

9.1.3 Controlling which rules are used

By default suricata-update will merge all rules into a single file "/var/lib/suricata/rules/suricata.rules".

To enable rules that are disabled by default, use */etc/suricata/enable.conf*

```
2019401          # enable signature with this sid
group:emerging-icmp.rules # enable this rulefile
re:trojan        # enable all rules with this string
```

Similarly, to disable rules use `/etc/suricata/disable.conf`:

```
2019401          # disable signature with this sid
group:emerging-info.rules # disable this rulefile
re:heartbleed     # disable all rules with this string
```

After updating these files, rerun `suricata-update` again:

```
sudo suricata-update
```

Finally restart Suricata.

9.1.4 Further reading

See <https://suricata-update.readthedocs.io/en/latest/>

9.2 Adding Your Own Rules

If you would like to create a rule yourself and use it with Suricata, this guide might be helpful.

Start creating a file for your rule. Use one of the following examples in your console/terminal window:

```
sudo nano local.rules
sudo vim local.rules
```

Write your rule, see [Rules Format](#) and save it.

Update the Suricata configuration file so your rule is included. Use one of the following examples:

```
sudo nano /etc/suricata/suricata.yaml
sudo vim /etc/suricata/suricata.yaml
```

and make sure your `local.rules` file is added to the list of rules:

```
default-rule-path: /usr/local/etc/suricata/rules

rule-files:
- suricata.rules
- /path/to/local.rules
```

Now, run Suricata and see if your rule is being loaded.

```
suricata -c /etc/suricata/suricata.yaml -i wlan0
```

If the rule failed to load, Suricata will display as much information as it has when it deemed the rule un-loadable. Pay special attention to the details: look for mistakes in special characters, spaces, capital characters, etc.

Next, check if your log-files are enabled in the Suricata configuration file `suricata.yaml`.

If you had to correct your rule and/or modify Suricata's YAML configuration file, you'll have to restart Suricata.

If you see your rule is successfully loaded, you can double check your rule by doing something that should trigger it.

By default, Suricata will log alerts to two places

- `eve.json`
- `fast.log`

These files will be located in the log output directory which is set by one of two methods:

1. Suricata configuration file: see `default-log-dir` for the name of the directory
2. Suricata command line: Using `-l /path/to/log-dir` creates log files in the named directory.

The following example assumes that the log directory is named `/var/log/suricata`

```
tail -f /var/log/suricata/fast.log
```

If you would make a rule like this:

```
alert http any any -> any any (msg:"Do not read gossip during work";  
content:"Scarlett"; nocase; classtype:policy-violation; sid:1; rev:1;)
```

Your alert should look like this:

```
09/15/2011-16:50:27.725288  [**] [1:1:1] Do not read gossip during work [**]  
[Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 192.168.0.  
->32:55604 -> 68.67.185.210:80
```

9.3 Rule Reloads

Suricata was designed to reload rules while it is actively processing network traffic to minimize service disruption.

Suricata must be administratively directed to reload rules while it is running.

It is also possible to get information about the last reload via dedicated commands. See *Commands in standard running mode* for more information.

9.3.1 Reload Triggers

There are multiple ways to trigger a rule reload. `suricatasc` is a program distributed with Suricata that provides client-side services, including the ability to trigger a Suricata rule reload..

Via process signal

The `USR2` signal will cause Suricata to start a rule reload. The signal can be sent from the command line or from a script/program. Escalation of privileges may be necessary to send the signal.

```
$ kill -USR2 $(pidof suricata)
```

Via the UNIX domain socket

The `suricatasc` program has two commands to initiate a Suricata rule reload.

Blocking reload

This will cause Suricata to reload rules while the caller blocks, or waits.

```
suricatasc -c reload-rules
```

Non-blocking reload

This will cause Suricata to reload rules without the caller blocking or waiting.

```
suricatasc -c ruleset-reload-nonblocking
```

9.3.2 Resources Reloaded

There are two types of resources that are reloaded during a rule reload.

- Rule-related configuration:
 - Suricata's configuration file(s): `suricata.yaml` and any specified with the command-line options `--include <config-file.yaml>`. Only rule-related information is reloaded.
 - * Rule variables: items in the `vars` section.
 - * Rule files from the `rule-files` section (if the `-S` command line option was not used)
 - Ancillary rule-related configuration files: `classification.config`, `reference.config` and `threshold.config`
 - Dataset(s) used by rules.
 - When multi-tenants are configured, rule-related configuration information for each tenant.

9.3.3 When to reload rules

Rule reloads are used in situations when:

- Rules have been changed since the last reload. Vendors often add rules frequently and sometimes update existing rules. Rules should be reloaded according to a security policy that includes Suricata rule and configuration settings.
- Rule variables have been changed. Rule reloads will use rule variables from the Suricata configuration file. When updating these, reload the rules in order for the updated rule variables to take effect.
- Ancillary rule-related configuration files are updated.

9.3.4 Advanced: Rule Reload Steps

When reloading rules, Suricata executes the following steps to ensure a safe and consistent update:

- The main Suricata configuration is reloaded to update rule variables and values, including the rule related files `classification.config`, `reference.config` and `theshold.config`.
- All rule files are reloaded with new rule variables applied.
- A new detection engine is created for the updated rules.
- The previous and newly created detection engines are swapped.
- Ensure all threads are updated.
- Free old detection engine and associated resources.

Suricata will continue to process packets during the update process. Note that additional system memory is used during the reload process as a new detection engine and the reloaded rules are associated with it.

9.4 Rules Profiling

If Suricata is built with the `--enable-profiling-rules` then the ruleset profiling can be activated on demand from the unix socket and dumped from it.

To start profiling

```
suricatasc -c ruleset-profile-start
```

To stop profiling

```
suricatasc -c ruleset-profile-stop
```

To dump profiling

```
suricatasc -c ruleset-profile
```

A typical scenario to get rules performance would be

```
suricatasc -c ruleset-profile-start  
sleep 30  
suricatasc -c ruleset-profile-stop  
suricatasc -c ruleset-profile
```

On busy systems, using the sampling capability to capture performance on a subset of packets can be obtained via the `sample-rate` variable in the `profiling` section in the `suricata.yaml` file.